

SAPIENZA
UNIVERSITÀ DI ROMA

Combinatorial Optimization

Volume 3 — Dynamic programming

Chapters 14–17 of the module «Combinatorial Optimization»: dynamic programming. Definitions, results with proofs and worked examples; exercises with complete solutions are in Exercises 3.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/combinatorial-optimization/
github.com/fabiofurini/combinatorial-optimization

Academic year 2026–2027

Contents

14 Principles of dynamic programming **2**

14.1 Why not enumerate all the solutions? . . . 3
14.2 Divide, conquer and combine . . . 3
14.3 Problem properties required by DP algorithms . . . 4
14.4 Steps for designing a DP algorithm . . . 5
14.5 The DP algorithms of this module . . . 5
14.6 Typical mistakes . . . 6
14.7 In summary . . . 6

15 Dynamic programming for knapsack problems **7**

15.1 A DP algorithm for the Knapsack Problem . . . 7
15.2 A DP algorithm for the Unbounded Knapsack Problem . . . 59
15.3 A DP algorithm for the Bounded Knapsack Problem . . . 83

16 Rod cutting and coin change **97**

16.1 A DP algorithm for the Rod Cutting Problem . . . 97
16.2 A DP algorithm for the Coin Change Problem . . . 122

17 Dynamic programming for lot sizing **134**

17.1 Problem definition . . . 135
17.2 A DP algorithm for the ULS . . . 140
17.3 A DP algorithm for the capacitated case . . . 153
17.4 Typical mistakes . . . 166
17.5 In summary . . . 167

Chapter 14

Principles of dynamic programming

Prerequisites

- From this module, the chapter «Combinatorial optimization problems»: the combinatorial optimization problems, their feasible solutions, and the fact that their number grows exponentially.
- From this module, the chapter «Knapsack problems»: the KP, used as the running example of the following chapters.
- From the module *Algorithms*: pseudocode, recursion, and the growth of the running time of an algorithm.

Objectives

At the end of this chapter you will be able to:

- explain why enumerating all the feasible solutions fails, and what a DP algorithm does instead;
- state the divide, conquer and combine principles, and the difference between independent subproblems and overlapping subproblems;
- check the optimal substructure and the overlapping subproblems on a problem;
- list the steps of a DP algorithm: recurrence, base cases, lookup table, and reconstruction of an optimal solution.

14.1 Why not enumerate all the solutions?

- A brute-force algorithm lists all the feasible solutions of a combinatorial optimization problem; their number grows exponentially (for the KP there are 2^n subsets of items). The table shows how the running time of an algorithm with $\mathcal{O}(n)$, $\mathcal{O}(n^2)$ and $\mathcal{O}(2^n)$ steps grows with the size n of the instance, at 1 μs per step.

	Size n					
Algorithm	10	20	30	40	50	60
$\mathcal{O}(n)$	.00001 seconds	.00002 seconds	.00003 seconds	.00004 seconds	.00005 seconds	.00006 seconds
$\mathcal{O}(n^2)$	.0001 seconds	.0004 seconds	.0009 seconds	.0016 seconds	.0025 seconds	.0036 seconds
$\mathcal{O}(2^n)$	.001 seconds	1.0 second	17.9 minutes	12.7 days	35.7 years	366 centuries

- Technology will not help.[1] With a CPU 1000 times faster, in one hour an $\mathcal{O}(n)$ algorithm solves an instance 1000 times larger, while an $\mathcal{O}(2^n)$ algorithm solves an instance only 10 units larger.

Brute-force algorithms do not work: a DP algorithm avoids the enumeration by reusing the optimal values of the subproblems.

14.2 Divide, conquer and combine

- A ***Dynamic Programming (DP) algorithm*** is an exact algorithm to find an optimal solution of a class of optimization problems with specific properties (see the next section).
- It is based on decomposing the original problem into smaller subproblems and reusing the information computed when solving them.
- A DP algorithm exploits the ***divide, conquer, and combine paradigm***, whose main principles are:
  1. **Dividing principle** $\rightsquigarrow$ decompose the problem into a number of subproblems that are smaller problems of the same type;
  2. **Conquering principle** $\rightsquigarrow$ solve the subproblems in a recursive manner. If the subproblem sizes are small enough just solve the subproblems in a straightforward manner;
  3. **Combining principle** $\rightsquigarrow$ combine optimal solutions of the subproblems to obtain an optimal solution for the original problem.

[1] M. Garey, D. Johnson, *Computers and Intractability: A Guide to NP-Completeness.*

- DP algorithms divide the problem into ***independent subproblems***, solve them recursively, and then combine their solutions to solve the original problem. Here, independent means that each subproblem generated by the division can be solved without affecting the others.

- In a DP algorithm, the same subproblem may be generated several times during the recursive solution process. In order to avoid redundant computations, the optimal value of each subproblem is stored in a ***lookup table***. Whenever the same subproblem is generated again, its optimal value is retrieved from the lookup table instead of being recomputed.

14.3 Problem properties required by DP algorithms

- DP algorithms work only for combinatorial optimization problems that allow a recursive decomposition process respecting the following properties:

  1. ***Optimal Substructure*** $\rightsquigarrow$ the idea is that, when a problem is decomposed into smaller subproblems, an optimal solution of the problem can be built from optimal solutions of the subproblems.

     In practice, to verify this property one usually checks the following points:

     - the problem can be decomposed into smaller subproblems of the same type;
     - subproblems can be solved independently;
     - optimal solutions of the subproblems can be combined to obtain an optimal solution of the original problem;
     - a **recurrence relation** to compute the optimal value of the problem can be established.

  2. ***Overlapping Subproblems*** $\rightsquigarrow$ the idea is that, when a problem is recursively decomposed into smaller subproblems, the same subproblems may be generated several times.

     In practice, to verify this property one usually checks the following points:

     - the recursive decomposition process generates the same subproblem more than once;
     - it is possible to store the optimal values of the subproblems in a lookup table; then whenever the same subproblem is required again, its optimal value can be retrieved from the lookup table instead of being recomputed.

DP algorithms can be used when a problem has both the Optimal Substructure and Overlapping Subproblems.

14.4 Steps for designing a DP algorithm

- Identify the ***recursive decomposition process*** that, starting from a given problem, generates its subproblems, which are smaller problems of the same type, and verify the following two properties:
  1. the Optimal Substructure property, by checking that an optimal solution of the problem can be obtained by combining the optimal solutions of its subproblems;
  2. the Overlapping Subproblems property, by checking whether the recursive decomposition process generates the same subproblems several times;
- The construction of a DP algorithm usually follows two main steps:
  1. computation of the optimal value
     - define a ***recurrence relation*** that expresses the optimal value of a problem in terms of the optimal values of its subproblems;
     - specify the ***base cases***, i.e., the subproblems whose optimal solution is immediate (the optimal value then follows directly from it);
     - determine a DP algorithm that computes the optimal values of all relevant subproblems, typically in a ***bottom-up fashion***, storing each value in a lookup table so that it is computed only once and retrieved whenever needed again;
  2. construction of an optimal solution
     - specify the information that must be additionally stored in order to reconstruct an optimal solution of the problem.
     - identify the way to construct an optimal solution from the information stored.

14.5 The DP algorithms of this module

- Every DP algorithm of the following chapters follows the two steps above: a recurrence with its base cases gives the optimal value, an additional table gives an optimal solution.

Problem	State	Recurrence over	Steps
KP	$v(j,k)$	items and capacities	$\Theta(nc)$
UKP	$v(k)$	capacities	$\Theta(nc)$
BKP	$v(j,k)$	items and capacities	$\Theta(nc\,b_{\max})$
RCP	$v(\ell)$	rod lengths	$\Theta(n^2)$
CCP	$g(k)$	amounts of change	$\Theta(nc)$
ULS	$v(t)$	periods	$\Theta(m^2)$
CLS	$v(t,k)$	periods and inventories	$O(mDC)$

- In all the numerical examples the data are integers: the number of states is finite, and each state is a table cell. The running times $\Theta(nc)$, $\Theta(nc\,b_{\max})$ and $O(mDC)$ contain numbers of the input (a capacity, a bound, a demand), not only its size: they are **pseudo-polynomial**.

14.6 Typical mistakes

Warning

- **Using DP without the optimal substructure.** If an optimal solution of the problem cannot be built from optimal solutions of the subproblems, the recurrence is wrong.
- **Using DP without overlapping subproblems.** If every subproblem appears once, storing the values in a lookup table gives nothing: the DP algorithm is a plain recursion.
- **Forgetting the base cases.** The recurrence needs the subproblems whose optimal value is immediate, otherwise the recursion does not stop.
- **Confusing the optimal value with an optimal solution.** The table of the optimal values does not give the solution: an additional table of the decisions is needed.

14.7 In summary

Summary

- A DP algorithm is an exact algorithm that divides a problem into subproblems of the same type, solves them recursively and combines the optimal solutions.
- It works when the problem has the optimal substructure and overlapping subproblems; the optimal values are stored in a lookup table.
- The design has two steps: the optimal value (recurrence, base cases, bottom-up table) and an optimal solution (decisions stored and read backwards).
- Brute force takes exponential time; DP takes a time that is polynomial in the number of states, and pseudo-polynomial when a number of the input is a table dimension.

Chapter 15

Dynamic programming for knapsack problems

15.1 A DP algorithm for the Knapsack Problem

Prerequisites

- From this module, the chapter «Principles of dynamic programming»: optimal substructure, overlapping subproblems, recurrence relations, base cases, and the top-down and bottom-up approaches.
- From this module, the chapter «Knapsack problems»: the KP, the binary encoding of a solution, and the 2^n subsets of items.
- From the module *Algorithms*: how to read pseudocode, and the size of the input of an algorithm (needed to explain why $\Theta(nc)$ is pseudo-polynomial).

Where we were

- **As a problem**: the KP, its instances and the 2^n subsets (chapter «Knapsack problems»).
- **As a model**: the ILP model, its LP relaxation and the dual bounds (chapter «Knapsack formulations»).
- **Now, as a table**: the DP algorithm computes the optimal value exactly, with no model and no solver.

Objectives

At the end of this chapter you will be able to:

- write the recurrence $v(j, k)$ of the KP, explain why it is correct, and compute the table by hand with the top-down approach, the bottom-up approach and a single vector;
- reconstruct an optimal solution from the matrix s of the decisions;
- count the steps of the DP algorithm and explain why $\Theta(nc)$ is pseudo-polynomial, not polynomial;
- describe the states (w, p), discard the dominated ones, and run the DP algorithm on

lists of states;

- exchange the roles of profits and weights, and compute the optimal value from the table $u(j, q)$.

15.1.1 Problem definition

Definition 15.1: Knapsack Problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Knapsack Problem** (KP) asks to decide which items to insert into the knapsack, so that the total weight of the selected items does not exceed the knapsack capacity, and the total profit of the selected items is maximized.

Example 15.1: KP instance

We consider a knapsack of capacity $c = 5$ and a set $\{1, 2, 3\}$ of 3 items ($n = 3$), with the following profits and weights:

	$j = 1$	$j = 2$	$j = 3$
p_j	6	7	8
w_j	2	3	3

We refer to this KP instance as Example 1.

The number of possible subsets of items is 2^n since, for each item, there is an independent option of inserting it or not into the knapsack. Some subsets may not respect the knapsack capacity. Accordingly, the number of feasible solutions is at most 2^n.

- The number of possible KP solutions grows exponentially with the number n of items. Therefore, enumerating all possible solutions quickly becomes impractical as n increases.

15.1.1.1 Encoding of feasible solutions

A *feasible solution* of the KP is a subset of items whose total weight does not exceed the knapsack capacity.

- A KP feasible solution can be described by the binary vector

$$(x_1, x_2, \ldots, x_n)$$

where each component has the following meaning:

$$x_j = \begin{cases} 1 & \text{if item } j \text{ is selected,} \\ 0 & \text{otherwise,} \end{cases} \qquad \forall j \in \{1, 2, \ldots, n\}.$$

Since the solution is feasible, the total weight of the selected items must not exceed the knapsack capacity:

$$\sum_{j=1}^{n} w_j x_j \leq c.$$

The total profit of a feasible solution is

$$\sum_{j=1}^{n} p_j x_j.$$

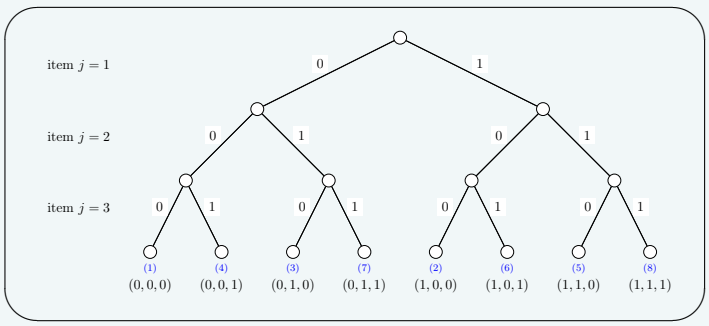
Example 15.2: Enumeration of all possible KP solutions

We consider the KP instance of Example 1 with a knapsack of capacity $c = 5$ and the following profits and weights:

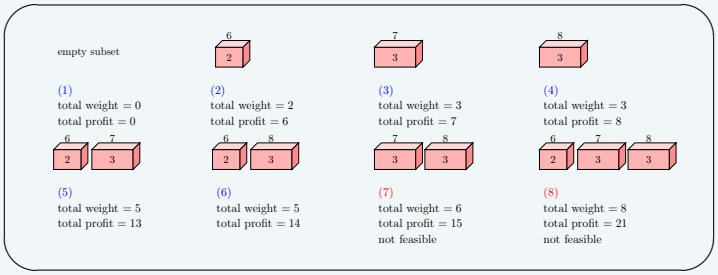
	$j = 1$	$j = 2$	$j = 3$
p_j	6	7	8
w_j	2	3	3

There are $2^3 = 8$ possible subsets of items, corresponding to the different ways of selecting or not selecting the items.

In the following binary tree, each level corresponds to an item: an edge labeled 1 means that the item is selected, while an edge labeled 0 means that it is not selected. Each leaf represents one complete subset of items and reports the corresponding vector.



The figure below shows the subsets associated with the leaves of the tree, through the labels $(1), (2), \ldots, (8)$. For each selected item, the number written inside the parallelepiped is its weight, while the number above it is its profit. The width of each parallelepiped is proportional to the item weight. Subsets whose total weight exceeds the knapsack capacity are marked as not feasible.



Label	Solution vector	Total weight	Total profit	Feasible?
(1)	$(0,0,0)$	0	0	yes
(2)	$(1,0,0)$	2	6	yes
(3)	$(0,1,0)$	3	7	yes
(4)	$(0,0,1)$	3	8	yes
(5)	$(1,1,0)$	$2+3=5$	$6+7=13$	yes
(6)	$(1,0,1)$	$2+3=5$	$6+8=14$	yes
(7)	$(0,1,1)$	$3+3=6$	$7+8=15$	no
(8)	$(1,1,1)$	$2+3+3=8$	$6+7+8=21$	no

15.1.2 A first DP algorithm

- Step 1: computation of the optimal value

For any $j \in \{0,1,\ldots,n\}$ and $k \in \{0,1,\ldots,c\}$, let $v(j,k)$ be the optimal value of the KP subinstance given by:

1. a knapsack of capacity k,
2. a set $\{1,2,\ldots,j\}$ of available items,
3. for each item $h \in \{1,2,\ldots,j\}$, a profit p_h and a weight w_h.

The recursive formula to compute $v(j,k)$ reads as follows:

$$v(j,k) = \begin{cases} 0 & \text{if } j=0 \text{ or } k=0, \\ v(j-1,k) & \text{if } 1 \le j \le n,\ 1 \le k \le c, \text{ and } w_j > k, \\ \max\left\{ v(j-1,k),\ p_j + v(j-1,k-w_j) \right\} & \text{if } 1 \le j \le n,\ 1 \le k \le c,\ w_j \le k. \end{cases} \tag{15.1}$$

  - The recursive decomposition process of formula (15.1) generates at most 2 **independent subproblems**, corresponding to whether or not to select item j for a knapsack

of capacity k. If $w_j > k$, item j cannot be selected, and therefore the only feasible option is not to select it. Here, $v(0, k) = 0$ since no item is available, and $v(j, 0) = 0$ since no capacity is available.

– If $w_j \leq k$, the optimal value $v(j, k)$ is obtained by trying the two possible options, selecting or not selecting item j, and selecting the one that maximizes the profit. The corresponding values are $p_j + v(j-1, k-w_j)$ and $v(j-1, k)$, respectively. If $w_j > k$, item j cannot be selected, and therefore $v(j, k) = v(j-1, k)$.

– Hence, an optimal solution for the subinstance with capacity k and available items $\{1, 2, \ldots, j\}$ is built by combining the chosen decision on item j with an optimal solution of the corresponding residual subproblem.

$\rightsquigarrow$ **Optimal Substructure verified!**

– Different sequences of decisions on the items, possibly made for different values of j and k, can lead to the same subproblem, and therefore the same value $v(j, k)$ may need to be computed several times during the recursive evaluation of formula (15.1).

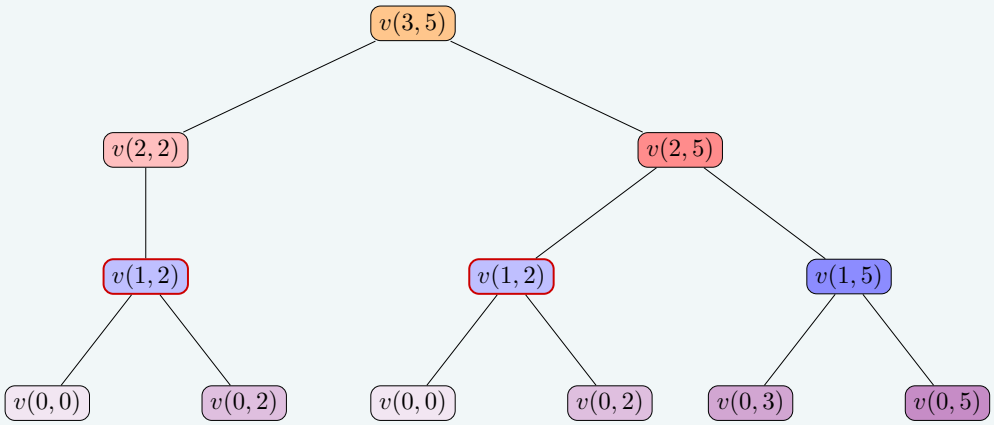
$\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(n, c)$ of the KP instance is the **top-down approach**. The method starts from the call with $j = n$ and $k = c$ and evaluates formula (15.1) recursively: each call considers item j and either does not select it, or also considers selecting it when it fits. The recursion stops when no items are left or when the residual capacity is equal to 0.

– The **recursion tree** of the function v has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 15.3: Recursion tree

We consider the KP instance of Example 1. This is the recursion tree of the top-down approach to compute $v(3,5)$:



The function is called 12 times, and the same subproblem is solved more than once: $v(1,2)$ is called 2 times (shown with a red border), and so are $v(0,2)$ and $v(0,0)$.

– The value $v(j,k)$ can be computed once the values required by the right-hand side of formula (15.1) are known. In particular, $v(j-1,k)$ is needed when item j is not selected and, if $w_j \leq k$, $v(j-1,k-w_j)$ is needed when item j is selected. Therefore, for a fixed value of j, all values $v(j,k)$ can be computed by considering the capacity values k in increasing order.

Example 15.4: Computing the values of the v function

We consider the KP instance of Example 1 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	6	7	8
w_j	2	3	3

The values of the function v are computed as follows:

* The base cases give $v(0,k) = 0$ for all $k \geq 0$.
* Once we have computed the base cases, we can compute $v(1,2)$:

$$v(1,2) = \max\{v(0,2),\ v(0,0) + p_1\} = \max\{0,\ 0+6\} = 6$$

* We can also compute $v(1,5)$:

$$v(1,5) = \max\{v(0,5),\ v(0,3) + p_1\} = \max\{0,\ 0+6\} = 6$$

* Since $w_2 = 3 > k = 2$, item 2 cannot fit, so:

$$v(2,2) = v(1,2) = 6$$

* Once we have computed $v(1,2)$ and $v(1,5)$, we can compute $v(2,5)$:

$$v(2,5) = \max\{v(1,5),\, v(1,2) + p_2\} = \max\{6,\, 6+7\} = 13$$

* Once we have computed $v(2,5)$ and $v(2,2)$, we can compute $v(3,5)$:

$$v(3,5) = \max\{v(2,5),\, v(2,2) + p_3\} = \max\{13,\, 6+8\} = 14$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving any particular subproblem depends only on subproblems with a smaller value of j, we can solve the subproblems in increasing order of j and, for each fixed value of j, in increasing order of capacity, starting from the base cases $v(0,k) = 0$ and $v(j,0) = 0$ and building up to $v(n,c)$: this observation naturally leads to a **much more efficient method** to compute the optimal value $v(n,c)$ of the KP instance.

– A **second method** to compute the optimal value $v(n,c)$ of the KP instance is the **bottom-up approach**. The method starts from the base cases and evaluates formula (15.1) for increasing values of $j \in \{1,2,\ldots,n\}$ and $k \in \{0,1,\ldots,c\}$: each value $v(j,k)$ is computed using already-computed values, until the value $v(n,c)$ of the KP instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: DP

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance

```
1  Function DP(n, p, w, c)
2      v := new array[0..n][0..c]
3      for k := 0 to c do
4          v[0][k] ← 0
5      end
6      for j := 1 to n do
7          for k := 0 to c do
8              if k < w[j] then
9                  v[j][k] ← v[j − 1][k]
10             else
11                 if v[j − 1][k − w[j]] + p[j] > v[j − 1][k] then
12                     v[j][k] ← v[j − 1][k − w[j]] + p[j]
13                 else
14                     v[j][k] ← v[j − 1][k]
15                 end
16             end
17         end
18     end
19     return v[n][c]
```

* The matrix $v[0..n][0..c]$ is the **lookup table**: it stores the optimal values of all subproblems.
* The first loop initializes the values $v[0][k] = 0$ for all capacities $k \in \{0, 1, \ldots, c\}$, corresponding to the case in which no item is available.
* The outer loop, over items $j \in \{1, 2, \ldots, n\}$, computes the rows of the matrix in increasing order of j.
* The inner loop, over $k \in \{0, 1, \ldots, c\}$, computes the optimal value $v[j][k]$ for each capacity k.
* If item j does not fit, then $v[j][k] \leftarrow v[j-1][k]$.
* Otherwise, the DP algorithm compares the value obtained by not selecting item j, namely $v[j-1][k]$, with the value obtained by selecting item j, namely $p[j] + v[j-1][k-w[j]]$.
* Finally, the optimal value of the KP instance is returned as $v[n][c]$.

Example 15.5: Execution of the DP algorithm

We consider the KP instance of Example 1 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	6	7	8
w_j	2	3	3

For each step of the DP algorithm, the table shows the values of j and k, the cell $v[j][k]$, and the operation performed.

j	k	$v[j][k]$	Action
0	0	0	$v[0][0] \leftarrow 0$
0	1	0	$v[0][1] \leftarrow 0$
0	2	0	$v[0][2] \leftarrow 0$
0	3	0	$v[0][3] \leftarrow 0$
0	4	0	$v[0][4] \leftarrow 0$
0	5	0	$v[0][5] \leftarrow 0$
1	0	0	$v[1][0] \leftarrow v[0][0] = 0$
1	1	0	$v[1][1] \leftarrow v[0][1] = 0$
1	2	6	$\max\{v[0][2], p[1] + v[0][0]\} = \max\{0, 6+0\} = 6$
1	3	6	$\max\{v[0][3], p[1] + v[0][1]\} = \max\{0, 6+0\} = 6$
1	4	6	$\max\{v[0][4], p[1] + v[0][2]\} = \max\{0, 6+0\} = 6$
1	5	6	$\max\{v[0][5], p[1] + v[0][3]\} = \max\{0, 6+0\} = 6$
2	0	0	$v[2][0] \leftarrow v[1][0] = 0$
2	1	0	$v[2][1] \leftarrow v[1][1] = 0$
2	2	6	$v[2][2] \leftarrow v[1][2] = 6$
2	3	7	$\max\{v[1][3], p[2] + v[1][0]\} = \max\{6, 7+0\} = 7$
2	4	7	$\max\{v[1][4], p[2] + v[1][1]\} = \max\{6, 7+0\} = 7$
2	5	13	$\max\{v[1][5], p[2] + v[1][2]\} = \max\{6, 7+6\} = 13$
3	0	0	$v[3][0] \leftarrow v[2][0] = 0$
3	1	0	$v[3][1] \leftarrow v[2][1] = 0$
3	2	6	$v[3][2] \leftarrow v[2][2] = 6$
3	3	8	$\max\{v[2][3], p[3] + v[2][0]\} = \max\{7, 8+0\} = 8$
3	4	8	$\max\{v[2][4], p[3] + v[2][1]\} = \max\{7, 8+0\} = 8$
3	5	14	$\max\{v[2][5], p[3] + v[2][2]\} = \max\{13, 8+6\} = 14$

After the execution of the DP algorithm, this is the matrix v:

$v[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$
$j=0$	0	0	0	0	0	0
$j=1$	0	0	6	6	6	6
$j=2$	0	0	6	7	7	13
$j=3$	0	0	6	8	8	14

– After the execution of the DP algorithm, the matrix v contains the optimal value

$v[j][k] = v(j,k)$ for every item $j \in \{0,1,\ldots,n\}$ and every capacity value $k \in \{0,1,\ldots,c\}$.

- The DP algorithm computes the row j of the matrix v using only the row $j-1$. Therefore, when only the optimal value of the KP instance is required, it is not necessary to store the whole matrix: a single vector $v[0..c]$ is sufficient, which uses $c+1$ cells instead of the $(n+1)(c+1)$ cells of the matrix.

- The vector is initialized with the base cases $v[k] = 0$ for all $k \in \{0,1,\ldots,c\}$, which correspond to the row $j = 0$ of the matrix. Then, for each item $j \in \{1,2,\ldots,n\}$, the vector is updated so that it contains the row j of the matrix.

- The order in which the cells are updated is essential. When the cell $v[k]$ is updated for item j, the value $v[k-w_j]$ must still be the one of the row $j-1$, that is $v(j-1,k-w_j)$. Since $k-w_j < k$, this is guaranteed if the capacity values are considered in **decreasing** order $k = c, c-1,\ldots,w_j$: when the cell k is updated, the cell $k-w_j$ has not been updated yet during the current pass.

- If the capacity values were considered in increasing order, the value $v[k-w_j]$ would already be the one of the row j, that is $v(j,k-w_j)$, and item j could be selected more than once, which is not allowed in the KP.

- For every capacity value $k < w_j$, item j does not fit and therefore $v[k]$ must keep the value $v(j-1,k)$: no update is performed, and the inner loop can stop at $k = w_j$.

This is the pseudo-code of the **DP algorithm using a single vector**:

Algorithm

Algorithm: DP_Vector

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance

```
1  Function DP_Vector(n, p, w, c)
2      v := new array[0..c]
3      for k := 0 to c do
4          v[k] ← 0
5      end
6      for j := 1 to n do
7          for k := c to w[j] do
8              if p[j] + v[k − w[j]] > v[k] then
9                  v[k] ← p[j] + v[k − w[j]]
10             end
11         end
12     end
13     return v[c]
```

* The vector $v[0..c]$ is the **lookup table**: at the end of the pass for item j, it stores the row j of the matrix v of the DP algorithm.
* The first loop initializes the values $v[k] = 0$ for all capacities $k \in \{0, 1, \ldots, c\}$, corresponding to the case in which no item is available.
* The outer loop, over items $j \in \{1, 2, \ldots, n\}$, performs one pass of the vector for each item.
* The inner loop, over $k \in \{w[j], w[j] + 1, \ldots, c\}$, considers the capacity values in decreasing order, so that the cells that are still needed with their previous value are not yet modified.
* The condition compares the value obtained by not selecting item j, namely the current value $v[k]$, with the value obtained by selecting item j, namely $p[j] + v[k - w[j]]$. The cell is updated only when selecting item j is strictly better.
* Finally, the optimal value of the KP instance is returned as $v[c]$.

Example 15.6: Execution of the DP algorithm using a single vector

We consider the KP instance of Example 1 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	6	7	8
w_j	2	3	3

The first loop initializes $v[k] = 0$ for all $k \in \{0, 1, \ldots, 5\}$. For each step of the two nested loops, the table shows the values of j and k, the cell $v[k]$ after the step, and the operation performed.

j	k	$v[k]$	Action
1	5	6	$p[1]+v[3] = 6+0 > v[5] = 0$: $v[5] \leftarrow 6$
1	4	6	$p[1]+v[2] = 6+0 > v[4] = 0$: $v[4] \leftarrow 6$
1	3	6	$p[1]+v[1] = 6+0 > v[3] = 0$: $v[3] \leftarrow 6$
1	2	6	$p[1]+v[0] = 6+0 > v[2] = 0$: $v[2] \leftarrow 6$
2	5	13	$p[2]+v[2] = 7+6 > v[5] = 6$: $v[5] \leftarrow 13$
2	4	7	$p[2]+v[1] = 7+0 > v[4] = 6$: $v[4] \leftarrow 7$
2	3	7	$p[2]+v[0] = 7+0 > v[3] = 6$: $v[3] \leftarrow 7$
3	5	14	$p[3]+v[2] = 8+6 > v[5] = 13$: $v[5] \leftarrow 14$
3	4	8	$p[3]+v[1] = 8+0 > v[4] = 7$: $v[4] \leftarrow 8$
3	3	8	$p[3]+v[0] = 8+0 > v[3] = 7$: $v[3] \leftarrow 8$

This is the content of the vector v after the initialization and after each pass of the outer loop:

$v[k]$	0	1	2	3	4	5
initialization	0	0	0	0	0	0
pass $j=1$	0	0	6	6	6	6
pass $j=2$	0	0	6	7	7	13
pass $j=3$	0	0	6	8	8	14

Each pass transforms the vector v from the row $j-1$ into the row j of the matrix v reported in Example 15.5. The optimal value of the KP instance is $v[5] = 14$.

  - The vector version computes only the optimal value of the KP instance. The information needed to construct an optimal solution, which is described in Step 2, records the decision taken for every item j and every capacity value k, and therefore cannot be stored in a single vector.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, i.e., the binary vector $(x_1, x_2, \ldots, x_n)$, it is necessary to store some additional information.
  - For each item j and capacity value k, in addition to the optimal value $v[j][k]$, we store $s[j][k]$, which tells whether item j is selected in an optimal solution for the subproblem restricted to items $\{1, 2, \ldots, j\}$ and capacity k.

– More precisely, $s[j][k]$ is equal to 1 if item j is selected when computing the value $v[j][k]$, and it is equal to 0 otherwise.

This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to construct an optimal solution:

Algorithm

Algorithm: DP_InfoSol

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance and the matrix of binary information $s[0..n][0..c]$

```
1  Function DP_InfoSol(n, p, w, c)
2      v := new array[0..n][0..c]
3      s := new array[0..n][0..c]
4      for k := 0 to c do
5          v[0][k] ← 0
6          s[0][k] ← 0
7      end
8      for j := 1 to n do
9          for k := 0 to c do
10             if k < w[j] then
11                 v[j][k] ← v[j − 1][k]
12                 s[j][k] ← 0
13             else
14                 if v[j − 1][k − w[j]] + p[j] > v[j − 1][k] then
15                     v[j][k] ← v[j − 1][k − w[j]] + p[j]
16                     s[j][k] ← 1
17                 else
18                     v[j][k] ← v[j − 1][k]
19                     s[j][k] ← 0
20                 end
21             end
22         end
23     end
24     return v[n][c], s
```

∗ The DP algorithm uses the additional matrix s, which keeps track of the decision made when $v[j][k]$ is computed. If item j is selected, $s[j][k] \leftarrow 1$ stores this decision; otherwise, $s[j][k] \leftarrow 0$ stores that item j is not selected.

Example 15.7: Computation of the additional information

We consider the KP instance of Example 1 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	6	7	8
w_j	2	3	3

For each step of the DP algorithm, the table shows the values of j and k, the cell $s[j][k]$, and the operation performed.

j	k	$s[j][k]$	Action
0	0	0	$s[0][0] \leftarrow 0$
0	1	0	$s[0][1] \leftarrow 0$
0	2	0	$s[0][2] \leftarrow 0$
0	3	0	$s[0][3] \leftarrow 0$
0	4	0	$s[0][4] \leftarrow 0$
0	5	0	$s[0][5] \leftarrow 0$
1	0	0	$0 < 2$: $s[1][0] \leftarrow 0$
1	1	0	$1 < 2$: $s[1][1] \leftarrow 0$
1	2	1	$p[1] + v[0][0] = 6 + 0 > v[0][2] = 0$: $s[1][2] \leftarrow 1$
1	3	1	$p[1] + v[0][1] = 6 + 0 > v[0][3] = 0$: $s[1][3] \leftarrow 1$
1	4	1	$p[1] + v[0][2] = 6 + 0 > v[0][4] = 0$: $s[1][4] \leftarrow 1$
1	5	1	$p[1] + v[0][3] = 6 + 0 > v[0][5] = 0$: $s[1][5] \leftarrow 1$
2	0	0	$0 < 3$: $s[2][0] \leftarrow 0$
2	1	0	$1 < 3$: $s[2][1] \leftarrow 0$
2	2	0	$2 < 3$: $s[2][2] \leftarrow 0$
2	3	1	$p[2] + v[1][0] = 7 + 0 > v[1][3] = 6$: $s[2][3] \leftarrow 1$
2	4	1	$p[2] + v[1][1] = 7 + 0 > v[1][4] = 6$: $s[2][4] \leftarrow 1$
2	5	1	$p[2] + v[1][2] = 7 + 6 > v[1][5] = 6$: $s[2][5] \leftarrow 1$
3	0	0	$0 < 3$: $s[3][0] \leftarrow 0$
3	1	0	$1 < 3$: $s[3][1] \leftarrow 0$
3	2	0	$2 < 3$: $s[3][2] \leftarrow 0$
3	3	1	$p[3] + v[2][0] = 8 + 0 > v[2][3] = 7$: $s[3][3] \leftarrow 1$
3	4	1	$p[3] + v[2][1] = 8 + 0 > v[2][4] = 7$: $s[3][4] \leftarrow 1$
3	5	1	$p[3] + v[2][2] = 8 + 6 > v[2][5] = 13$: $s[3][5] \leftarrow 1$

This is the matrix s after the execution of the DP algorithm:

$s[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$
$j=0$	0	0	0	0	0	0
$j=1$	0	0	1	1	1	1
$j=2$	0	0	0	1	1	1
$j=3$	0	0	0	1	1	1

– Once the matrix s has been computed, an optimal solution can be reconstructed by scanning the items from $j = n$ down to $j = 1$, starting from capacity $k = c$ and

$x_j = 0$ for all $j \in \{1, 2, \ldots, n\}$:

$$\begin{aligned} x_j &\leftarrow 1, \quad k \leftarrow k - w_j \quad \text{if } s[j][k] = 1, \\ x_j &\leftarrow 0 \qquad\qquad\quad \text{if } s[j][k] = 0. \end{aligned}$$

This yields the vector $(x_1, x_2, \ldots, x_n)$, where x_j is equal to 1 if item j is selected in an optimal solution, and 0 otherwise.

Algorithm

Algorithm: DP_Reconstruct

Input: The number n of items, the knapsack capacity c, the array $w[1..n]$ of item weights, and the matrix $s[0..n][0..c]$

Output: An optimal solution of the KP instance

```
1  Function DP_Reconstruct(n, c, w, s)
2      x := new array[1..n]
3      for j := 1 to n do
4          x[j] ← 0
5      end
6      k := c
7      for j := n to 1 do
8          if s[j][k] = 1 then
9              x[j] ← 1
10             k ← k − w[j]
11         end
12     end
13     return x
```

Example 15.8: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, we have the following matrix s:

$s[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$
$j=0$	0	0	0	0	0	0
$j=1$	0	0	1	1	1	1
$j=2$	0	0	0	1	1	1
$j=3$	0	0	0	1	1	1

We can reconstruct an optimal solution by starting from $j = 3$ and $k = 5$ as follows:

Iteration	j	k	$s[j][k]$	$x[1]$	$x[2]$	$x[3]$	Action
–	–	–	–	0	0	0	$k = 5$
1	3	5	1	0	0	1	$k \leftarrow 5 - 3 = 2$
2	2	2	0	0	0	1	no update
3	1	2	1	1	0	1	$k \leftarrow 2 - 2 = 0$

Thus, the vector

$$(1, 0, 1)$$

is an optimal solution. The corresponding optimal value is $6 + 8 = 14$.

15.1.2.1 Number of steps

- Let $S(n, c)$ denote the number of steps executed by `DP` in the worst case, where n is the number of items and c is the knapsack capacity. We compute $S(n, c)$ by counting how many times the lines of the pseudo-code of `DP` are executed.

- In the worst case, the condition at line 11 is checked whenever the item fits. Since the item weights are positive integers, this happens at most nc times. The number of executions of the relevant lines is:

  - Line 2 is executed 1 time.
  - Line 3 is executed $c + 1$ times.
  - Line 4 is executed $c + 1$ times.
  - Line 6 is executed n times.
  - Line 7 is executed $n(c + 1)$ times.
  - Line 8 is executed $n(c + 1)$ times.
  - Line 11 is executed at most nc times.
  - one among Line 9, Line 12, and Line 14 is executed for each pair (j, k), and therefore these assignment lines are executed $n(c + 1)$ times in total.
  - Line 19 is executed 1 time.

  Therefore, the total number of steps in the worst case is

$$1 + (c + 1) + (c + 1) + n + n(c + 1) + n(c + 1) + nc + n(c + 1) + 1.$$

  Thus,

$$S(n, c) = 4nc + 4n + 2c + 4.$$

  Hence,

$$S(n, c) = \Theta(nc).$$

  Indeed,

$$\lim_{n,c\to\infty} \frac{4nc + 4n + 2c + 4}{nc} = \lim_{n,c\to\infty} \left(4 + \frac{4}{c} + \frac{2}{n} + \frac{4}{nc}\right) = 4 \in \mathbb{R}_{>0}.$$

The DP algorithm executes

$$S(n, c) = \Theta(nc)$$

steps in the worst case.

- The version `DP_InfoSol` stores the additional matrix s and performs only a constant number of extra operations inside the same loops. Therefore, its number of steps remains $\Theta(nc)$.
- The version `DP_Vector` uses a single vector instead of the matrix and executes the same two nested loops, whose inner loop performs at most $c+1$ iterations for each item. Therefore, its number of steps remains $\Theta(nc)$, while the number of cells of the lookup table decreases from $(n+1)(c+1)$ to $c+1$.

15.1.2.2 An implementation with lists

- Step 1: computation of the optimal value
  - The DP algorithm of this section stores one value for every pair given by an item and a capacity value, that is $(n+1)(c+1)$ values in total. Each row of the matrix v is non-decreasing in the capacity value and changes only at a few capacity values: every other cell repeats the value of the cell on its left.
  - This subsection describes a **different implementation of the same DP algorithm**. The recursive decomposition and the optimal substructure are the ones of formula (15.1): what changes is the data structure. Instead of a whole row of the matrix, only the pairs at which the row changes are stored, that is only the pairs that are actually obtained by a subset of items. These pairs are kept in a list, one list for each item.

A **state** is a pair (w, p), where w is the total weight and p is the total profit of a subset of items whose total weight does not exceed the knapsack capacity c.

  - The empty subset gives the state $(0, 0)$. A subset of the items $\{1, 2, \ldots, j\}$ gives a state obtained by summing the weights and the profits of its items.
  - Two different subsets can give the same state. In this case the two subsets are interchangeable: both occupy the same weight in the knapsack and both give the same profit, and therefore it is enough to keep one state.
  - More generally, a state can be discarded whenever another state is at least as good in both components.

Given two states (w, p) and (w', p') with $(w, p) \neq (w', p')$, the state (w, p) **dominates** the state (w', p') if

$$w \leq w' \qquad \text{and} \qquad p \geq p'.$$

A state is **non-dominated** if no other state dominates it.

  - A dominated state can be discarded. Indeed, let (w, p) dominate (w', p') and let S' be a subset of the remaining items that can be added to (w', p') without exceeding the capacity. Since $w \leq w'$, the same subset S' can also be added to (w, p), and

since $p \geq p'$ the resulting total profit is not smaller. Therefore no optimal solution is lost.

For any $j \in \{0, 1, \ldots, n\}$, let L_j be the list of the non-dominated states of the KP subinstance given by:

1. a knapsack of capacity c,
2. a set $\{1, 2, \ldots, j\}$ of available items,
3. for each item $h \in \{1, 2, \ldots, j\}$, a profit p_h and a weight w_h.

The states of L_j are sorted by strictly increasing weight.

The recursive formula to compute L_j reads as follows:

$$L_j = \begin{cases} \{(0,0)\} & \text{if } j = 0, \\ \mathrm{nd}\big(L_{j-1} \cup M_j\big) & \text{if } 1 \leq j \leq n, \end{cases} \tag{15.2}$$

where $\mathrm{nd}(S)$ denotes the list of the non-dominated states of a set S of states, and

$$M_j = \big\{\, (w + w_j,\ p + p_j) \ :\ (w,p) \in L_{j-1},\ w + w_j \leq c \,\big\}.$$

– The recursive decomposition process of formula (15.2) generates at most 2 **independent subproblems** for each state of L_{j-1}, corresponding to whether or not to select item j. If item j is not selected, the state is unchanged and belongs to L_{j-1}. If item j is selected, the state becomes $(w + w_j, p + p_j)$ and belongs to M_j; this option is available only when $w + w_j \leq c$. Here, $L_0 = \{(0,0)\}$ since the only subset of an empty set of items is the empty subset.

– Hence, every non-dominated state reachable with the items $\{1, 2, \ldots, j\}$ is built by combining the chosen decision on item j with a non-dominated state reachable with the items $\{1, 2, \ldots, j-1\}$.

$\rightsquigarrow$ **Optimal Substructure verified!**

– Different subsets of the items $\{1, 2, \ldots, j\}$ can give the same state, and a state can also be dominated by another one. In both cases a single state is kept in L_j, and all the subsets that lead to it are treated at once in the following iterations.

$\rightsquigarrow$ **Overlapping Subproblems verified!**

– The optimal value of the KP instance is the largest profit stored in the last list.

The optimal value of the KP instance is

$$\max \{\, p \;:\; (w,p) \in L_n \,\}. \tag{15.3}$$

– In a list of non-dominated states sorted by strictly increasing weight, the profits are also strictly increasing: if a state had a weight larger than another one and a profit not larger, it would be dominated. Therefore the maximum of formula (15.3) is the profit of the last state of L_n.

– The list L_j holds the information of the row j of the matrix v in compressed form:

$$v[j][k] = \max \{\, p \;:\; (w,p) \in L_j,\ w \le k \,\} \qquad \forall k \in \{0,1,\ldots,c\}.$$

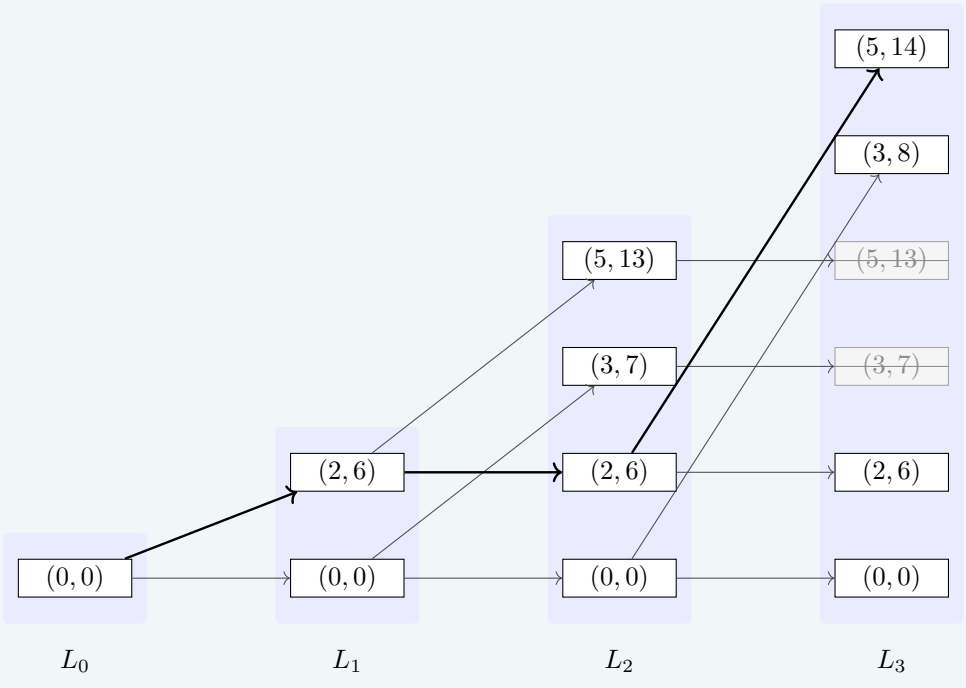
Every state (w,p) of L_j is a capacity value at which the row increases, and p is the new value.

Example 15.9: The lists of states

We consider the KP instance of Example 1 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	6	7	8
w_j	2	3	3

Each shaded band collects the states of one list. An arc entering the band of L_j corresponds to the decision taken on item j: a horizontal arc means that item j is not selected, while a slanted arc means that item j is selected. The thick arcs give the states of an optimal solution.



The two crossed states of the last band are dominated and are therefore discarded: $(3, 7)$ is dominated by $(3, 8)$ and $(5, 13)$ is dominated by $(5, 14)$. Selecting item 3 from the states $(3, 7)$ and $(5, 13)$ of L_2 would give the total weights 6 and 8, which exceed the knapsack capacity $c = 5$, and therefore no arc leaves these two states towards the states of M_3.

– The lists are computed for increasing values of $j \in \{1, 2, \ldots, n\}$, starting from $L_0 = \{(0, 0)\}$: each list L_j is computed from the already-computed list L_{j-1}, until the list L_n is obtained.

– The list L_{j-1} and the list M_j are both sorted by increasing weight, because M_j is obtained from L_{j-1} by adding the same weight w_j to every state. Therefore the union of formula (15.2) is computed by scanning the two lists at the same time, as in the merge of two sorted lists.

– To describe the scanning order we use the following convention.

The state (w, p) **precedes** the state (w', p') if

$$w < w' \qquad \text{or} \qquad (w = w' \text{ and } p > p').$$

∗ If the states are scanned in this order, a state is non-dominated if and only if its profit is strictly larger than the largest profit of the states already scanned. Indeed, every state already scanned has a weight not larger, and therefore it dominates the current state as soon as its profit is not smaller.

This is the pseudo-code of the **DP algorithm using lists**:

Algorithm

Algorithm: DP_Lists

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance

```
1  Function DP_Lists(n, p, w, c)
2      L := new list containing the single state (0, 0)
3      for j := 1 to n do
4          M := new empty list
5          for h := 1 to |L| do
6              if L[h].w + w[j] ≤ c then
7                  append (L[h].w + w[j], L[h].p + p[j]) to M
8              end
9          end
10         L ← Merge(L, M)
11     end
12     return L[|L|].p
```

This is the pseudo-code of the procedure that merges two lists and discards the dominated states:

Algorithm

Algorithm: Merge

Input: Two lists L and M of states, both sorted by increasing weight
Output: The list of the non-dominated states of $L \cup M$, sorted by increasing weight

```
1  Function Merge(L, M)
2      R := new empty list
3      h := 1
4      k := 1
5      π := −1
6      while h ≤ |L| or k ≤ |M| do
7          if k > |M| or (h ≤ |L| and L[h] precedes M[k]) then
8              (a, b) ← L[h]
9              h ← h + 1
10         else
11             (a, b) ← M[k]
12             k ← k + 1
13         end
14         if b > π then
15             append (a, b) to R
16             π ← b
17         end
18     end
19     return R
```

* The list L is the **lookup table**: at the end of the iteration for item j, it stores the list L_j.
* The list L is initialized with the single state $(0, 0)$, which is the list L_0.
* The outer loop, over items $j \in \{1, 2, \ldots, n\}$, computes one list for each item.
* The inner loop builds the list M of the states obtained by selecting item j. A state is added to M only when the resulting total weight does not exceed the knapsack capacity.
* The procedure `Merge` scans the two lists at the same time and keeps a state only when its profit is strictly larger than the largest profit already appended, which is stored in the auxiliary variable π. The initial value $\pi = -1$ guarantees that the first scanned state is always appended.
* Since the states of L are sorted by strictly increasing weight and strictly increasing profit, the largest profit is the one of the last state. Therefore the optimal value of the KP instance is returned as $L[|L|].p$.

Example 15.10: Execution of the DP algorithm

We consider the KP instance of Example 1 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	6	7	8
w_j	2	3	3

For each item j, the table shows the list M_j of the states obtained by selecting item j and the resulting list L_j.

j	M_j	L_j
0	–	$\{(0,0)\}$
1	$\{(2,6)\}$	$\{(0,0),(2,6)\}$
2	$\{(3,7),(5,13)\}$	$\{(0,0),(2,6),(3,7),(5,13)\}$
3	$\{(3,8),(5,14)\}$	$\{(0,0),(2,6),(3,8),(5,14)\}$

For each state scanned by `Merge`, the table shows the value of j, the state, the list it is taken from, and the operation performed.

j	state	from	Action
1	$(0,0)$	L_0	$0 > \pi = -1$: append
1	$(2,6)$	M_1	$6 > \pi = 0$: append
2	$(0,0)$	L_1	$0 > \pi = -1$: append
2	$(2,6)$	L_1	$6 > \pi = 0$: append
2	$(3,7)$	M_2	$7 > \pi = 6$: append
2	$(5,13)$	M_2	$13 > \pi = 7$: append
3	$(0,0)$	L_2	$0 > \pi = -1$: append
3	$(2,6)$	L_2	$6 > \pi = 0$: append
3	$(3,8)$	M_3	$8 > \pi = 6$: append
3	$(3,7)$	L_2	$7 \leq \pi = 8$: discard (dominated)
3	$(5,14)$	M_3	$14 > \pi = 8$: append
3	$(5,13)$	L_2	$13 \leq \pi = 14$: discard (dominated)

The last state of L_3 is $(5,14)$, and therefore the optimal value of the KP instance is 14.

  - After the execution of the DP algorithm, the list L contains the non-dominated states L_n of the KP instance, sorted by strictly increasing weight and strictly increasing profit.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, i.e., the binary vector $(x_1, x_2, \ldots, x_n)$, it is necessary to store some additional information. The list L_n alone gives the optimal value, but it does not say which items produce the state that attains it.

– For each item j and each position $h \in \{1, 2, \ldots, |L_j|\}$, in addition to the state $L_j[h]$, we store two values:

  * $\sigma_j[h]$, which is equal to 1 if item j is selected in the state $L_j[h]$, and is equal to 0 otherwise;

  * $\rho_j[h]$, which is the position in L_{j-1} of the state from which $L_j[h]$ is obtained.

– A state taken from L_{j-1} gives $\sigma_j[h] = 0$, while a state taken from M_j gives $\sigma_j[h] = 1$. In both cases $\rho_j[h]$ is the position of the state of L_{j-1} that generated it.

– The DP algorithm must also keep all the lists $L_0, L_1, \ldots, L_n$, and not only the last one, because the reconstruction goes backwards through them.

Example 15.11: Computation of the additional information

We consider the KP instance of Example 1. For each item j and each position h, the table shows the state $L_j[h]$ and the stored values $\sigma_j[h]$ and $\rho_j[h]$.

j	h	$L_j[h]$	$\sigma_j[h]$	$\rho_j[h]$
1	1	$(0, 0)$	0	1
1	2	$(2, 6)$	1	1
2	1	$(0, 0)$	0	1
2	2	$(2, 6)$	0	2
2	3	$(3, 7)$	1	1
2	4	$(5, 13)$	1	2
3	1	$(0, 0)$	0	1
3	2	$(2, 6)$	0	2
3	3	$(3, 8)$	1	1
3	4	$(5, 14)$	1	2

For instance, the state $L_3[4] = (5, 14)$ has $\sigma_3[4] = 1$ and $\rho_3[4] = 2$: it is obtained by selecting item 3 from the state $L_2[2] = (2, 6)$, since $2 + w_3 = 2 + 3 = 5$ and $6 + p_3 = 6 + 8 = 14$.

– Once the lists and the stored values have been computed, an optimal solution is reconstructed by scanning the items from $j = n$ down to $j = 1$, starting from the last position $h = |L_n|$, which holds the state of largest profit:

$$x_j \leftarrow \sigma_j[h], \quad h \leftarrow \rho_j[h] \quad \text{for } j = n, n-1, \ldots, 1.$$

This yields the vector $(x_1, x_2, \ldots, x_n)$, where x_j is equal to 1 if item j is selected in an optimal solution, and 0 otherwise.

Algorithm

Algorithm: DP_Lists_Reconstruct

Input: The number n of items, the lists $L_0, L_1, \ldots, L_n$, and the stored values σ and ρ

Output: An optimal solution of the KP instance

```
1  Function DP_Lists_Reconstruct(n, L, σ, ρ)
2      x := new array[1..n]
3      for j := 1 to n do
4          x[j] ← 0
5      end
6      h := |L_n|
7      for j := n to 1 do
8          x[j] ← σ_j[h]
9          h ← ρ_j[h]
10     end
11     return x
```

Example 15.12: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, the last list is

$$L_3 = \{(0,0), (2,6), (3,8), (5,14)\}$$

and the optimal value is the profit of its last state, that is 14.

We can reconstruct an optimal solution by starting from $j = 3$ and $h = |L_3| = 4$ as follows:

Iteration	j	h	$L_j[h]$	$\sigma_j[h]$	$\rho_j[h]$	$x[1]$	$x[2]$	$x[3]$	Action
–	–	–	–	–	–	0	0	0	$h = 4$
1	3	4	$(5,14)$	1	2	0	0	1	$h \leftarrow 2$
2	2	2	$(2,6)$	0	2	0	0	1	$h \leftarrow 2$
3	1	2	$(2,6)$	1	1	1	0	1	$h \leftarrow 1$

Thus, the vector

$$(1,0,1)$$

is an optimal solution. The corresponding optimal value is $6 + 8 = 14$.

15.1.2.3 Number of steps

- Let $S(n,c)$ denote the number of steps executed by `DP_Lists` in the worst case, where n is the number of items and c is the knapsack capacity.

- The states of a list have strictly increasing weights, and every weight belongs to $\{0, 1, \ldots, c\}$: therefore $|L_j| \leq c + 1$ for all $j \in \{0, 1, \ldots, n\}$, and $|M_j| \leq |L_{j-1}|$.

- For each item, `DP_Lists` scans L_{j-1} once to build M_j and then calls `Merge`, which scans $|L_{j-1}| + |M_j| \leq 2(c + 1)$ states and performs a constant number of operations for each of them. Therefore the number of steps executed for each item is $\Theta(c)$ in the worst case, and
$$S(n, c) = \Theta(nc).$$

- The three implementations have the same order of growth, but they do not store the same amount of information. The matrix v always has $(n+1)(c+1)$ cells, while the lists contain only the states that are actually reachable: a list reaches the bound $c + 1$ only when every capacity value is the total weight of a subset of the items. On the KP instance of Example 1 the lists contain 1, 2, 4, and 4 states, against the 24 cells of the matrix v.

- The list L_j also satisfies $|L_j| \leq 2^j$, since the items $\{1, 2, \ldots, j\}$ have 2^j subsets. Therefore
$$|L_j| \leq \min\left\{2^j,\, c + 1\right\},$$
and the number of states stays small while j is small, whatever the knapsack capacity.

15.1.3 A second DP algorithm

- Step 1: computation of the optimal value

  - In the DP algorithm of the previous section, a subproblem is identified by a capacity value, and its optimal value is the maximum total profit that can be obtained without exceeding that capacity. In the DP algorithm of this section, the roles of the profits and of the weights are exchanged: a subproblem is identified by a profit value, and its optimal value is the minimum total weight needed to obtain exactly that profit.

  - Since each profit p_j is a positive integer and each item can be selected at most once, the total profit of a feasible solution is an integer between 0 and the total profit of all the items
$$P = \sum_{j=1}^{n} p_j.$$
Therefore, it is enough to consider the profit values $q \in \{0, 1, \ldots, P\}$.

  - The number of subproblems of this DP algorithm grows with the total profit P. To keep the number of subproblems small, the DP algorithm is illustrated on the following KP instance, which has the same item weights and the same knapsack capacity as the KP instance of Example 1, and smaller item profits.

Example 15.13: KP instance

We consider a knapsack of capacity $c = 5$ and a set $\{1, 2, 3\}$ of 3 items ($n = 3$), with the following profits and weights:

	$j = 1$	$j = 2$	$j = 3$
p_j	3	3	4
w_j	2	3	3

We refer to this KP instance as Example 15.13.

For any $j \in \{0, 1, \ldots, n\}$ and $q \in \{0, 1, \ldots, P\}$, let $u(j, q)$ be the optimal value of the minimum-weight subinstance given by:

1. a target profit q,
2. a set $\{1, 2, \ldots, j\}$ of available items,
3. for each item $h \in \{1, 2, \ldots, j\}$, a profit p_h and a weight w_h,

which asks to select a subset of the available items whose total profit is equal to q, so that the total weight of the selected items is minimized.

The recursive formula to compute $u(j, q)$ reads as follows:

$$u(j,q) = \begin{cases} 0 & \text{if } q = 0, \\ +\infty & \text{if } j = 0 \text{ and } 1 \le q \le P, \\ u(j-1,q) & \text{if } 1 \le j \le n,\ 1 \le q \le P, \text{ and } p_j > q, \\ \min\left\{ u(j-1,q),\ w_j + u(j-1,q-p_j) \right\} & \text{if } 1 \le j \le n,\ 1 \le q \le P,\ p_j \le q. \end{cases} \tag{15.4}$$

– The recursive decomposition process of formula (15.4) generates at most 2 **independent subproblems**, corresponding to whether or not to select item j to reach the target profit q. If $p_j > q$, selecting item j would exceed the target profit, and therefore the only feasible option is not to select it. Here, $u(j, 0) = 0$ since the empty subset has total profit 0 and total weight 0. Moreover, $u(0, q) = +\infty$ for $q \ge 1$: in this case no item is available, so no subset has total profit equal to q. The value $+\infty$ is used to rule out this infeasible case.

– If $p_j \le q$, the optimal value $u(j, q)$ is obtained by trying the two possible options, selecting or not selecting item j, and selecting the one that minimizes the total

weight. The corresponding values are $w_j + u(j-1, q-p_j)$ and $u(j-1, q)$, respectively. If $p_j > q$, item j cannot be selected, and therefore $u(j, q) = u(j-1, q)$.

– Hence, a minimum-weight subset with total profit q and available items $\{1, 2, \ldots, j\}$ is built by combining the chosen decision on item j with a minimum-weight subset of the corresponding residual subproblem.

$\rightsquigarrow$ **Optimal Substructure verified!**

– Different sequences of decisions on the items, possibly made for different values of j and q, can lead to the same subproblem, and therefore the same value $u(j, q)$ may need to be computed several times during the recursive evaluation of formula (15.4).

$\rightsquigarrow$ **Overlapping Subproblems verified!**

– The values $u(n, q)$ do not give the optimal value of the KP instance directly: they give, for each target profit q, the minimum total weight needed to obtain it. A subset with total profit q fits into the knapsack if and only if its minimum total weight does not exceed the capacity, that is, if and only if $u(n, q) \leq c$.

The optimal value of the KP instance is

$$\max \Big\{ q \in \{0, 1, \ldots, P\} \ : \ u(n, q) \leq c \Big\}. \tag{15.5}$$

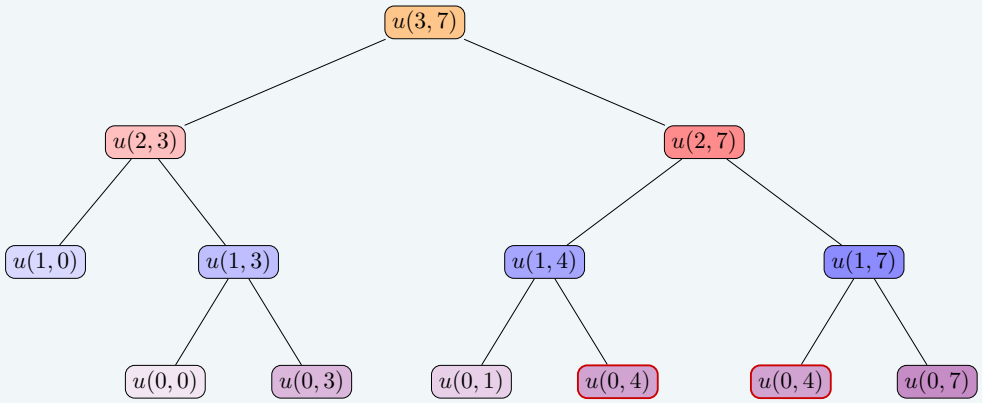
∗ The set in formula (15.5) is never empty, since $u(n, 0) = 0 \leq c$, and therefore the maximum is always well defined.

– A **first method** to compute the optimal value of the KP instance is the **top-down approach**. The method evaluates formula (15.4) recursively, starting from the call with $j = n$, for each target profit $q \in \{0, 1, \ldots, P\}$: each call considers item j and either does not select it, or also considers selecting it when its profit does not exceed the target profit. The recursion stops when no items are left or when the target profit is equal to 0. Finally, formula (15.5) is applied to the computed values.

– The **recursion tree** of the function u has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 15.14: Recursion tree

We consider the KP instance of Example 15.13. This is the recursion tree of the top-down approach to compute $u(3,7)$:



The function is called 13 times, and the same subproblem is solved more than once: $u(0,4)$ is called 2 times (shown with a red border). Moreover, the top-down approach must be applied to every target profit $q \in \{0,1,\ldots,10\}$, and the same subproblem is generated by different initial calls: for instance, the initial call $u(3,3)$ generates the calls $u(2,3)$, $u(1,3)$, $u(1,0)$, $u(0,3)$, and $u(0,0)$, which are already generated by the initial call $u(3,7)$ shown above.

– The value $u(j,q)$ can be computed once the values required by the right-hand side of formula (15.4) are known. In particular, $u(j-1,q)$ is needed when item j is not selected and, if $p_j \leq q$, $u(j-1,q-p_j)$ is needed when item j is selected. Therefore, for a fixed value of j, all values $u(j,q)$ can be computed by considering the profit values q in increasing order.

Example 15.15: Computing the values of the u function

We consider the KP instance of Example 15.13 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	3	3	4
w_j	2	3	3

The total profit of all the items is $P = 3+3+4 = 10$. The values of the function u are computed as follows:

* The base cases give $u(0,0) = 0$ and $u(0,q) = +\infty$ for all $q \geq 1$.
* The first case of formula (15.4) ($q = 0$) gives the base case:

$$u(1,0) = u(0,0) = 0$$

* Once we have computed the base cases, we can compute $u(1,3)$:

$$u(1,3) = \min\{u(0,3),\ w_1 + u(0,0)\} = \min\{+\infty,\ 2+0\} = 2$$

* We can also compute $u(1,4)$ and $u(1,7)$:

$$u(1,4) = \min\{u(0,4),\ w_1 + u(0,1)\} = \min\{+\infty,\ +\infty\} = +\infty$$

$$u(1,7) = \min\{u(0,7),\ w_1 + u(0,4)\} = \min\{+\infty,\ +\infty\} = +\infty$$

* Once we have computed $u(1,3)$ and $u(1,0)$, we can compute $u(2,3)$:

$$u(2,3) = \min\{u(1,3),\ w_2 + u(1,0)\} = \min\{2,\ 3+0\} = 2$$

* Once we have computed $u(1,7)$ and $u(1,4)$, we can compute $u(2,7)$:

$$u(2,7) = \min\{u(1,7),\ w_2 + u(1,4)\} = \min\{+\infty,\ +\infty\} = +\infty$$

* Once we have computed $u(2,7)$ and $u(2,3)$, we can compute $u(3,7)$:

$$u(3,7) = \min\{u(2,7),\ w_3 + u(2,3)\} = \min\{+\infty,\ 3+2\} = 5$$

* Therefore, a total profit of 7 can be obtained with a total weight of 5, which does not exceed the knapsack capacity $c = 5$.

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving any particular subproblem depends only on subproblems with a smaller value of j, we can solve the subproblems in increasing order of j and, for each fixed value of j, in increasing order of target profit, starting from the base cases $u(0,0) = 0$, $u(0,q) = +\infty$ for all $q \in \{1,2,\ldots,P\}$, and $u(j,0) = 0$, and building up to the values $u(n,q)$ for all $q \in \{0,1,\ldots,P\}$: this observation naturally leads to a **much more efficient method** to compute the optimal value of the KP instance.

– A **second method** to compute the optimal value of the KP instance is the **bottom-up approach**. The method starts from the base cases and evaluates formula (15.4) for increasing values of $j \in \{1,2,\ldots,n\}$ and $q \in \{0,1,\ldots,P\}$: each value $u(j,q)$ is computed using already-computed values, until all the values $u(n,q)$ are obtained. The optimal value of the KP instance is then obtained by applying formula (15.5).

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: DP2

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance

```
1  Function DP2(n, p, w, c)
2      P := 0
3      for j := 1 to n do
4          P ← P + p[j]
5      end
6      u := new array[0..n][0..P]
7      u[0][0] ← 0
8      for q := 1 to P do
9          u[0][q] ← +∞
10     end
11     for j := 1 to n do
12         for q := 0 to P do
13             if q < p[j] then
14                 u[j][q] ← u[j − 1][q]
15             else
16                 if w[j] + u[j − 1][q − p[j]] < u[j − 1][q] then
17                     u[j][q] ← w[j] + u[j − 1][q − p[j]]
18                 else
19                     u[j][q] ← u[j − 1][q]
20                 end
21             end
22         end
23     end
24     z := 0
25     for q := 0 to P do
26         if u[n][q] ≤ c then
27             z ← q
28         end
29     end
30     return z
```

* The first loop computes the total profit P of all the items, which is the largest target profit that has to be considered.
* The matrix $u[0..n][0..P]$ is the **lookup table**: it stores the optimal values of all subproblems.
* The second loop initializes the values $u[0][0] = 0$ and $u[0][q] = +\infty$ for all target profits $q \in \{1, 2, \ldots, P\}$, corresponding to the case in which no item is available.

* The outer loop, over items $j \in \{1, 2, \ldots, n\}$, computes the rows of the matrix in increasing order of j.
* The inner loop, over $q \in \{0, 1, \ldots, P\}$, computes the optimal value $u[j][q]$ for each target profit q.
* If the profit of item j exceeds the target profit, then $u[j][q] \leftarrow u[j-1][q]$.
* Otherwise, the DP algorithm compares the value obtained by not selecting item j, namely $u[j-1][q]$, with the value obtained by selecting item j, namely $w[j] + u[j-1][q-p[j]]$.
* The auxiliary variable z stores the largest target profit whose minimum total weight does not exceed the knapsack capacity. It is initialized to 0, which is always a feasible target profit since $u[n][0] = 0 \leq c$. The last loop scans the target profits in increasing order and updates z whenever $u[n][q] \leq c$, so that at the end z is the maximum of formula (15.5).
* Finally, the optimal value of the KP instance is returned as z.

Example 15.16: Execution of the DP algorithm

We consider the KP instance of Example 15.13 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	3	3	4
w_j	2	3	3

The total profit of all the items is $P = 3 + 3 + 4 = 10$. For each step of the DP algorithm, the table shows the values of j and q, the cell $u[j][q]$, and the operation performed.

j	q	$u[j][q]$	Action
0	0	0	$u[0][0] \leftarrow 0$
0	1	$+\infty$	$u[0][1] \leftarrow +\infty$
0	2	$+\infty$	$u[0][2] \leftarrow +\infty$
0	3	$+\infty$	$u[0][3] \leftarrow +\infty$
0	4	$+\infty$	$u[0][4] \leftarrow +\infty$
0	5	$+\infty$	$u[0][5] \leftarrow +\infty$
0	6	$+\infty$	$u[0][6] \leftarrow +\infty$
0	7	$+\infty$	$u[0][7] \leftarrow +\infty$
0	8	$+\infty$	$u[0][8] \leftarrow +\infty$
0	9	$+\infty$	$u[0][9] \leftarrow +\infty$
0	10	$+\infty$	$u[0][10] \leftarrow +\infty$
1	0	0	$u[1][0] \leftarrow u[0][0] = 0$
1	1	$+\infty$	$u[1][1] \leftarrow u[0][1] = +\infty$
1	2	$+\infty$	$u[1][2] \leftarrow u[0][2] = +\infty$
1	3	2	$\min\{u[0][3], w[1] + u[0][0]\} = \min\{+\infty, 2 + 0\} = 2$

1	4	$+\infty$	$\min\{u[0][4], w[1] + u[0][1]\} = \min\{+\infty, +\infty\} = +\infty$
1	5	$+\infty$	$\min\{u[0][5], w[1] + u[0][2]\} = \min\{+\infty, +\infty\} = +\infty$
1	6	$+\infty$	$\min\{u[0][6], w[1] + u[0][3]\} = \min\{+\infty, +\infty\} = +\infty$
1	7	$+\infty$	$\min\{u[0][7], w[1] + u[0][4]\} = \min\{+\infty, +\infty\} = +\infty$
1	8	$+\infty$	$\min\{u[0][8], w[1] + u[0][5]\} = \min\{+\infty, +\infty\} = +\infty$
1	9	$+\infty$	$\min\{u[0][9], w[1] + u[0][6]\} = \min\{+\infty, +\infty\} = +\infty$
1	10	$+\infty$	$\min\{u[0][10], w[1] + u[0][7]\} = \min\{+\infty, +\infty\} = +\infty$
2	0	0	$u[2][0] \leftarrow u[1][0] = 0$
2	1	$+\infty$	$u[2][1] \leftarrow u[1][1] = +\infty$
2	2	$+\infty$	$u[2][2] \leftarrow u[1][2] = +\infty$
2	3	2	$\min\{u[1][3], w[2] + u[1][0]\} = \min\{2, 3 + 0\} = 2$
2	4	$+\infty$	$\min\{u[1][4], w[2] + u[1][1]\} = \min\{+\infty, +\infty\} = +\infty$
2	5	$+\infty$	$\min\{u[1][5], w[2] + u[1][2]\} = \min\{+\infty, +\infty\} = +\infty$
2	6	5	$\min\{u[1][6], w[2] + u[1][3]\} = \min\{+\infty, 3 + 2\} = 5$
2	7	$+\infty$	$\min\{u[1][7], w[2] + u[1][4]\} = \min\{+\infty, +\infty\} = +\infty$
2	8	$+\infty$	$\min\{u[1][8], w[2] + u[1][5]\} = \min\{+\infty, +\infty\} = +\infty$
2	9	$+\infty$	$\min\{u[1][9], w[2] + u[1][6]\} = \min\{+\infty, +\infty\} = +\infty$
2	10	$+\infty$	$\min\{u[1][10], w[2] + u[1][7]\} = \min\{+\infty, +\infty\} = +\infty$
3	0	0	$u[3][0] \leftarrow u[2][0] = 0$
3	1	$+\infty$	$u[3][1] \leftarrow u[2][1] = +\infty$
3	2	$+\infty$	$u[3][2] \leftarrow u[2][2] = +\infty$
3	3	2	$u[3][3] \leftarrow u[2][3] = 2$
3	4	3	$\min\{u[2][4], w[3] + u[2][0]\} = \min\{+\infty, 3 + 0\} = 3$
3	5	$+\infty$	$\min\{u[2][5], w[3] + u[2][1]\} = \min\{+\infty, +\infty\} = +\infty$
3	6	5	$\min\{u[2][6], w[3] + u[2][2]\} = \min\{5, +\infty\} = 5$
3	7	5	$\min\{u[2][7], w[3] + u[2][3]\} = \min\{+\infty, 3 + 2\} = 5$
3	8	$+\infty$	$\min\{u[2][8], w[3] + u[2][4]\} = \min\{+\infty, +\infty\} = +\infty$
3	9	$+\infty$	$\min\{u[2][9], w[3] + u[2][5]\} = \min\{+\infty, +\infty\} = +\infty$
3	10	8	$\min\{u[2][10], w[3] + u[2][6]\} = \min\{+\infty, 3 + 5\} = 8$

After the execution of the two nested loops of the DP algorithm, this is the matrix u:

$u[j][q]$	0	1	2	3	4	5	6	7	8	9	10
$j=0$	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
$j=1$	0	$+\infty$	$+\infty$	2	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
$j=2$	0	$+\infty$	$+\infty$	2	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	$+\infty$	$+\infty$
$j=3$	0	$+\infty$	$+\infty$	2	3	$+\infty$	5	5	$+\infty$	$+\infty$	8

The last loop of the DP algorithm scans the last row of the matrix u and keeps the largest target profit whose minimum total weight does not exceed the knapsack capacity $c = 5$:

q	$u[3][q]$	Action
0	0	$u[3][0] = 0 \leq 5$: $z \leftarrow 0$
1	$+\infty$	$u[3][1] = +\infty > 5$: no update
2	$+\infty$	$u[3][2] = +\infty > 5$: no update
3	2	$u[3][3] = 2 \leq 5$: $z \leftarrow 3$
4	3	$u[3][4] = 3 \leq 5$: $z \leftarrow 4$
5	$+\infty$	$u[3][5] = +\infty > 5$: no update
6	5	$u[3][6] = 5 \leq 5$: $z \leftarrow 6$
7	5	$u[3][7] = 5 \leq 5$: $z \leftarrow 7$
8	$+\infty$	$u[3][8] = +\infty > 5$: no update
9	$+\infty$	$u[3][9] = +\infty > 5$: no update
10	8	$u[3][10] = 8 > 5$: no update

Thus, the optimal value of the KP instance is $z = 7$.

- After the execution of the DP algorithm, the matrix u contains the optimal value $u[j][q] = u(j, q)$ for every item $j \in \{0, 1, \ldots, n\}$ and every target profit $q \in \{0, 1, \ldots, P\}$.

- The DP algorithm computes the row j of the matrix u using only the row $j-1$. Therefore, when only the optimal value of the KP instance is required, it is not necessary to store the whole matrix: a single vector $u[0..P]$ is sufficient, which uses $P+1$ cells instead of the $(n+1)(P+1)$ cells of the matrix.

- The vector is initialized with the base cases $u[0] = 0$ and $u[q] = +\infty$ for all $q \in \{1, 2, \ldots, P\}$, which correspond to the row $j = 0$ of the matrix. Then, for each item $j \in \{1, 2, \ldots, n\}$, the vector is updated so that it contains the row j of the matrix.

- The order in which the cells are updated is essential. When the cell $u[q]$ is updated for item j, the value $u[q-p_j]$ must still be the one of the row $j-1$, that is $u(j-1, q-p_j)$. Since $q - p_j < q$, this is guaranteed if the target profits are considered in **decreasing** order $q = P, P-1, \ldots, p_j$: when the cell q is updated, the cell $q - p_j$ has not been updated yet during the current pass.

– If the target profits were considered in increasing order, the value $u[q - p_j]$ would already be the one of the row j, that is $u(j, q - p_j)$, and item j could be selected more than once, which is not allowed in the KP.

– For every target profit $q < p_j$, the profit of item j exceeds the target profit and therefore $u[q]$ must keep the value $u(j-1, q)$: no update is performed, and the inner loop can stop at $q = p_j$.

This is the pseudo-code of the **DP algorithm using a single vector**:

Algorithm

Algorithm: DP2_Vector

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance

```
1  Function DP2_Vector(n, p, w, c)
2      P := 0
3      for j := 1 to n do
4          P ← P + p[j]
5      end
6      u := new array[0..P]
7      u[0] ← 0
8      for q := 1 to P do
9          u[q] ← +∞
10     end
11     for j := 1 to n do
12         for q := P to p[j] do
13             if w[j] + u[q − p[j]] < u[q] then
14                 u[q] ← w[j] + u[q − p[j]]
15             end
16         end
17     end
18     z := 0
19     for q := 0 to P do
20         if u[q] ≤ c then
21             z ← q
22         end
23     end
24     return z
```

∗ The vector $u[0..P]$ is the **lookup table**: at the end of the pass for item j, it stores the row j of the matrix u of the DP algorithm.

* The second loop initializes the values $u[0] = 0$ and $u[q] = +\infty$ for all target profits $q \in \{1, 2, \ldots, P\}$, corresponding to the case in which no item is available.

* The outer loop, over items $j \in \{1, 2, \ldots, n\}$, performs one pass of the vector for each item.

* The inner loop, over $q \in \{p[j], p[j]+1, \ldots, P\}$, considers the target profits in decreasing order, so that the cells that are still needed with their previous value are not yet modified.

* The condition compares the value obtained by not selecting item j, namely the current value $u[q]$, with the value obtained by selecting item j, namely $w[j] + u[q - p[j]]$. The cell is updated only when selecting item j is strictly better.

* The last loop scans the vector and returns the largest target profit whose minimum total weight does not exceed the knapsack capacity, as in formula (15.5).

Example 15.17: Execution of the DP algorithm using a single vector

We consider the KP instance of Example 15.13 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	3	3	4
w_j	2	3	3

The total profit of all the items is $P = 3 + 3 + 4 = 10$ and the second loop initializes $u[0] = 0$ and $u[q] = +\infty$ for all $q \in \{1, 2, \ldots, 10\}$. For each step of the two nested loops, the table shows the values of j and q, the cell $u[q]$ after the step, and the operation performed.

j	q	$u[q]$	Action
1	10	$+\infty$	$w[1] + u[7] = +\infty \geq u[10] = +\infty$: no update
1	9	$+\infty$	$w[1] + u[6] = +\infty \geq u[9] = +\infty$: no update
1	8	$+\infty$	$w[1] + u[5] = +\infty \geq u[8] = +\infty$: no update
1	7	$+\infty$	$w[1] + u[4] = +\infty \geq u[7] = +\infty$: no update
1	6	$+\infty$	$w[1] + u[3] = +\infty \geq u[6] = +\infty$: no update
1	5	$+\infty$	$w[1] + u[2] = +\infty \geq u[5] = +\infty$: no update
1	4	$+\infty$	$w[1] + u[1] = +\infty \geq u[4] = +\infty$: no update
1	3	2	$w[1] + u[0] = 2 < u[3] = +\infty$: $u[3] \leftarrow 2$
2	10	$+\infty$	$w[2] + u[7] = +\infty \geq u[10] = +\infty$: no update
2	9	$+\infty$	$w[2] + u[6] = +\infty \geq u[9] = +\infty$: no update
2	8	$+\infty$	$w[2] + u[5] = +\infty \geq u[8] = +\infty$: no update
2	7	$+\infty$	$w[2] + u[4] = +\infty \geq u[7] = +\infty$: no update
2	6	5	$w[2] + u[3] = 5 < u[6] = +\infty$: $u[6] \leftarrow 5$
2	5	$+\infty$	$w[2] + u[2] = +\infty \geq u[5] = +\infty$: no update

2	4	$+\infty$	$w[2]+u[1]=+\infty \geq u[4]=+\infty$: no update
2	3	2	$w[2]+u[0]=3 \geq u[3]=2$: no update
3	10	8	$w[3]+u[6]=8 < u[10]=+\infty$: $u[10] \leftarrow 8$
3	9	$+\infty$	$w[3]+u[5]=+\infty \geq u[9]=+\infty$: no update
3	8	$+\infty$	$w[3]+u[4]=+\infty \geq u[8]=+\infty$: no update
3	7	5	$w[3]+u[3]=5 < u[7]=+\infty$: $u[7] \leftarrow 5$
3	6	5	$w[3]+u[2]=+\infty \geq u[6]=5$: no update
3	5	$+\infty$	$w[3]+u[1]=+\infty \geq u[5]=+\infty$: no update
3	4	3	$w[3]+u[0]=3 < u[4]=+\infty$: $u[4] \leftarrow 3$

This is the content of the vector u after the initialization and after each pass of the outer loop:

$u[q]$	0	1	2	3	4	5	6	7	8	9	10
initialization	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
pass $j=1$	0	$+\infty$	$+\infty$	2	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
pass $j=2$	0	$+\infty$	$+\infty$	2	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	$+\infty$	$+\infty$
pass $j=3$	0	$+\infty$	$+\infty$	2	3	$+\infty$	5	5	$+\infty$	$+\infty$	8

Each pass transforms the vector u from the row $j-1$ into the row j of the matrix u reported in Example 15.16. The last loop then scans the vector and gives the optimal value $z=7$ of the KP instance, as in Example 15.16.

- The vector version computes only the optimal value of the KP instance. The information needed to construct an optimal solution, which is described in Step 2, records the decision taken for every item j and every target profit q, and therefore cannot be stored in a single vector.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, i.e., the binary vector $(x_1, x_2, \ldots, x_n)$, it is necessary to store some additional information.
  - For each item j and target profit q, in addition to the optimal value $u[j][q]$, we store $s[j][q]$, which tells whether item j is selected in a minimum-weight subset of the items $\{1, 2, \ldots, j\}$ with total profit equal to q.
  - More precisely, $s[j][q]$ is equal to 1 if item j is selected when computing the value $u[j][q]$, and it is equal to 0 otherwise.

  This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to construct an optimal solution:

Algorithm

Algorithm: DP2_InfoSol

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance and the matrix of binary information $s[0..n][0..P]$

```
1  Function DP2_InfoSol(n, p, w, c)
2      P := 0
3      for j := 1 to n do
4          P ← P + p[j]
5      end
6      u := new array[0..n][0..P]
7      s := new array[0..n][0..P]
8      u[0][0] ← 0
9      s[0][0] ← 0
10     for q := 1 to P do
11         u[0][q] ← +∞
12         s[0][q] ← 0
13     end
14     for j := 1 to n do
15         for q := 0 to P do
16             if q < p[j] then
17                 u[j][q] ← u[j − 1][q]
18                 s[j][q] ← 0
19             else
20                 if w[j] + u[j − 1][q − p[j]] < u[j − 1][q] then
21                     u[j][q] ← w[j] + u[j − 1][q − p[j]]
22                     s[j][q] ← 1
23                 else
24                     u[j][q] ← u[j − 1][q]
25                     s[j][q] ← 0
26                 end
27             end
28         end
29     end
30     z := 0
31     for q := 0 to P do
32         if u[n][q] ≤ c then
33             z ← q
34         end
35     end
36     return z, s
```

∗ The DP algorithm uses the additional matrix s, which keeps track of the decision

made when $u[j][q]$ is computed. If item j is selected, $s[j][q] \leftarrow 1$ stores this decision; otherwise, $s[j][q] \leftarrow 0$ stores that item j is not selected.

Example 15.18: Computation of the additional information

We consider the KP instance of Example 15.13 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	3	3	4
w_j	2	3	3

For each step of the DP algorithm, the table shows the values of j and q, the cell $s[j][q]$, and the operation performed.

j	q	$s[j][q]$	Action
0	0	0	$s[0][0] \leftarrow 0$
0	1	0	$s[0][1] \leftarrow 0$
0	2	0	$s[0][2] \leftarrow 0$
0	3	0	$s[0][3] \leftarrow 0$
0	4	0	$s[0][4] \leftarrow 0$
0	5	0	$s[0][5] \leftarrow 0$
0	6	0	$s[0][6] \leftarrow 0$
0	7	0	$s[0][7] \leftarrow 0$
0	8	0	$s[0][8] \leftarrow 0$
0	9	0	$s[0][9] \leftarrow 0$
0	10	0	$s[0][10] \leftarrow 0$
1	0	0	$0 < 3$: $s[1][0] \leftarrow 0$
1	1	0	$1 < 3$: $s[1][1] \leftarrow 0$
1	2	0	$2 < 3$: $s[1][2] \leftarrow 0$
1	3	1	$w[1] + u[0][0] = 2 + 0 < u[0][3] = +\infty$: $s[1][3] \leftarrow 1$
1	4	0	$w[1] + u[0][1] = +\infty \geq u[0][4] = +\infty$: $s[1][4] \leftarrow 0$
1	5	0	$w[1] + u[0][2] = +\infty \geq u[0][5] = +\infty$: $s[1][5] \leftarrow 0$
1	6	0	$w[1] + u[0][3] = +\infty \geq u[0][6] = +\infty$: $s[1][6] \leftarrow 0$
1	7	0	$w[1] + u[0][4] = +\infty \geq u[0][7] = +\infty$: $s[1][7] \leftarrow 0$
1	8	0	$w[1] + u[0][5] = +\infty \geq u[0][8] = +\infty$: $s[1][8] \leftarrow 0$
1	9	0	$w[1] + u[0][6] = +\infty \geq u[0][9] = +\infty$: $s[1][9] \leftarrow 0$
1	10	0	$w[1] + u[0][7] = +\infty \geq u[0][10] = +\infty$: $s[1][10] \leftarrow 0$
2	0	0	$0 < 3$: $s[2][0] \leftarrow 0$
2	1	0	$1 < 3$: $s[2][1] \leftarrow 0$
2	2	0	$2 < 3$: $s[2][2] \leftarrow 0$
2	3	0	$w[2] + u[1][0] = 3 + 0 \geq u[1][3] = 2$: $s[2][3] \leftarrow 0$
2	4	0	$w[2] + u[1][1] = +\infty \geq u[1][4] = +\infty$: $s[2][4] \leftarrow 0$
2	5	0	$w[2] + u[1][2] = +\infty \geq u[1][5] = +\infty$: $s[2][5] \leftarrow 0$
2	6	1	$w[2] + u[1][3] = 3 + 2 < u[1][6] = +\infty$: $s[2][6] \leftarrow 1$
2	7	0	$w[2] + u[1][4] = +\infty \geq u[1][7] = +\infty$: $s[2][7] \leftarrow 0$
2	8	0	$w[2] + u[1][5] = +\infty \geq u[1][8] = +\infty$: $s[2][8] \leftarrow 0$
2	9	0	$w[2] + u[1][6] = +\infty \geq u[1][9] = +\infty$: $s[2][9] \leftarrow 0$

2	10	0	$w[2]+u[1][7]=+\infty \geq u[1][10]=+\infty$: $s[2][10] \leftarrow 0$
3	0	0	$0<4$: $s[3][0] \leftarrow 0$
3	1	0	$1<4$: $s[3][1] \leftarrow 0$
3	2	0	$2<4$: $s[3][2] \leftarrow 0$
3	3	0	$3<4$: $s[3][3] \leftarrow 0$
3	4	1	$w[3]+u[2][0]=3+0<u[2][4]=+\infty$: $s[3][4] \leftarrow 1$
3	5	0	$w[3]+u[2][1]=+\infty \geq u[2][5]=+\infty$: $s[3][5] \leftarrow 0$
3	6	0	$w[3]+u[2][2]=+\infty \geq u[2][6]=5$: $s[3][6] \leftarrow 0$
3	7	1	$w[3]+u[2][3]=3+2<u[2][7]=+\infty$: $s[3][7] \leftarrow 1$
3	8	0	$w[3]+u[2][4]=+\infty \geq u[2][8]=+\infty$: $s[3][8] \leftarrow 0$
3	9	0	$w[3]+u[2][5]=+\infty \geq u[2][9]=+\infty$: $s[3][9] \leftarrow 0$
3	10	1	$w[3]+u[2][6]=3+5<u[2][10]=+\infty$: $s[3][10] \leftarrow 1$

This is the matrix s after the execution of the DP algorithm:

$s[j][q]$	0	1	2	3	4	5	6	7	8	9	10
$j=0$	0	0	0	0	0	0	0	0	0	0	0
$j=1$	0	0	0	1	0	0	0	0	0	0	0
$j=2$	0	0	0	0	0	0	1	0	0	0	0
$j=3$	0	0	0	0	1	0	0	1	0	0	1

– Once the matrix s has been computed, an optimal solution can be reconstructed by scanning the items from $j=n$ down to $j=1$, starting from the target profit $q=z$ returned by the DP algorithm and $x_j=0$ for all $j \in \{1,2,\ldots,n\}$:

$$\begin{aligned} &x_j \leftarrow 1, \quad q \leftarrow q-p_j && \text{if } s[j][q]=1, \\ &x_j \leftarrow 0 && \text{if } s[j][q]=0. \end{aligned}$$

This yields the vector $(x_1,x_2,\ldots,x_n)$, where x_j is equal to 1 if item j is selected in an optimal solution, and 0 otherwise.

Algorithm

Algorithm: DP2_Reconstruct

Input: The number n of items, the optimal value z of the KP instance, the array $p[1..n]$ of item profits, and the matrix $s[0..n][0..P]$

Output: An optimal solution of the KP instance

```
1  Function DP2_Reconstruct(n, z, p, s)
2      x := new array[1..n]
3      for j := 1 to n do
4          x[j] ← 0
5      end
6      q := z
7      for j := n to 1 do
8          if s[j][q] = 1 then
9              x[j] ← 1
10             q ← q − p[j]
11         end
12     end
13     return x
```

Example 15.19: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 15.13, the optimal value is $z = 7$ and we have the following matrix s:

$s[j][q]$	0	1	2	3	4	5	6	7	8	9	10
$j = 0$	0	0	0	0	0	0	0	0	0	0	0
$j = 1$	0	0	0	1	0	0	0	0	0	0	0
$j = 2$	0	0	0	0	0	0	1	0	0	0	0
$j = 3$	0	0	0	0	1	0	0	1	0	0	1

We can reconstruct an optimal solution by starting from $j = 3$ and $q = z = 7$ as follows:

Iteration	j	q	$s[j][q]$	$x[1]$	$x[2]$	$x[3]$	Action
–	–	–	–	0	0	0	$q = 7$
1	3	7	1	0	0	1	$q \leftarrow 7 - 4 = 3$
2	2	3	0	0	0	1	no update
3	1	3	1	1	0	1	$q \leftarrow 3 - 3 = 0$

Thus, the vector

$$(1, 0, 1)$$

is an optimal solution. The corresponding optimal value is $3 + 4 = 7$.

15.1.3.1 Number of steps

- Let $S(n, P)$ denote the number of steps executed by `DP2` in the worst case, where n is the number of items and $P = \sum_{j=1}^{n} p_j$ is the total profit of all the items. We compute $S(n, P)$ by counting how many times the lines of the pseudo-code of `DP2` are executed.

- The condition at line 16 is checked only when $q \geq p[j]$, that is for $P+1-p_j$ target profits for each item j, and therefore $n(P+1) - P$ times in total. The number of executions of the relevant lines is:

  - Line 2 is executed 1 time.
  - Line 3 is executed n times.
  - Line 4 is executed n times.
  - Line 6 is executed 1 time.
  - Line 7 is executed 1 time.
  - Line 8 is executed P times.
  - Line 9 is executed P times.
  - Line 11 is executed n times.
  - Line 12 is executed $n(P+1)$ times.
  - Line 13 is executed $n(P+1)$ times.
  - Line 16 is executed $n(P+1) - P$ times.
  - one among Line 14, Line 17, and Line 19 is executed for each pair (j, q), and therefore these assignment lines are executed $n(P+1)$ times in total.
  - Line 24 is executed 1 time.
  - Line 25 is executed $P+1$ times.
  - Line 26 is executed $P+1$ times.
  - Line 27 is executed at most $P+1$ times.
  - Line 30 is executed 1 time.

  Therefore, the total number of steps in the worst case is

$$\begin{aligned} &1 + n + n + 1 + 1 + P + P + n + n(P+1) + n(P+1) \\ &+ \big(n(P+1) - P\big) + n(P+1) + 1 + (P+1) + (P+1) + (P+1) + 1. \end{aligned}$$

  Thus,

$$S(n, P) = 4nP + 7n + 4P + 8.$$

  Hence,

$$S(n, P) = \Theta(nP).$$

  Indeed,

$$\lim_{n,P\to\infty} \frac{4nP + 7n + 4P + 8}{nP} = \lim_{n,P\to\infty} \left(4 + \frac{7}{P} + \frac{4}{n} + \frac{8}{nP}\right) = 4 \in \mathbb{R}_{>0}.$$

The DP algorithm executes

$$S(n, P) = \Theta(nP)$$

steps in the worst case, where $P = \sum_{j=1}^{n} p_j$.

- The version `DP2_InfoSol` stores the additional matrix s and performs only a constant number of extra operations inside the same loops. Therefore, its number of steps remains $\Theta(nP)$.
- The version `DP2_Vector` uses a single vector instead of the matrix and executes the same two nested loops, whose inner loop performs at most $P + 1$ iterations for each item. Therefore, its number of steps remains $\Theta(nP)$, while the number of cells of the lookup table decreases from $(n + 1)(P + 1)$ to $P + 1$.

15.1.3.2 An implementation with lists

- Step 1: computation of the optimal value
  - The DP algorithm of this section stores one value for every pair given by an item and a target profit, that is $(n + 1)(P + 1)$ values in total. Many of these values are equal to $+\infty$, because no subset of the available items has that total profit.
  - This subsection describes a **different implementation of the same DP algorithm**. The recursive decomposition and the optimal substructure are the ones of formula (15.4): what changes is the data structure. Instead of a whole row of the matrix, only the pairs that are actually obtained by a subset of items are stored, and among them only those that can lead to an optimal solution. These pairs are kept in a list, one list for each item.

A **state** is a pair (w, p), where w is the total weight and p is the total profit of a subset of items.

  - The empty subset gives the state $(0, 0)$. A subset of the items $\{1, 2, \ldots, j\}$ gives a state obtained by summing the weights and the profits of its items.
  - Two different subsets can give the same state, and in this case it is enough to keep one of them. More generally, a state can be discarded whenever another state is at least as good in both components.

Given two states (w, p) and (w', p') with $(w, p) \neq (w', p')$, the state (w, p) **dominates** the state (w', p') if

$$w \leq w' \qquad \text{and} \qquad p \geq p'.$$

A state is **non-dominated** if no other state dominates it.

– A dominated state can be discarded. Indeed, let (w, p) dominate (w', p') and let S' be a subset of the remaining items that can be added to (w', p') without exceeding the capacity. Since $w \leq w'$, the same subset S' can also be added to (w, p), and since $p \geq p'$ the resulting total profit is not smaller. Therefore no optimal solution is lost.

For any $j \in \{0, 1, \ldots, n\}$, let L_j be the list of the non-dominated states that can be obtained by selecting a subset of the items $\{1, 2, \ldots, j\}$, sorted by strictly increasing weight.

The recursive formula to compute L_j reads as follows:

$$L_j = \begin{cases} \{(0,0)\} & \text{if } j = 0, \\ \mathrm{nd}\big(L_{j-1} \cup M_j\big) & \text{if } 1 \leq j \leq n, \end{cases} \tag{15.6}$$

where $\mathrm{nd}(S)$ denotes the list of the non-dominated states of a set S of states, and

$$M_j = \big\{\, (w + w_j,\ p + p_j) \ :\ (w, p) \in L_{j-1} \,\big\}.$$

– The recursive decomposition process of formula (15.6) generates 2 **independent subproblems** for each state of L_{j-1}, corresponding to whether or not to select item j. If item j is not selected, the state is unchanged and belongs to L_{j-1}. If item j is selected, the state becomes $(w + w_j, p + p_j)$ and belongs to M_j. Here, $L_0 = \{(0, 0)\}$ since the only subset of an empty set of items is the empty subset.

– As in formula (15.4), the knapsack capacity does not appear in the recursion: the list L_j collects the states of *all* the subsets of the items $\{1, 2, \ldots, j\}$, including those whose total weight exceeds the capacity.

– Hence, every non-dominated state reachable with the items $\{1, 2, \ldots, j\}$ is built by combining the chosen decision on item j with a non-dominated state reachable with the items $\{1, 2, \ldots, j-1\}$.

  ⇝ **Optimal Substructure verified!**

– Different subsets of the items $\{1, 2, \ldots, j\}$ can give the same state, and a state can also be dominated by another one. In both cases a single state is kept in L_j, and all the subsets that lead to it are treated at once in the following iterations.

  ⇝ **Overlapping Subproblems verified!**

– The knapsack capacity is used only at the end, exactly as in formula (15.5): among the states of L_n, only those whose total weight does not exceed the capacity correspond to a feasible solution.

The optimal value of the KP instance is

$$\max\{\, p \;:\; (w,p) \in L_n,\ w \le c \,\}. \tag{15.7}$$

– The set in formula (15.7) is never empty, since $(0,0) \in L_n$ and $0 \le c$, and therefore the maximum is always well defined.

– The list L_j holds the information of the row j of the matrix u in compressed form:

$$\min\{\, w \;:\; (w,p) \in L_j,\ p \ge q \,\} = \min_{q' \ge q} u[j][q'] \qquad \forall q \in \{0,1,\ldots,P\}.$$

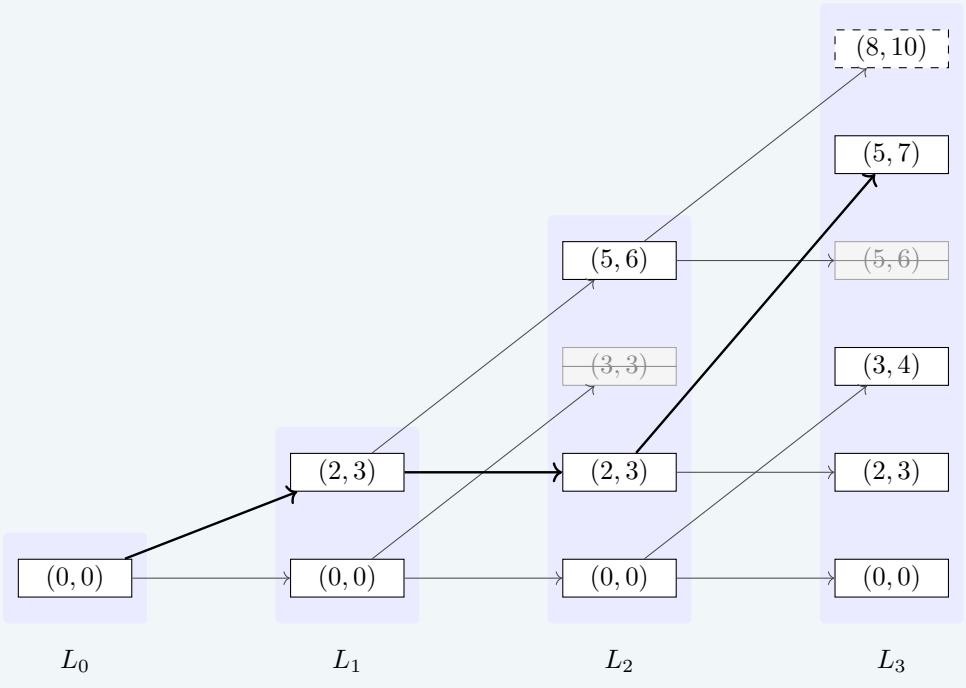
The target profits whose row entry is $+\infty$ leave no state in the list, and a target profit whose minimum weight is not smaller than the one of a larger target profit leaves no state either.

Example 15.20: The lists of states

We consider the KP instance of Example 15.13 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	3	3	4
w_j	2	3	3

Each shaded band collects the states of one list. An arc entering the band of L_j corresponds to the decision taken on item j: a horizontal arc means that item j is not selected, while a slanted arc means that item j is selected. The thick arcs give the states of an optimal solution.



The two crossed states are dominated and are therefore discarded: $(3, 3)$ is dominated by $(2, 3)$ and $(5, 6)$ is dominated by $(5, 7)$. The state $(8, 10)$, drawn with a dashed border, has a total weight larger than the knapsack capacity $c = 5$: it is kept in the list, because the capacity is not used during the recursion, and it is discarded only by formula (15.7).

– The list L_{j-1} and the list M_j are both sorted by increasing weight, because M_j is obtained from L_{j-1} by adding the same weight w_j to every state. Therefore the union of formula (15.6) is computed by scanning the two lists at the same time, as in the merge of two sorted lists.

– To describe the scanning order we use the following convention.

The state (w, p) **precedes** the state (w', p') if

$$w < w' \qquad \text{or} \qquad (w = w' \text{ and } p > p').$$

∗ If the states are scanned in this order, a state is non-dominated if and only if its profit is strictly larger than the largest profit of the states already scanned. Indeed, every state already scanned has a weight not larger, and therefore it dominates the current state as soon as its profit is not smaller.

This is the pseudo-code of the **DP algorithm using lists**:

Algorithm

Algorithm: DP2_Lists

Input: A KP instance given by the number n of items, arrays $p[1..n]$ and $w[1..n]$ of item profits and weights, and the knapsack capacity c

Output: The optimal value of the KP instance

```
1  Function DP2_Lists(n, p, w, c)
2      L := new list containing the single state (0, 0)
3      for j := 1 to n do
4          M := new empty list
5          for h := 1 to |L| do
6              append (L[h].w + w[j], L[h].p + p[j]) to M
7          end
8          L ← Merge(L, M)
9      end
10     z := 0
11     for h := 1 to |L| do
12         if L[h].w ≤ c then
13             z ← L[h].p
14         end
15     end
16     return z
```

This is the pseudo-code of the procedure that merges two lists and discards the dominated states:

Algorithm

Algorithm: Merge

Input: Two lists L and M of states, both sorted by increasing weight
Output: The list of the non-dominated states of $L \cup M$, sorted by increasing weight

```
1  Function Merge(L, M)
2      R := new empty list
3      h := 1
4      k := 1
5      π := −1
6      while h ≤ |L| or k ≤ |M| do
7          if k > |M| or (h ≤ |L| and L[h] precedes M[k]) then
8              (a, b) ← L[h]
9              h ← h + 1
10         else
11             (a, b) ← M[k]
12             k ← k + 1
13         end
14         if b > π then
15             append (a, b) to R
16             π ← b
17         end
18     end
19     return R
```

* The list L is the **lookup table**: at the end of the iteration for item j, it stores the list L_j.
* The list L is initialized with the single state $(0, 0)$, which is the list L_0.
* The outer loop, over items $j \in \{1, 2, \ldots, n\}$, computes one list for each item.
* The inner loop builds the list M of the states obtained by selecting item j. No state is discarded here, because the knapsack capacity is not used during the recursion.
* The procedure `Merge` scans the two lists at the same time and keeps a state only when its profit is strictly larger than the largest profit already appended, which is stored in the auxiliary variable π. The initial value $\pi = -1$ guarantees that the first scanned state is always appended.
* The last loop implements formula (15.7). The states of L have strictly increasing weights and strictly increasing profits, and therefore the auxiliary variable z ends with the profit of the last state whose weight does not exceed the knapsack capacity.
* Finally, the optimal value of the KP instance is returned as z.

Example 15.21: Execution of the DP algorithm

We consider the KP instance of Example 15.13 with a knapsack of capacity $c = 5$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$
p_j	3	3	4
w_j	2	3	3

For each item j, the table shows the list M_j of the states obtained by selecting item j and the resulting list L_j.

j	M_j	L_j
0	–	$\{(0,0)\}$
1	$\{(2,3)\}$	$\{(0,0),(2,3)\}$
2	$\{(3,3),(5,6)\}$	$\{(0,0),(2,3),(5,6)\}$
3	$\{(3,4),(5,7),(8,10)\}$	$\{(0,0),(2,3),(3,4),(5,7),(8,10)\}$

For each state scanned by `Merge`, the table shows the value of j, the state, the list it is taken from, and the operation performed.

j	state	from	Action
1	$(0,0)$	L_0	$0 > \pi = -1$: append
1	$(2,3)$	M_1	$3 > \pi = 0$: append
2	$(0,0)$	L_1	$0 > \pi = -1$: append
2	$(2,3)$	L_1	$3 > \pi = 0$: append
2	$(3,3)$	M_2	$3 \leq \pi = 3$: discard (dominated)
2	$(5,6)$	M_2	$6 > \pi = 3$: append
3	$(0,0)$	L_2	$0 > \pi = -1$: append
3	$(2,3)$	L_2	$3 > \pi = 0$: append
3	$(3,4)$	M_3	$4 > \pi = 3$: append
3	$(5,7)$	M_3	$7 > \pi = 4$: append
3	$(5,6)$	L_2	$6 \leq \pi = 7$: discard (dominated)
3	$(8,10)$	M_3	$10 > \pi = 7$: append

The last loop scans L_3 and keeps the profit of the last state whose weight does not exceed $c = 5$: the states $(0,0)$, $(2,3)$, $(3,4)$ and $(5,7)$ give $z \leftarrow 0$, $z \leftarrow 3$, $z \leftarrow 4$ and $z \leftarrow 7$, while the state $(8,10)$ causes no update since $8 > 5$. Therefore the optimal value of the KP instance is 7.

  - After the execution of the DP algorithm, the list L contains the non-dominated states L_n, sorted by strictly increasing weight and strictly increasing profit.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, i.e., the binary vector $(x_1, x_2, \ldots, x_n)$, it is necessary to store some additional information. The list L_n alone gives the optimal value, but

it does not say which items produce the state that attains it.

- For each item j and each position $h \in \{1, 2, \ldots, |L_j|\}$, in addition to the state $L_j[h]$, we store two values:
  - $\sigma_j[h]$, which is equal to 1 if item j is selected in the state $L_j[h]$, and is equal to 0 otherwise;
  - $\rho_j[h]$, which is the position in L_{j-1} of the state from which $L_j[h]$ is obtained.
- A state taken from L_{j-1} gives $\sigma_j[h] = 0$, while a state taken from M_j gives $\sigma_j[h] = 1$. In both cases $\rho_j[h]$ is the position of the state of L_{j-1} that generated it.
- The DP algorithm must also keep all the lists $L_0, L_1, \ldots, L_n$, and not only the last one, because the reconstruction goes backwards through them.

Example 15.22: Computation of the additional information

We consider the KP instance of Example 15.13. For each item j and each position h, the table shows the state $L_j[h]$ and the stored values $\sigma_j[h]$ and $\rho_j[h]$.

j	h	$L_j[h]$	$\sigma_j[h]$	$\rho_j[h]$
1	1	$(0,0)$	0	1
1	2	$(2,3)$	1	1
2	1	$(0,0)$	0	1
2	2	$(2,3)$	0	2
2	3	$(5,6)$	1	2
3	1	$(0,0)$	0	1
3	2	$(2,3)$	0	2
3	3	$(3,4)$	1	1
3	4	$(5,7)$	1	2
3	5	$(8,10)$	1	3

For instance, the state $L_3[4] = (5, 7)$ has $\sigma_3[4] = 1$ and $\rho_3[4] = 2$: it is obtained by selecting item 3 from the state $L_2[2] = (2, 3)$, since $2 + w_3 = 2 + 3 = 5$ and $3 + p_3 = 3 + 4 = 7$.

- The reconstruction starts from the position $h^\star$ of the state that attains the maximum of formula (15.7), that is the position at which the last loop of `DP2_Lists` updates z for the last time. It is not in general the last position of L_n, because the last states of L_n may have a total weight larger than the knapsack capacity.
- An optimal solution is then reconstructed by scanning the items from $j = n$ down to $j = 1$, starting from $h = h^\star$:

$$x_j \leftarrow \sigma_j[h], \quad h \leftarrow \rho_j[h] \quad \text{for } j = n, n-1, \ldots, 1.$$

This yields the vector $(x_1, x_2, \ldots, x_n)$, where x_j is equal to 1 if item j is selected in an optimal solution, and 0 otherwise.

Algorithm

Algorithm: DP2_Lists_Reconstruct

Input: The number n of items, the position $h^\star$ of the optimal state in L_n, and the stored values σ and ρ

Output: An optimal solution of the KP instance

```
1  Function DP2_Lists_Reconstruct(n, h*, σ, ρ)
2      x := new array[1..n]
3      for j := 1 to n do
4          x[j] ← 0
5      end
6      h := h*
7      for j := n to 1 do
8          x[j] ← σ_j[h]
9          h ← ρ_j[h]
10     end
11     return x
```

Example 15.23: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 15.13, the last list is

$$L_3 = \{(0,0),(2,3),(3,4),(5,7),(8,10)\}$$

and the maximum of formula (15.7) is attained by the state $L_3[4] = (5,7)$, so that $h^\star = 4$ and the optimal value is 7.

We can reconstruct an optimal solution by starting from $j = 3$ and $h = h^\star = 4$ as follows:

Iteration	j	h	$L_j[h]$	$\sigma_j[h]$	$\rho_j[h]$	$x[1]$	$x[2]$	$x[3]$	Action
–	–	–	–	–	–	0	0	0	$h = 4$
1	3	4	$(5,7)$	1	2	0	0	1	$h \leftarrow 2$
2	2	2	$(2,3)$	0	2	0	0	1	$h \leftarrow 2$
3	1	2	$(2,3)$	1	1	1	0	1	$h \leftarrow 1$

Thus, the vector

$$(1,0,1)$$

is an optimal solution. The corresponding optimal value is $3 + 4 = 7$.

15.1.3.3 Number of steps

- Let $S(n,P)$ denote the number of steps executed by `DP2_Lists` in the worst case, where n is the number of items and $P = \sum_{j=1}^{n} p_j$ is the total profit of all the items.

- The states of a list have strictly increasing profits, and every profit belongs to $\{0, 1, \ldots, P\}$: therefore $|L_j| \leq P + 1$ for all $j \in \{0, 1, \ldots, n\}$, and $|M_j| = |L_{j-1}|$.

- For each item, `DP2_Lists` scans L_{j-1} once to build M_j and then calls `Merge`, which scans $|L_{j-1}| + |M_j| \leq 2(P + 1)$ states and performs a constant number of operations for each of them. The final loop scans L_n once. Therefore

$$S(n, P) = \Theta(nP).$$

- The matrix u always has $(n+1)(P+1)$ cells, most of which are equal to $+\infty$, while the lists contain only the states that are actually reachable. On the KP instance of Example 15.13 the lists contain 1, 2, 3, and 5 states, against the 44 cells of the matrix u.

- The list L_j also satisfies $|L_j| \leq 2^j$, since the items $\{1, 2, \ldots, j\}$ have 2^j subsets. Therefore

$$|L_j| \leq \min\left\{2^j,\ P + 1\right\}.$$

15.1.4 Typical mistakes

Warning

- **Updating the single vector in increasing order of capacity.** The cell $v[k - w_j]$ would already contain row j, and item j could be selected more than once; the capacities are scanned from c downwards.

- **Forgetting the base cases.** With no item available the optimal value is 0 for every capacity: $v(0, k) = 0$ for all $k \in \{0, 1, \ldots, c\}$, not only for $k = 0$.

- **Reconstructing with the wrong index.** At every step the item index decreases by one, while the capacity decreases by w_j only if item j is selected.

- **Calling $\Theta(nc)$ polynomial.** The capacity c is written with $\log c$ digits, so the running time is exponential in the size of the input.

- **Reading $u(n, q)$ as the optimal value.** In the second DP algorithm $u(n, q)$ is a minimum weight; the optimal value is the largest q with $u(n, q) \leq c$.

15.1.5 In summary

Summary

- The recurrence $v(j,k)$ chooses between leaving item j out and taking it; it has the optimal substructure and overlapping subproblems.
- The matrix is filled bottom-up in $\Theta(nc)$ steps; a single vector scanned downwards gives the optimal value with only $c+1$ cells.
- An optimal solution comes from the matrix s of the decisions, read backwards from (n,c).
- The running time $\Theta(nc)$ is pseudo-polynomial; the lists of non-dominated states store only the states that can be reached.
- Exchanging profits and weights gives $\Theta(nP)$ steps, which is better when the total profit P is small.

Dashboard of the KP example. Example 1 ($c=5$, three items) with the bounds found so far:

Method	Lower bound	Upper bound	Origin
DP algorithm	14	14	optimal value, $\Theta(nc)$ steps

15.2 A DP algorithm for the Unbounded Knapsack Problem

Prerequisites

- From this chapter, the DP algorithm for the KP: the recurrence $v(j,k)$ over items and capacities, and the reconstruction of a solution.
- From this module, the chapter «Principles of dynamic programming»: optimal substructure, overlapping subproblems, base cases.
- From the module *Algorithms*: pseudocode conventions and the size of the input.

Objectives

At the end of this chapter you will be able to:

- explain why the UKP differs from the KP, and write the recurrence $v(k)$ over the capacities only;
- run the DP algorithm on the capacities and on the pairs (item type, capacity), by hand;
- reconstruct an optimal solution, including the case in which part of the capacity is left unused;
- count the steps of the two DP algorithms and compare them.

15.2.1 Problem definition

Definition 15.2: Unbounded Knapsack Problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n item types,
3. for each item type $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Unbounded Knapsack Problem** (UKP) asks to decide how many copies of each item type to insert into the knapsack, so that the total weight does not exceed the capacity and the total profit is maximized. Each item type is available in infinitely many copies.

Example 15.24: UKP instance

We consider a knapsack of capacity $c = 7$ and 4 item types, with the following profits and weights:

	$j=1$	$j=2$	$j=3$	$j=4$
p_j	4	5	10	11
w_j	2	3	5	6

We refer to this UKP instance as Example 1.

For each item type j, a feasible solution can use at most $\lfloor c/w_j \rfloor$ copies of item type j, since using more copies would already exceed the capacity c with item type j alone. Accordingly, the number of feasible solutions is at most $\prod_{j=1}^{n} (\lfloor c/w_j \rfloor + 1)$.

- The number of possible UKP solutions grows quickly with the number n of item types and the capacity c, since each variable x_j can independently take any of $\lfloor c/w_j \rfloor + 1$ values. Therefore, enumerating all possible solutions quickly becomes impractical as n or c increases.

15.2.1.1 Encoding of feasible solutions

A feasible solution is described by the integer vector

$$(x_1, x_2, \ldots, x_n),$$

where $x_j \in \mathbb{Z}_{\geq 0}$ is the number of copies of item type j selected. The solution is feasible if

$$\sum_{j=1}^{n} w_j x_j \leq c.$$

Its total profit is

$$\sum_{j=1}^{n} p_j x_j.$$

The main difference with the 0–1 Knapsack Problem is that the variables are no longer binary. In UKP, each variable x_j can take any nonnegative integer value, provided that the capacity constraint is respected.

15.2.2 A first DP algorithm

- Step 1: computation of the optimal value

  For any $k \in \{0, 1, \ldots, c\}$, let $v(k)$ be the optimal value of the UKP subinstance given by:

  1. a knapsack of capacity k,
  2. the set $\{1, 2, \ldots, n\}$ of n item types,
  3. for each item type $j \in \{1, 2, \ldots, n\}$, a profit p_j and a weight w_j.

  The recursive formula to compute $v(k)$ reads as follows:

$$v(k) = \begin{cases} 0, & \text{if } k = 0, \\ \max\limits_{j\,:\,w_j \le k} \{p_j + v(k - w_j)\}, & \text{if } k \ge 1 \text{ and } w_j \le k \text{ for some } j, \\ 0, & \text{if } k \ge 1 \text{ and } w_j > k \text{ for every } j. \end{cases} \tag{15.8}$$

  - The recursive decomposition process of formula (15.8) generates at most n **independent subproblems**, each corresponding to a possible choice of an item type j, among those with $w_j \le k$, to be inserted as one of the items of the knapsack of capacity k. Here, $v(0) = 0$ since a knapsack of capacity 0 cannot contain any item and has profit 0. Moreover, if $w_j > k$ for every item type j, no item fits into the knapsack: the only feasible choice is to insert no item, and therefore $v(k) = 0$.
  - The optimal value $v(k)$ is obtained by trying every item type j with $w_j \le k$ and selecting the one that maximizes the sum $p_j + v(k - w_j)$ of the profit of the chosen item type and the optimal value of the residual subproblem. Since item type j is available in infinitely many copies, it may also be chosen again while solving the residual subproblem of capacity $k - w_j$.
  - Hence, an optimal solution for a knapsack of capacity k is built by combining the chosen item type j with an optimal solution of the residual subproblem of capacity $k - w_j$.

$\rightsquigarrow$ **Optimal Substructure verified!**

– Different choices of the first item type, possibly made for different values of k, can lead to the same residual capacity, and therefore the same value $v(k)$ may need to be computed several times during the recursive evaluation of formula (15.8).

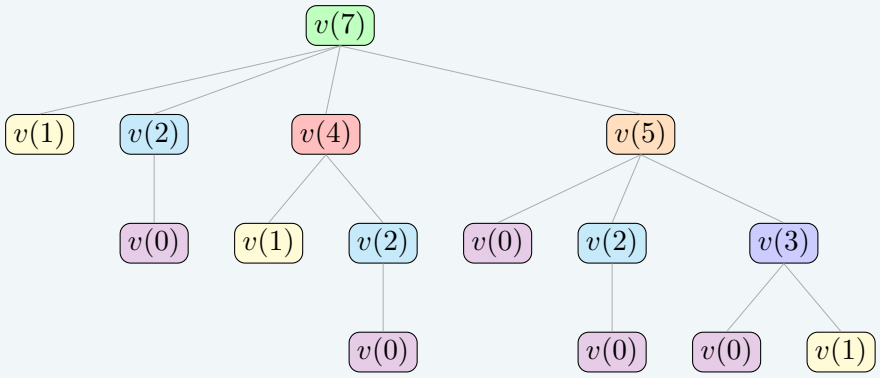
$\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(c)$ of the UKP instance is the **top-down approach**. The method starts from the call with $k = c$ and evaluates formula (15.8) recursively: each call for a knapsack of capacity k triggers further calls for capacity $k - w_j$, for every item type j with $w_j \leq k$, until the base case $v(0)$ is reached or no item type fits into the residual capacity.

– The **recursion tree** of the function v has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 15.25: Recursion tree

We consider the UKP instance of Example 1, with capacity $c = 7$ and item types with weights $w_1 = 2$, $w_2 = 3$, $w_3 = 5$, $w_4 = 6$. This is the recursion tree of the top-down approach to compute $v(7)$:



The function v is called 15 times, and the same subproblems are solved multiple times: $v(2)$ is called 3 times, $v(1)$ is called 3 times, and $v(0)$ is called 5 times. All the other calls are made only once.

– The value $v(k)$, for a generic capacity k, can be computed once the values $v(k')$ are known for all smaller capacities $k' < k$.

Example 15.26: Computing the values of the v function

We consider the UKP instance of Example 1 with a knapsack of capacity $c = 7$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$	$j=4$
p_j	4	5	10	11
w_j	2	3	5	6

The value of the function v can be computed by starting from the base case $v(0) = 0$ and then computing the values of $v(1), v(2), \ldots, v(7)$ in order, as follows:

* Since no item type has $w_j \leq 1$, we have:

$$v(1) = 0.$$

* Once we have computed $v(0)$, we can compute $v(2)$, as follows:

$$v(2) = p_1 + v(0) = 4 + 0 = 4.$$

* Once we have computed $v(1)$ and $v(0)$, we can compute $v(3)$, as follows:

$$v(3) = \max\{p_1 + v(1), p_2 + v(0)\} = \max\{4 + 0, 5 + 0\} = 5.$$

* Once we have computed $v(2)$ and $v(1)$, we can compute $v(4)$, as follows:

$$v(4) = \max\{p_1 + v(2), p_2 + v(1)\} = \max\{4 + 4, 5 + 0\} = 8.$$

* Once we have computed $v(3)$, $v(2)$ and $v(0)$, we can compute $v(5)$, as follows:

$$v(5) = \max\{p_1 + v(3), p_2 + v(2), p_3 + v(0)\} = \max\{4+5, 5+4, 10+0\} = 10.$$

* Once we have computed $v(4)$, $v(3)$, $v(1)$ and $v(0)$, we can compute $v(6)$, as follows:

$$\begin{aligned} v(6) &= \max\{p_1 + v(4), p_2 + v(3), p_3 + v(1), p_4 + v(0)\} \\ &= \max\{4 + 8, 5 + 5, 10 + 0, 11 + 0\} = 12. \end{aligned}$$

* Once we have computed $v(5)$, $v(4)$, $v(2)$ and $v(1)$, we can compute $v(7)$, as follows:

$$\begin{aligned} v(7) &= \max\{p_1 + v(5), p_2 + v(4), p_3 + v(2), p_4 + v(1)\} \\ &= \max\{4 + 10, 5 + 8, 10 + 4, 11 + 0\} = 14. \end{aligned}$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving a subproblem with value $v(k)$ depends only on subproblems with values $v(k')$, with $k' < k$, we can solve the subproblems for increasing values of k, starting from the base case $v(0) = 0$ and building up to $v(c)$: this observation naturally leads

to a **much more efficient method** to compute the optimal value $v(c)$ of the UKP instance.

– A **second method** to compute the optimal value $v(c)$ of the UKP instance is the **bottom-up approach**. The method starts from the base case $v(0) = 0$ and evaluates formula (15.8) for increasing capacities $k \in \{1, 2, \ldots, c\}$: each value $v(k)$ is computed using the already-computed values $v(0), v(1), \ldots, v(k-1)$, until the value $v(c)$ of the UKP instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: UKP_DP1

Input: A UKP instance given by n, arrays $p[1..n]$, $w[1..n]$, and capacity c
Output: The optimal value of the UKP instance

```
1  Function UKP_DP1(n, p, w, c)
2      v := new array[0..c]
3      v[0] ← 0
4      for k := 1 to c do
5          m := 0
6          for j := 1 to n do
7              if w[j] ≤ k then
8                  if p[j] + v[k − w[j]] > m then
9                      m ← p[j] + v[k − w[j]]
10                 end
11             end
12         end
13         v[k] ← m
14     end
15     return v[c]
```

* The vector $v[0..c]$ is the **lookup table**: it stores the optimal values of all subproblems, with the base case $v[0] = 0$.
* The outer loop, over $k \in \{1, 2, \ldots, c\}$, computes $v[k]$ in increasing order of capacity.
* The inner loop, over $j \in \{1, 2, \ldots, n\}$, examines every item type; the condition $w[j] \leq k$ checks whether item type j fits into capacity k, and keeps in m the largest value $p[j] + v[k - w[j]]$ found so far. If no item type fits, m remains equal to its initial value 0.
* At the end of the inner loop, $v[k] \leftarrow m$ stores the optimal value $v[k]$.
* Finally, the optimal value $v[c]$ of the UKP instance is returned.

Example 15.27: Execution of the DP algorithm

We consider the UKP instance of Example 1 with a knapsack of capacity $c = 7$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$	$j=4$
p_j	4	5	10	11
w_j	2	3	5	6

For each step of the DP algorithm, the table shows the values of k and j, the current value of m, and the cell $v[k]$. The cell of m is highlighted in red when updated; the cell of $v[k]$ is highlighted in blue when assigned.

k	j	$v[k]$	m	Action
–	–	0	–	$v[0] \leftarrow 0$
	–	–	0	$m := 0$
1	1	–	0	$w[1] = 2 > 1$: skip
	2	–	0	$w[2] = 3 > 1$: skip
	3	–	0	$w[3] = 5 > 1$: skip
	4	–	0	$w[4] = 6 > 1$: skip
	–	0	0	$v[1] \leftarrow m = 0$
	–	–	0	$m := 0$
2	1	–	4	$p[1]+v[0] = 4+0 = 4 > m = 0$: $m \leftarrow 4$
	2	–	4	$w[2] = 3 > 2$: skip
	3	–	4	$w[3] = 5 > 2$: skip
	4	–	4	$w[4] = 6 > 2$: skip
	–	4	4	$v[2] \leftarrow m = 4$
	–	–	0	$m := 0$
3	1	–	4	$p[1]+v[1] = 4+0 = 4 > m = 0$: $m \leftarrow 4$
	2	–	5	$p[2]+v[0] = 5+0 = 5 > m = 4$: $m \leftarrow 5$
	3	–	5	$w[3] = 5 > 3$: skip
	4	–	5	$w[4] = 6 > 3$: skip
	–	5	5	$v[3] \leftarrow m = 5$
	–	–	0	$m := 0$
4	1	–	8	$p[1]+v[2] = 4+4 = 8 > m = 0$: $m \leftarrow 8$
	2	–	8	$p[2]+v[1] = 5+0 = 5 \leq m = 8$: no update
	3	–	8	$w[3] = 5 > 4$: skip
	4	–	8	$w[4] = 6 > 4$: skip
	–	8	8	$v[4] \leftarrow m = 8$
	–	–	0	$m := 0$
5	1	–	9	$p[1]+v[3] = 4+5 = 9 > m = 0$: $m \leftarrow 9$
	2	–	9	$p[2]+v[2] = 5+4 = 9 = m$: no update
	3	–	10	$p[3]+v[0] = 10+0 = 10 > m = 9$: $m \leftarrow 10$
	4	–	10	$w[4] = 6 > 5$: skip
	–	10	10	$v[5] \leftarrow m = 10$

k	j	$v[k]$	m	Action
	–	–	0	$m := 0$
6	1	–	12	$p[1]+v[4] = 4+8 = 12 > m = 0$: $m \leftarrow 12$
	2	–	12	$p[2]+v[3] = 5+5 = 10 \leq m = 12$: no update
	3	–	12	$p[3]+v[1] = 10+0 = 10 \leq m = 12$: no update
	4	–	12	$p[4]+v[0] = 11+0 = 11 \leq m = 12$: no update
	–	12	12	$v[6] \leftarrow m = 12$
	–	–	0	$m := 0$
7	1	–	14	$p[1]+v[5] = 4+10 = 14 > m = 0$: $m \leftarrow 14$
	2	–	14	$p[2]+v[4] = 5+8 = 13 \leq m = 14$: no update
	3	–	14	$p[3]+v[2] = 10+4 = 14 = m$: no update
	4	–	14	$p[4]+v[1] = 11+0 = 11 \leq m = 14$: no update
	–	14	14	$v[7] \leftarrow m = 14$

This is the vector v after the execution of the DP algorithm:

	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$
$v[k]$	0	0	4	5	8	10	12	14

- After the execution of the DP algorithm, the vector v contains the optimal value $v[k]$ for every capacity.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, it is necessary to store some additional information.
  - For each capacity k, in addition to the optimal value $v[k]$, we store $s[k]$, the item type selected as first item in an optimal solution for a knapsack of capacity k; if no item type fits into capacity k, we set $s[k] = 0$.
  - Starting from $k = c$, the value $s[k]$ gives the item type of the first selected item; the residual capacity becomes $k - w_{s[k]}$, and the same rule can be applied again, and so on, until the residual capacity reaches 0 or no item type fits into it ($s[k] = 0$), in which case the residual capacity is left unused.

    This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to compute an optimal solution:

Algorithm

Algorithm: UKP_DP1_InfoSol

Input: A UKP instance given by n, arrays $p[1..n]$, $w[1..n]$, and capacity c
Output: The optimal value of the UKP instance and the information vector $s[0..c]$

```
1  Function UKP_DP1_InfoSol(n, p, w, c)
2      v := new array[0..c]; s := new array[0..c]
3      v[0] ← 0; s[0] ← 0
4      for k := 1 to c do
5          m := 0; t := 0
6          for j := 1 to n do
7              if w[j] ≤ k then
8                  if p[j] + v[k − w[j]] > m then
9                      m ← p[j] + v[k − w[j]]; t ← j
10                 end
11             end
12         end
13         v[k] ← m; s[k] ← t
14     end
15     return v[c], s
```

∗ The DP algorithm uses the additional temporary variable t to keep track of the decision made when m is updated. If item type j is selected, $t \leftarrow j$ stores this decision; if no item type fits into capacity k, t keeps its initial value 0. At the end of the inner loop, $s[k] \leftarrow t$ stores the selected item type, together with the optimal value $v[k] \leftarrow m$.

Example 15.28: Computation of the additional information

We consider the UKP instance of Example 1 with a knapsack of capacity $c = 7$ and the following profits and weights:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
p_j	4	5	10	11
w_j	2	3	5	6

For each step of the DP algorithm, the table shows the values of k and j, the cell $s[k]$, the current value of m, and the current value of t. The cell of m is highlighted in red when updated; the cell of t and the cell of $s[k]$ are highlighted in blue when updated.

k	j	$s[k]$	m	t	Action
–	–	–	–	–	Initial state: $v[0] \leftarrow 0$, $s[0] \leftarrow 0$
	–	–	0	0	$m := 0$, $t := 0$
1	1	–	0	0	$w[1] = 2 > 1$: skip
	2	–	0	0	$w[2] = 3 > 1$: skip
	3	–	0	0	$w[3] = 5 > 1$: skip
	4	–	0	0	$w[4] = 6 > 1$: skip
	–	0	0	0	$v[1] \leftarrow m = 0$, $s[1] \leftarrow t = 0$
	–	–	0	0	$m := 0$, $t := 0$
2	1	–	4	1	$4{+}0 = 4 > m = 0$: $m \leftarrow 4$, $t \leftarrow 1$
	2	–	4	1	$w[2] = 3 > 2$: skip
	3	–	4	1	$w[3] = 5 > 2$: skip
	4	–	4	1	$w[4] = 6 > 2$: skip
	–	1	4	1	$v[2] \leftarrow m = 4$, $s[2] \leftarrow t = 1$
	–	–	0	0	$m := 0$, $t := 0$
3	1	–	4	1	$4{+}0 = 4 > m = 0$: $m \leftarrow 4$, $t \leftarrow 1$
	2	–	5	2	$5{+}0 = 5 > m = 4$: $m \leftarrow 5$, $t \leftarrow 2$
	3	–	5	2	$w[3] = 5 > 3$: skip
	4	–	5	2	$w[4] = 6 > 3$: skip
	–	2	5	2	$v[3] \leftarrow m = 5$, $s[3] \leftarrow t = 2$
	–	–	0	0	$m := 0$, $t := 0$
4	1	–	8	1	$4{+}4 = 8 > m = 0$: $m \leftarrow 8$, $t \leftarrow 1$
	2	–	8	1	$5{+}0 = 5 \leq m = 8$: no update
	3	–	8	1	$w[3] = 5 > 4$: skip
	4	–	8	1	$w[4] = 6 > 4$: skip
	–	1	8	1	$v[4] \leftarrow m = 8$, $s[4] \leftarrow t = 1$
	–	–	0	0	$m := 0$, $t := 0$
5	1	–	9	1	$4{+}5 = 9 > m = 0$: $m \leftarrow 9$, $t \leftarrow 1$
	2	–	9	1	$5{+}4 = 9 = m$: no update
	3	–	10	3	$10{+}0 = 10 > m = 9$: $m \leftarrow 10$, $t \leftarrow 3$
	4	–	10	3	$w[4] = 6 > 5$: skip
	–	3	10	3	$v[5] \leftarrow m = 10$, $s[5] \leftarrow t = 3$

k	j	$s[k]$	m	t	Action
	–	–	0	0	$m := 0$, $t := 0$
6	1	–	12	1	$4{+}8 = 12 > m = 0$: $m \leftarrow 12$, $t \leftarrow 1$
	2	–	12	1	$5{+}5 = 10 \leq m = 12$: no update
	3	–	12	1	$10{+}0 = 10 \leq m = 12$: no update
	4	–	12	1	$11{+}0 = 11 \leq m = 12$: no update
	–	1	12	1	$v[6] \leftarrow m = 12$, $s[6] \leftarrow t = 1$
	–	–	0	0	$m := 0$, $t := 0$
7	1	–	14	1	$4{+}10 = 14 > m = 0$: $m \leftarrow 14$, $t \leftarrow 1$
	2	–	14	1	$5{+}8 = 13 \leq m = 14$: no update
	3	–	14	1	$10{+}4 = 14 = m$: no update
	4	–	14	1	$11{+}0 = 11 \leq m = 14$: no update
	–	1	14	1	$v[7] \leftarrow m = 14$, $s[7] \leftarrow t = 1$

This is the vector s after the execution of the DP algorithm:

	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$
$s[k]$	0	0	1	2	1	3	1	1

– Once the vector s has been computed, an optimal solution of the UKP instance can be computed by repeatedly applying the following rule, starting from $k = c$ and $x_j = 0$ for all $j \in \{1, 2, \ldots, n\}$:

$$x_{s[k]} \leftarrow x_{s[k]} + 1, \qquad k \leftarrow k - w_{s[k]},$$

until $k = 0$ or $s[k] = 0$. The second case occurs when no item type fits into the residual capacity k, which is then left unused. This yields the vector $(x_1, x_2, \ldots, x_n)$, where each entry x_j is the number of copies of item type j in an optimal solution.

– The following pseudo-code implements the computation of an optimal solution, given the vector s.

Algorithm

Algorithm: UKP_DP1_SolComputation

Input: The number n of item types, the capacity c, the array $w[1..n]$, and the vector $s[0..c]$

Output: An optimal solution of the UKP instance

```
1  Function UKP_DP1_SolComputation(n, w, c, s)
2      x := new array[1..n]
3      for j := 1 to n do
4          x[j] ← 0
5      end
6      k := c
7      while k > 0 and s[k] > 0 do
8          x[s[k]] ← x[s[k]] + 1
9          k ← k − w[s[k]]
10     end
11     return x
```

Example 15.29: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, we have the following vector s:

	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$
$s[k]$	0	0	1	2	1	3	1	1

We can compute an optimal solution for a knapsack of capacity $c = 7$ as follows:

Iteration	k	$s[k]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$k \leftarrow k - w_{s[k]}$
–	–	–	0	0	0	0	$k = 7$
1	7	1	1	0	0	0	$k = 7 - 2 = 5$
2	5	3	1	0	1	0	$k = 5 - 5 = 0$

An optimal solution is the vector:

$$(1, 0, 1, 0)$$

Its total weight is $w_1 + w_3 = 2 + 5 = 7 = c$, and its total profit is $p_1 + p_3 = 4 + 10 = 14$, which matches the optimal value $v(7) = 14$.

15.2.2.1 Number of steps

- Let $S(n, c)$ denote the number of steps executed by `UKP_DP1` in the worst case, where n is the number of item types and c is the knapsack capacity. We compute $S(n, c)$ by counting how many times the lines of the pseudo-code of `UKP_DP1` are executed.

- In the worst case, the conditions at lines 7 and 8 are true at every iteration of the inner loop, and therefore line 9 is executed as often as line 8. The number of executions of the relevant lines is:

  – Line 2 is executed 1 time.
  – Line 3 is executed 1 time.
  – Line 4 is executed c times.
  – Line 5 is executed c times.
  – Line 6 is executed cn times.
  – Line 7 is executed cn times.
  – Line 8 is executed cn times.
  – Line 9 is executed cn times.
  – Line 13 is executed c times.
  – Line 15 is executed 1 time.

  Therefore, the total number of steps in the worst case is

  $$1 + 1 + c + c + cn + cn + cn + cn + c + 1.$$

  Thus,

  $$S(n, c) = 3 + 3c + 4cn.$$

  Hence,

  $$S(n, c) = \Theta(nc).$$

  Indeed,

  $$\lim_{n,c\to\infty} \frac{4cn + 3c + 3}{nc} = \lim_{n,c\to\infty} \left(4 + \frac{3}{n} + \frac{3}{nc}\right) = 4 \in \mathbb{R}_{>0}.$$

The DP algorithm executes

$$S(n, c) = \Theta(nc)$$

steps in the worst case.

- The version `UKP_DP1_InfoSol` stores the additional vector s and performs only a constant number of extra operations inside the same loops. Therefore, its number of steps remains $\Theta(nc)$.

15.2.3 A second DP algorithm

- Step 1: computation of the optimal value

For any $j \in \{0, 1, \ldots, n\}$ and $k \in \{0, 1, \ldots, c\}$, let $v(j, k)$ be the optimal value of the UKP subinstance given by:

1. a knapsack of capacity k,
2. the set $\{1, 2, \ldots, j\}$ of j item types,
3. for each item type $h \in \{1, 2, \ldots, j\}$, a profit p_h and a weight w_h.

The recursive formula to compute $v(j, k)$ reads as follows:

$$v(j,k) = \begin{cases} 0, & \text{if } j = 0, \\ v(j-1,k), & \text{if } j \geq 1 \text{ and } k < w_j, \\ \max\{v(j-1,k),\ p_j + v(j,k-w_j)\}, & \text{if } j \geq 1 \text{ and } k \geq w_j. \end{cases} \tag{15.9}$$

  - The recursive decomposition process of formula (15.9) generates at most 2 **independent subproblems**, corresponding to whether or not to insert one copy of item type j into a knapsack of capacity k, using only item types $1, 2, \ldots, j$. If $w_j > k$, item type j cannot be inserted, and therefore the only feasible option is not to use it. Here, $v(0, k) = 0$ for every k, since with no item type available the only feasible choice is to insert no item, which always has profit 0.
  - If $j \geq 1$ and $k \geq w_j$, the optimal value $v(j, k)$ is obtained by trying the two possible options, not inserting item type j or inserting one further copy of it, and selecting the one that maximizes the profit. The corresponding values are $v(j-1, k)$ and $p_j + v(j, k-w_j)$, respectively; the term $p_j + v(j, k-w_j)$ refers to the same row j, because after inserting one copy of item type j, additional copies of the same item type are still available. If $j \geq 1$ and $k < w_j$, item type j cannot be used, and therefore $v(j, k) = v(j-1, k)$.

– Hence, an optimal solution for a knapsack of capacity k using only item types $1, 2, \ldots, j$ is built by combining the chosen decision on item type j with an optimal solution of the corresponding residual subproblem.

$\rightsquigarrow$ **Optimal Substructure verified!**

– Different sequences of decisions on the item types, possibly made for different values of j and k, can lead to the same subproblem, and therefore the same value $v(j, k)$ may need to be computed several times during the recursive evaluation of formula (15.9).

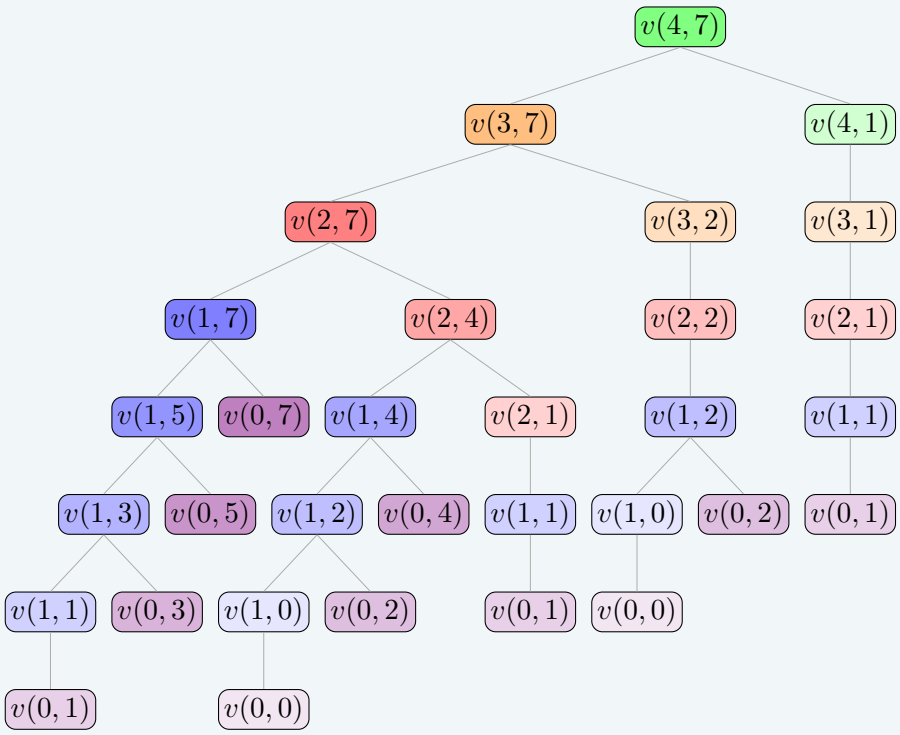
$\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(n, c)$ of the UKP instance is the **top-down approach**. The method starts from the call with $(j, k) = (n, c)$ and evaluates formula (15.9) recursively: each call for capacity k using only item types $1, 2, \ldots, j$ triggers further calls corresponding to the feasible decisions on item type j, until one of the base cases $v(0, k)$ is reached.

– The **recursion tree** of the function v has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 15.30: Recursion tree

We consider the UKP instance of Example 1, with capacity $c = 7$ and item types with weights $w_1 = 2$, $w_2 = 3$, $w_3 = 5$, $w_4 = 6$. This is the recursion tree of the top-down approach to compute $v(4, 7)$:



The function v is called 32 times, and the same subproblems are solved multiple times: $v(1,1)$ is called 3 times, $v(0,1)$ is called 3 times, $v(1,2)$ is called 2 times, $v(0,2)$ is called 2 times, $v(1,0)$ is called 2 times, $v(0,0)$ is called 2 times, and $v(2,1)$ is called 2 times. All the other calls are made only once.

– The value $v(j,k)$ can be computed once $v(j-1,k)$ is known and, when $k \geq w_j$, $v(j,k-w_j)$ is also known. Therefore, for a fixed value of j, all values $v(j,k)$ can be computed by considering the values of k in increasing order. Starting from the base case $v(0,k) = 0$ for every k, the values of the function v can be computed row by row, moving to the next value of j only after all values $v(j,k)$ for the current value of j have been computed.

Example 15.31: Computing the values of the v function

We consider the UKP instance of Example 1 with a knapsack of capacity $c = 7$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$	$j=4$
p_j	4	5	10	11
w_j	2	3	5	6

The values of the function v are computed as follows:

* For $j = 1$, only item type 1 ($w_1 = 2$) is available. Therefore:

$$v(1,0) = 0, \qquad v(1,1) = v(0,1) = 0,$$

$$v(1,2) = \max\{v(0,2), p_1 + v(1,0)\} = \max\{0, 4+0\} = 4,$$

$$v(1,3) = \max\{v(0,3), p_1 + v(1,1)\} = \max\{0, 4+0\} = 4,$$
$$v(1,4) = \max\{v(0,4), p_1 + v(1,2)\} = \max\{0, 4+4\} = 8,$$
$$v(1,5) = \max\{v(0,5), p_1 + v(1,3)\} = \max\{0, 4+4\} = 8,$$
$$v(1,6) = \max\{v(0,6), p_1 + v(1,4)\} = \max\{0, 4+8\} = 12,$$
$$v(1,7) = \max\{v(0,7), p_1 + v(1,5)\} = \max\{0, 4+8\} = 12.$$

* For $j = 2$, item types 1 and 2 are available. We obtain:

$$v(2,0) = 0, \qquad v(2,1) = v(1,1) = 0, \qquad v(2,2) = v(1,2) = 4,$$
$$v(2,3) = \max\{v(1,3), p_2 + v(2,0)\} = \max\{4, 5+0\} = 5,$$
$$v(2,4) = \max\{v(1,4), p_2 + v(2,1)\} = \max\{8, 5+0\} = 8,$$
$$v(2,5) = \max\{v(1,5), p_2 + v(2,2)\} = \max\{8, 5+4\} = 9,$$
$$v(2,6) = \max\{v(1,6), p_2 + v(2,3)\} = \max\{12, 5+5\} = 12,$$
$$v(2,7) = \max\{v(1,7), p_2 + v(2,4)\} = \max\{12, 5+8\} = 13.$$

* For $j = 3$, item types 1, 2 and 3 are available. We obtain:

$$v(3,0) = 0, \qquad v(3,1) = v(2,1) = 0, \qquad v(3,2) = v(2,2) = 4,$$
$$v(3,3) = v(2,3) = 5, \qquad v(3,4) = v(2,4) = 8,$$
$$v(3,5) = \max\{v(2,5), p_3 + v(3,0)\} = \max\{9, 10+0\} = 10,$$
$$v(3,6) = \max\{v(2,6), p_3 + v(3,1)\} = \max\{12, 10+0\} = 12,$$
$$v(3,7) = \max\{v(2,7), p_3 + v(3,2)\} = \max\{13, 10+4\} = 14.$$

* For $j = 4$, all item types are available. We obtain:

$$v(4,0) = 0, \quad v(4,1) = v(3,1) = 0, \quad v(4,2) = v(3,2) = 4,$$
$$v(4,3) = v(3,3) = 5, \quad v(4,4) = v(3,4) = 8, \quad v(4,5) = v(3,5) = 10,$$
$$v(4,6) = \max\{v(3,6), p_4 + v(4,0)\} = \max\{12, 11+0\} = 12,$$
$$v(4,7) = \max\{v(3,7), p_4 + v(4,1)\} = \max\{14, 11+0\} = 14.$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving any particular subproblem depends only on subproblems with a smaller value of j, or with the same value of j and a smaller capacity, we can solve the subproblems in increasing order of j and, for each fixed value of j, in increasing order of capacity, starting from the base cases $v(0,k) = 0$ for every $k \in \{0, 1, \ldots, c\}$ and building up to $v(n,c)$: this observation naturally leads to a **much more efficient method** to compute the optimal value $v(n,c)$ of the UKP instance.

– A **second method** to compute the optimal value $v(n, c)$ of the UKP instance is the **bottom-up approach**. The method starts from the base cases $v(0, k) = 0$ for every $k \in \{0, 1, \ldots, c\}$ and evaluates formula (15.9) for increasing values of $j \in \{1, 2, \ldots, n\}$ and $k \in \{0, 1, \ldots, c\}$: each value $v(j, k)$ is computed using already-computed values, until the value $v(n, c)$ of the UKP instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: UKP_DP2

Input: A UKP instance given by n, arrays $p[1..n]$, $w[1..n]$, and capacity c
Output: The optimal value of the UKP instance

```
1  Function UKP_DP2(n, p, w, c)
2      v := new array[0..n][0..c]
3      for k := 0 to c do
4          v[0][k] ← 0
5      end
6      for j := 1 to n do
7          for k := 0 to c do
8              if k < w[j] then
9                  v[j][k] ← v[j − 1][k]
10             else
11                 if p[j] + v[j][k − w[j]] > v[j − 1][k] then
12                     v[j][k] ← p[j] + v[j][k − w[j]]
13                 else
14                     v[j][k] ← v[j − 1][k]
15                 end
16             end
17         end
18     end
19     return v[n][c]
```

* The matrix $v[0..n][0..c]$ is the **lookup table**: it stores the optimal values of all subproblems.
* The initialization sets $v[0][k] = 0$ for every capacity $k \in \{0, 1, \ldots, c\}$, since selecting no item is always feasible with profit 0.
* The outer loop, over $j \in \{1, 2, \ldots, n\}$, considers the available item types in increasing order.
* The inner loop, over $k \in \{0, 1, \ldots, c\}$, computes the best value for each capacity using only item types $1, 2, \ldots, j$.
* If $k < w[j]$, item type j cannot be used, and therefore $v[j][k] \leftarrow v[j - 1][k]$.

* Otherwise, the DP algorithm compares the value obtained by not using item type j, namely $v[j-1][k]$, with the value obtained by inserting one further copy of item type j, namely $p[j]+v[j][k-w[j]]$.

* Finally, the optimal value of the UKP instance is returned as $v[n][c]$.

Example 15.32: Execution of the DP algorithm

We consider the UKP instance of Example 1 with a knapsack of capacity $c=7$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$	$j=4$
p_j	4	5	10	11
w_j	2	3	5	6

For each step of the DP algorithm, the table shows the values of j and k, the cell $v[j][k]$, and the operation performed.

j	k	$v[j][k]$	Action
0	0	0	$v[0][0] \leftarrow 0$
0	1	0	$v[0][1] \leftarrow 0$
0	2	0	$v[0][2] \leftarrow 0$
0	3	0	$v[0][3] \leftarrow 0$
0	4	0	$v[0][4] \leftarrow 0$
0	5	0	$v[0][5] \leftarrow 0$
0	6	0	$v[0][6] \leftarrow 0$
0	7	0	$v[0][7] \leftarrow 0$
1	0	0	$k=0<w[1]=2$: $v[1][0] \leftarrow v[0][0]=0$
1	1	0	$k=1<w[1]=2$: $v[1][1] \leftarrow v[0][1]=0$
1	2	4	$\max\{v[0][2], p[1]+v[1][0]\} = \max\{0, 4+0\} = 4$
1	3	4	$\max\{v[0][3], p[1]+v[1][1]\} = \max\{0, 4+0\} = 4$
1	4	8	$\max\{v[0][4], p[1]+v[1][2]\} = \max\{0, 4+4\} = 8$
1	5	8	$\max\{v[0][5], p[1]+v[1][3]\} = \max\{0, 4+4\} = 8$
1	6	12	$\max\{v[0][6], p[1]+v[1][4]\} = \max\{0, 4+8\} = 12$
1	7	12	$\max\{v[0][7], p[1]+v[1][5]\} = \max\{0, 4+8\} = 12$
2	0	0	$k=0<w[2]=3$: $v[2][0] \leftarrow v[1][0]=0$
2	1	0	$k=1<w[2]=3$: $v[2][1] \leftarrow v[1][1]=0$
2	2	4	$k=2<w[2]=3$: $v[2][2] \leftarrow v[1][2]=4$
2	3	5	$\max\{v[1][3], p[2]+v[2][0]\} = \max\{4, 5+0\} = 5$
2	4	8	$\max\{v[1][4], p[2]+v[2][1]\} = \max\{8, 5+0\} = 8$
2	5	9	$\max\{v[1][5], p[2]+v[2][2]\} = \max\{8, 5+4\} = 9$
2	6	12	$\max\{v[1][6], p[2]+v[2][3]\} = \max\{12, 5+5\} = 12$
2	7	13	$\max\{v[1][7], p[2]+v[2][4]\} = \max\{12, 5+8\} = 13$

j	k	$v[j][k]$	Action
3	0	0	$k = 0 < w[3] = 5$: $v[3][0] \leftarrow v[2][0] = 0$
3	1	0	$k = 1 < w[3] = 5$: $v[3][1] \leftarrow v[2][1] = 0$
3	2	4	$k = 2 < w[3] = 5$: $v[3][2] \leftarrow v[2][2] = 4$
3	3	5	$k = 3 < w[3] = 5$: $v[3][3] \leftarrow v[2][3] = 5$
3	4	8	$k = 4 < w[3] = 5$: $v[3][4] \leftarrow v[2][4] = 8$
3	5	10	$\max\{v[2][5], p[3] + v[3][0]\} = \max\{9, 10 + 0\} = 10$
3	6	12	$\max\{v[2][6], p[3] + v[3][1]\} = \max\{12, 10 + 0\} = 12$
3	7	14	$\max\{v[2][7], p[3] + v[3][2]\} = \max\{13, 10 + 4\} = 14$
4	0	0	$k = 0 < w[4] = 6$: $v[4][0] \leftarrow v[3][0] = 0$
4	1	0	$k = 1 < w[4] = 6$: $v[4][1] \leftarrow v[3][1] = 0$
4	2	4	$k = 2 < w[4] = 6$: $v[4][2] \leftarrow v[3][2] = 4$
4	3	5	$k = 3 < w[4] = 6$: $v[4][3] \leftarrow v[3][3] = 5$
4	4	8	$k = 4 < w[4] = 6$: $v[4][4] \leftarrow v[3][4] = 8$
4	5	10	$k = 5 < w[4] = 6$: $v[4][5] \leftarrow v[3][5] = 10$
4	6	12	$\max\{v[3][6], p[4] + v[4][0]\} = \max\{12, 11 + 0\} = 12$
4	7	14	$\max\{v[3][7], p[4] + v[4][1]\} = \max\{14, 11 + 0\} = 14$

This is the matrix v after the execution of the DP algorithm:

$v[j][k]$	$k = 0$	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k = 7$
$j = 0$	0	0	0	0	0	0	0	0
$j = 1$	0	0	4	4	8	8	12	12
$j = 2$	0	0	4	5	8	9	12	13
$j = 3$	0	0	4	5	8	10	12	14
$j = 4$	0	0	4	5	8	10	12	14

  - After the execution of the DP algorithm, the matrix v contains the optimal value $v[j][k]$ for every subproblem (j, k).

- Step 2: computation of an optimal solution
  - To compute an optimal solution, it is necessary to store some additional information.
  - For each item type j and each capacity k, in addition to the optimal value $v[j][k]$, we store $s[j][k]$, which is equal to 1 if one copy of item type j is used when computing $v[j][k]$, and to 0 otherwise.
  - Starting from $j = n$ and $k = c$, if $s[j][k] = 0$, then item type j is not used and j is decreased by 1. If $s[j][k] = 1$, then one copy of item type j is used and the residual capacity becomes $k - w_j$. The procedure stops when the residual capacity reaches 0.

    This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to compute an optimal solution:

Algorithm

Algorithm: UKP_DP2_InfoSol

Input: A UKP instance given by n, arrays $p[1..n]$, $w[1..n]$, and capacity c
Output: The optimal value of the UKP instance and the matrix of binary information $s[0..n][0..c]$

```
1  Function UKP_DP2_InfoSol(n, p, w, c)
2      v := new array[0..n][0..c]; s := new array[0..n][0..c]
3      for k := 0 to c do
4          v[0][k] ← 0; s[0][k] ← 0
5      end
6      for j := 1 to n do
7          for k := 0 to c do
8              if k < w[j] then
9                  v[j][k] ← v[j − 1][k]; s[j][k] ← 0
10             else
11                 if p[j] + v[j][k − w[j]] > v[j − 1][k] then
12                     v[j][k] ← p[j] + v[j][k − w[j]]; s[j][k] ← 1
13                 else
14                     v[j][k] ← v[j − 1][k]; s[j][k] ← 0
15                 end
16             end
17         end
18     end
19     return v[n][c], s
```

∗ The DP algorithm uses the additional matrix s, which keeps track of the decision made when $v[j][k]$ is computed. If one copy of item type j is selected, $s[j][k] \leftarrow 1$ stores this decision; otherwise, $s[j][k] \leftarrow 0$ stores that item type j is not used.

Example 15.33: Computation of the additional information

We consider the UKP instance of Example 1 with a knapsack of capacity $c = 7$ and the following profits and weights:

	$j=1$	$j=2$	$j=3$	$j=4$
p_j	4	5	10	11
w_j	2	3	5	6

For each step of the DP algorithm, the table shows the values of j and k, the cell $s[j][k]$, and the operation performed.

j	k	$s[j][k]$	Action
0	0	0	$s[0][0] \leftarrow 0$
0	1	0	$s[0][1] \leftarrow 0$
0	2	0	$s[0][2] \leftarrow 0$
0	3	0	$s[0][3] \leftarrow 0$
0	4	0	$s[0][4] \leftarrow 0$
0	5	0	$s[0][5] \leftarrow 0$
0	6	0	$s[0][6] \leftarrow 0$
0	7	0	$s[0][7] \leftarrow 0$
1	0	0	$k < w[1]$: $s[1][0] \leftarrow 0$
1	1	0	$k < w[1]$: $s[1][1] \leftarrow 0$
1	2	1	$p[1] + v[1][0] = 4 + 0 > v[0][2] = 0$: $s[1][2] \leftarrow 1$
1	3	1	$p[1] + v[1][1] = 4 + 0 > v[0][3] = 0$: $s[1][3] \leftarrow 1$
1	4	1	$p[1] + v[1][2] = 4 + 4 > v[0][4] = 0$: $s[1][4] \leftarrow 1$
1	5	1	$p[1] + v[1][3] = 4 + 4 > v[0][5] = 0$: $s[1][5] \leftarrow 1$
1	6	1	$p[1] + v[1][4] = 4 + 8 > v[0][6] = 0$: $s[1][6] \leftarrow 1$
1	7	1	$p[1] + v[1][5] = 4 + 8 > v[0][7] = 0$: $s[1][7] \leftarrow 1$
2	0	0	$k < w[2]$: $s[2][0] \leftarrow 0$
2	1	0	$k < w[2]$: $s[2][1] \leftarrow 0$
2	2	0	$k < w[2]$: $s[2][2] \leftarrow 0$
2	3	1	$p[2] + v[2][0] = 5 + 0 > v[1][3] = 4$: $s[2][3] \leftarrow 1$
2	4	0	$p[2] + v[2][1] = 5 + 0 \leq v[1][4] = 8$: $s[2][4] \leftarrow 0$
2	5	1	$p[2] + v[2][2] = 5 + 4 > v[1][5] = 8$: $s[2][5] \leftarrow 1$
2	6	0	$p[2] + v[2][3] = 5 + 5 \leq v[1][6] = 12$: $s[2][6] \leftarrow 0$
2	7	1	$p[2] + v[2][4] = 5 + 8 > v[1][7] = 12$: $s[2][7] \leftarrow 1$

j	k	$s[j][k]$	Action
3	0	0	$k < w[3]$: $s[3][0] \leftarrow 0$
3	1	0	$k < w[3]$: $s[3][1] \leftarrow 0$
3	2	0	$k < w[3]$: $s[3][2] \leftarrow 0$
3	3	0	$k < w[3]$: $s[3][3] \leftarrow 0$
3	4	0	$k < w[3]$: $s[3][4] \leftarrow 0$
3	5	1	$p[3] + v[3][0] = 10 + 0 > v[2][5] = 9$: $s[3][5] \leftarrow 1$
3	6	0	$p[3] + v[3][1] = 10 + 0 \leq v[2][6] = 12$: $s[3][6] \leftarrow 0$
3	7	1	$p[3] + v[3][2] = 10 + 4 > v[2][7] = 13$: $s[3][7] \leftarrow 1$
4	0	0	$k < w[4]$: $s[4][0] \leftarrow 0$
4	1	0	$k < w[4]$: $s[4][1] \leftarrow 0$
4	2	0	$k < w[4]$: $s[4][2] \leftarrow 0$
4	3	0	$k < w[4]$: $s[4][3] \leftarrow 0$
4	4	0	$k < w[4]$: $s[4][4] \leftarrow 0$
4	5	0	$k < w[4]$: $s[4][5] \leftarrow 0$
4	6	0	$p[4] + v[4][0] = 11 + 0 \leq v[3][6] = 12$: $s[4][6] \leftarrow 0$
4	7	0	$p[4] + v[4][1] = 11 + 0 \leq v[3][7] = 14$: $s[4][7] \leftarrow 0$

This is the matrix s after the execution of the DP algorithm:

$s[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$
$j=0$	0	0	0	0	0	0	0	0
$j=1$	0	0	1	1	1	1	1	1
$j=2$	0	0	0	1	0	1	0	1
$j=3$	0	0	0	0	0	1	0	1
$j=4$	0	0	0	0	0	0	0	0

– Once the matrix s has been computed, an optimal solution for a knapsack of capacity c can be computed by repeatedly applying the following rule, starting from $(j,k) = (n,c)$ and $x_h = 0$ for all $h \in \{1,2,\ldots,n\}$:

$$\begin{aligned} &j \leftarrow j-1 && \text{if } s[j][k]=0,\\ &x_j \leftarrow x_j+1, \quad k \leftarrow k-w_j && \text{if } s[j][k]=1. \end{aligned}$$

The procedure stops when $k=0$ or $j=0$; the second case occurs when the residual capacity is left unused. This yields the vector $(x_1,x_2,\ldots,x_n)$, where each entry x_h is the number of copies of item type h in an optimal solution.

– The following pseudo-code implements the computation of an optimal solution, given the matrix s.

Algorithm

Algorithm: UKP_DP2_SolComputation

Input: The number of item types n, the array $w[1..n]$, the capacity c, and the matrix $s[0..n][0..c]$

Output: An optimal solution of the UKP instance

```
1  Function UKP_DP2_SolComputation(n, w, c, s)
2      x := new array[1..n]
3      for h := 1 to n do
4          x[h] ← 0
5      end
6      j := n
7      k := c
8      while k > 0 and j > 0 do
9          if s[j][k] = 0 then
10             j ← j − 1
11         else
12             x[j] ← x[j] + 1
13             k ← k − w[j]
14         end
15     end
16     return x
```

Example 15.34: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, we have the following matrix s:

$s[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$
$j=0$	0	0	0	0	0	0	0	0
$j=1$	0	0	1	1	1	1	1	1
$j=2$	0	0	0	1	0	1	0	1
$j=3$	0	0	0	0	0	1	0	1
$j=4$	0	0	0	0	0	0	0	0

We can compute an optimal solution for a knapsack of capacity $c=7$ as follows:

Iteration	j	k	$s[j][k]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	Action
1	4	7	0	0	0	0	0	$j \leftarrow 3$
2	3	7	1	0	0	1	0	$x[3] \leftarrow 1$, $k \leftarrow 7-5=2$
3	3	2	0	0	0	1	0	$j \leftarrow 2$
4	2	2	0	0	0	1	0	$j \leftarrow 1$
5	1	2	1	1	0	1	0	$x[1] \leftarrow 1$, $k \leftarrow 2-2=0$

An optimal solution is the vector:

$$(1,0,1,0)$$

Its total weight is $w_1 + w_3 = 2+5 = 7 = c$, and its total profit is $p_1 + p_3 = 4+10 = 14$, which matches the optimal value $v(4,7) = 14$.

15.2.3.1 Number of steps

- Let $S(n,c)$ denote the number of steps executed by `UKP_DP2` in the worst case, where n is the number of item types and c is the knapsack capacity. We compute $S(n,c)$ by counting how many times the lines of the pseudo-code of `UKP_DP2` are executed.

- In the worst case, line 11 is checked whenever the item type fits. Since the item weights are positive integers, this happens at most nc times. The number of executions of the relevant lines is:
  - Line 2 is executed 1 time.
  - Line 3 is executed $c+1$ times.
  - Line 4 is executed $c+1$ times.
  - Line 6 is executed n times.
  - Line 7 is executed $n(c+1)$ times.
  - Line 8 is executed $n(c+1)$ times.
  - Line 11 is executed at most nc times.

- one among Line 9, Line 12, and Line 14 is executed for each pair (j, k), and therefore these assignment lines are executed $n(c+1)$ times in total.
- Line 19 is executed 1 time.

Therefore, the total number of steps in the worst case is

$$1 + (c+1) + (c+1) + n + n(c+1) + n(c+1) + nc + n(c+1) + 1.$$

Thus,

$$S(n, c) = 4nc + 4n + 2c + 4.$$

Hence,

$$S(n, c) = \Theta(nc).$$

Indeed,

$$\lim_{n,c\to\infty} \frac{4nc + 4n + 2c + 4}{nc} = \lim_{n,c\to\infty} \left(4 + \frac{4}{c} + \frac{2}{n} + \frac{4}{nc}\right) = 4 \in \mathbb{R}_{>0}.$$

The DP algorithm executes

$$S(n, c) = \Theta(nc)$$

steps in the worst case.

- The version `UKP_DP2_InfoSol` stores the additional matrix s and performs only a constant number of extra operations inside the same loops. Therefore, its number of steps remains $\Theta(nc)$.

15.2.4 Typical mistakes

Warning

- **Using row $j-1$ in the second recurrence.** The term $p_j + v(j, k - w_j)$ refers to the same row j: this is what allows several copies of item type j.
- **Assuming that the capacity is filled.** When no item type fits, $v(k)$ is not $-\infty$ but the value of a smaller capacity; in Example 1, $v(1) = 0$.
- **Reconstructing past the end.** The reconstruction stops when the residual capacity is 0, or when $s[k] = 0$ in the first DP algorithm and $j = 0$ in the second one; otherwise it reads outside the arrays.
- **Forgetting the base case.** $v(0) = 0$, and $v[0..c]$ has $c+1$ cells, not c.

15.2.5 In summary

Summary

- In the UKP an item type can be used any number of times, so the optimal value depends only on the residual capacity: $v(k) = \max_{j:\, w_j \le k}\{p_j + v(k - w_j)\}$.
- The first DP algorithm fills a vector of $c+1$ cells in $\Theta(nc)$ steps; the second one fills the matrix $v(j,k)$ and reuses the same row for further copies.
- An optimal solution is read backwards from the information stored, stopping when the residual capacity is zero or unused.
- The running time is pseudo-polynomial, as for the KP.

Dashboard of the UKP example. Example 1 ($c = 7$, four item types):

Method	Lower bound	Upper bound	Origin
DP algorithm	14	14	optimal value, solution $(1, 0, 1, 0)$

15.3 A DP algorithm for the Bounded Knapsack Problem

Prerequisites

- From this chapter, the DP algorithm for the KP and the one for the UKP: the recurrence $v(j,k)$ over items and capacities, and the reconstruction of a solution.
- From this module, the chapter «Principles of dynamic programming»: optimal substructure, overlapping subproblems, base cases.
- From the module *Algorithms*: pseudocode conventions and the size of the input.

Objectives

At the end of this chapter you will be able to:

- say how the BKP sits between the KP ($b_j = 1$) and the UKP (no bound), and write its recurrence;
- run the DP algorithm by hand, trying every feasible number d of copies of each item type;
- fill the matrix s of the numbers of copies and reconstruct an optimal solution from it;
- count the steps of the DP algorithm and say when $\Theta(nc\,b_{\max})$ is tight.

15.3.1 Problem definition

Definition 15.3: Bounded Knapsack Problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n item types,
3. for each item type $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$, and a *bound* $b_j \in \mathbb{Z}_{\geq 1}$,

the **Bounded Knapsack Problem** (BKP) asks to decide how many copies of each item type to insert into the knapsack, so that the total weight does not exceed the capacity and the total profit is maximized. Item type j is available in at most b_j copies.

Example 15.35: BKP instance

We consider a knapsack of capacity $c = 10$ and 3 item types, with the following profits, weights, and bounds:

	$j=1$	$j=2$	$j=3$
p_j	6	10	12
w_j	2	3	5
b_j	2	2	1

We refer to this BKP instance as Example 1.

The main difference with the Knapsack Problem is that each item type may be selected more than once: variable x_j is no longer binary, but it is bounded by b_j. The BKP reduces to the 0–1 Knapsack Problem when $b_j = 1$ for every item type j, and to the Unbounded Knapsack Problem when every bound b_j is large enough not to be binding.

- The number of possible vectors $(x_1, x_2, \ldots, x_n)$, with $x_j \in \{0, 1, \ldots, b_j\}$ for every item type j, is
$$\prod_{j=1}^{n} (b_j + 1).$$
Some of these vectors may not respect the knapsack capacity. Therefore, the number of feasible solutions is at most $\prod_{j=1}^{n}(b_j + 1)$, a quantity that grows exponentially with n. Enumerating all possible solutions quickly becomes impractical as n increases.

15.3.1.1 Encoding of feasible solutions

A *feasible solution* of the BKP is a choice of the number of copies of each item type, respecting both the bounds b_j and the knapsack capacity c.

- A BKP feasible solution can be described by the integer vector

$$(x_1, x_2, \ldots, x_n)$$

where each component has the following meaning:

$$x_j = \text{number of copies of item type } j \text{ selected}, \qquad \forall j \in \{1, 2, \ldots, n\}, \quad 0 \leq x_j \leq b_j.$$

Since the solution is feasible, the total weight of the selected items must not exceed the knapsack capacity:

$$\sum_{j=1}^{n} w_j x_j \leq c.$$

The total profit of a feasible solution is

$$\sum_{j=1}^{n} p_j x_j.$$

15.3.2 A DP algorithm

- Step 1: computation of the optimal value

For any $j \in \{0, 1, \ldots, n\}$ and $k \in \{0, 1, \ldots, c\}$, let $v(j, k)$ be the optimal value of the BKP subinstance given by:

1. a knapsack of capacity k,
2. a set $\{1, 2, \ldots, j\}$ of available item types,
3. for each item type $h \in \{1, 2, \ldots, j\}$, a profit p_h, a weight w_h, and a bound b_h.

The recursive formula to compute $v(j, k)$ reads as follows:

$$v(j,k) = \begin{cases} 0 & \text{if } j = 0 \text{ or } k = 0, \\ \max\limits_{d \in \{0,1,\ldots,\min\{b_j, \lfloor k/w_j \rfloor\}\}} \left\{ d\, p_j + v(j-1, k - d\, w_j) \right\} & \text{if } 1 \leq j \leq n \\ & \text{and } 1 \leq k \leq c. \end{cases} \tag{15.10}$$

– The recursive decomposition process of formula (15.10) generates at most $b_j + 1$ **independent subproblems**, corresponding to the possible numbers $d \in \{0, 1, \ldots, \min\{b_j, \lfloor k/w_j \rfloor\}\}$ of copies of item type j inserted into a knapsack of capacity k, using only item types $1, 2, \ldots, j$. The base cases are $v(0, k) = 0$ since no item type is available, and $v(j, 0) = 0$ since no capacity is available.

– The optimal value $v(j, k)$ is obtained by trying every feasible number d of copies of item type j and selecting the one that maximizes the sum $d\,p_j + v(j-1, k-d\,w_j)$ of the profit of the d copies and the optimal value of the residual subproblem, restricted to item types $1, 2, \ldots, j-1$.

– Hence, an optimal solution for the subinstance with capacity k and available item types $\{1, 2, \ldots, j\}$ is built by combining the chosen number d of copies of item type j with an optimal solution of the corresponding residual subproblem.

  $\rightsquigarrow$ **Optimal Substructure verified!**

– Different choices of the number of copies of an item type, possibly made for different values of j and k, can lead to the same subproblem, and therefore the same value $v(j, k)$ may need to be computed several times during the recursive evaluation of formula (15.10).

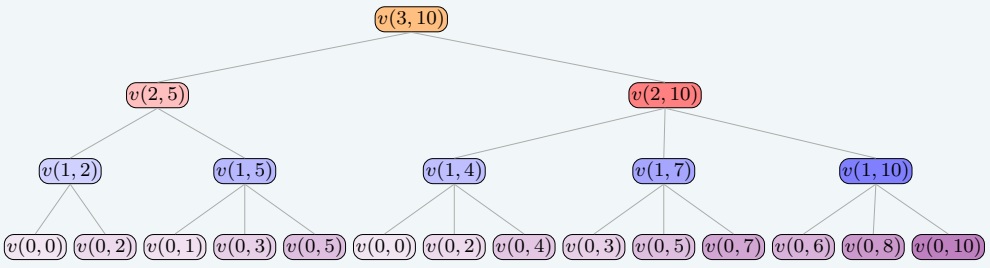
  $\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(n, c)$ of the BKP instance is the **top-down approach**. The method starts from the call with $j = n$ and $k = c$ and evaluates formula (15.10) recursively: each call considers item type j and tries every feasible number d of copies of it, until no item type is left or the residual capacity is equal to 0.

– The **recursion tree** of the function v has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 15.36: Recursion tree

We consider the BKP instance of Example 1. This is the recursion tree of the top-down approach to compute $v(3, 10)$:



The function v is called 22 times, and the same subproblems are solved multiple times: $v(0,5)$ is called 2 times, $v(0,3)$ is called 2 times, $v(0,2)$ is called 2 times, and $v(0,0)$ is called 2 times. All the other calls are made only once.

– The value $v(j,k)$ can be computed once the values $v(j-1,k')$, for every residual capacity $k' \leq k$, are known. Therefore, for a fixed value of j, all values $v(j,k)$ can be computed by considering the capacity values k in increasing order.

Example 15.37: Computing the values of the v function

We consider the BKP instance of Example 1 with a knapsack of capacity $c = 10$ and the following profits, weights, and bounds:

	$j=1$	$j=2$	$j=3$
p_j	6	10	12
w_j	2	3	5
b_j	2	2	1

The values of the function v are computed as follows:

* The base cases give $v(0,k) = 0$ for all $k \geq 0$.
* Once we have computed the base cases, we can compute $v(1,4)$, trying $d = 0,1,2$ copies of item type 1 (since $b_1 = 2$ and $\lfloor 4/2 \rfloor = 2$):

$$v(1,4) = \max\{v(0,4),\ p_1 + v(0,2),\ 2p_1 + v(0,0)\} = \max\{0,\ 6,\ 12\} = 12$$

* We can also compute $v(1,10)$, trying $d = 0,1,2$ copies (since $b_1 = 2$ caps the number of copies even though $\lfloor 10/2 \rfloor = 5$):

$$v(1,10) = \max\{v(0,10),\ p_1 + v(0,8),\ 2p_1 + v(0,6)\} = \max\{0,\ 6,\ 12\} = 12$$

* Once we have computed $v(1,4)$, we can compute $v(2,4)$, trying $d = 0,1$ copies of item type 2 (since $b_2 = 2$ but $\lfloor 4/3 \rfloor = 1$):

$$v(2,4) = \max\{v(1,4),\ p_2 + v(1,1)\} = \max\{12,\ 10+0\} = 12$$

* Once we have computed $v(1,10)$, we can compute $v(2,10)$, trying $d = 0,1,2$ copies of item type 2:

$$\begin{aligned} v(2,10) &= \max\{v(1,10),\ p_2 + v(1,7),\ 2p_2 + v(1,4)\} \\ &= \max\{12,\ 10+12,\ 20+12\} = 32 \end{aligned}$$

* Finally, once we have computed $v(2,10)$, we can compute $v(3,10)$, trying $d = 0,1$ copies of item type 3 (since $b_3 = 1$):

$$v(3,10) = \max\{v(2,10),\ p_3 + v(2,5)\} = \max\{32,\ 12+16\} = 32$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving any particular subproblem depends only on subproblems with a smaller value of j, we can solve the subproblems in increasing order of j and, for each fixed value of j, in increasing order of capacity, starting from the base cases $v(0,k) = 0$ and $v(j,0) = 0$ and building up to $v(n,c)$: this observation naturally leads to a **much more efficient method** to compute the optimal value $v(n,c)$ of the BKP instance.

– A **second method** to compute the optimal value $v(n,c)$ of the BKP instance is the **bottom-up approach**. The method starts from the base cases and evaluates formula (15.10) for increasing values of $j \in \{1,2,\ldots,n\}$ and $k \in \{0,1,\ldots,c\}$: each value $v(j,k)$ is computed using already-computed values, until the value $v(n,c)$ of the BKP instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: BKP_DP

Input: A BKP instance given by n, arrays $p[1..n]$, $w[1..n]$, $b[1..n]$, and capacity c

Output: The optimal value of the BKP instance

```
1  Function BKP_DP(n, p, w, b, c)
2      v := new array[0..n][0..c]
3      for k := 0 to c do
4          v[0][k] ← 0
5      end
6      for j := 1 to n do
7          for k := 0 to c do
8              m := 0
9              for d := 0 to min{b[j], ⌊k/w[j]⌋} do
10                 if d p[j] + v[j − 1][k − d w[j]] > m then
11                     m ← d p[j] + v[j − 1][k − d w[j]]
12                 end
13             end
14             v[j][k] ← m
15         end
16     end
17     return v[n][c]
```

∗ The matrix $v[0..n][0..c]$ is the **lookup table**: it stores the optimal values of all subproblems.

* The first loop initializes the values $v[0][k] = 0$ for all capacities $k \in \{0, 1, \ldots, c\}$, corresponding to the case in which no item type is available.
* The outer loop, over $j \in \{1, 2, \ldots, n\}$, computes the rows of the matrix in increasing order of item type.
* The inner loop, over $k \in \{0, 1, \ldots, c\}$, computes the optimal value $v[j][k]$ for each capacity k.
* The innermost loop, over $d \in \{0, 1, \ldots, \min\{b[j], \lfloor k/w[j] \rfloor\}\}$, tries every feasible number of copies of item type j and keeps in m the largest value found so far.
* At the end of the innermost loop, $v[j][k] \leftarrow m$ stores the optimal value $v[j][k]$.
* Finally, the optimal value of the BKP instance is returned as $v[n][c]$.

Example 15.38: Execution of the DP algorithm

We consider the BKP instance of Example 1 with a knapsack of capacity $c = 10$ and the following profits, weights, and bounds:

	$j = 1$	$j = 2$	$j = 3$
p_j	6	10	12
w_j	2	3	5
b_j	2	2	1

For each step of the DP algorithm, the table shows the values of j and k, the cell $v[j][k]$, and the operation performed.

j	k	$v[j][k]$	Action
0	0	0	$v[0][0] \leftarrow 0$
0	1	0	$v[0][1] \leftarrow 0$
0	2	0	$v[0][2] \leftarrow 0$
0	3	0	$v[0][3] \leftarrow 0$
0	4	0	$v[0][4] \leftarrow 0$
0	5	0	$v[0][5] \leftarrow 0$
0	6	0	$v[0][6] \leftarrow 0$
0	7	0	$v[0][7] \leftarrow 0$
0	8	0	$v[0][8] \leftarrow 0$
0	9	0	$v[0][9] \leftarrow 0$
0	10	0	$v[0][10] \leftarrow 0$
1	0	0	$\max\{v[0][0]\} = 0$
1	1	0	$\max\{v[0][1]\} = 0$
1	2	6	$\max\{v[0][2],\ p[1]+v[0][0]\} = \max\{0, 6+0\} = 6$
1	3	6	$\max\{v[0][3],\ p[1]+v[0][1]\} = \max\{0, 6+0\} = 6$
1	4	12	$\max\{v[0][4],\ p[1]+v[0][2],\ 2p[1]+v[0][0]\} = \max\{0, 6, 12\} = 12$
1	5	12	$\max\{v[0][5],\ p[1]+v[0][3],\ 2p[1]+v[0][1]\} = \max\{0, 6, 12\} = 12$
1	6	12	$\max\{v[0][6],\ p[1]+v[0][4],\ 2p[1]+v[0][2]\} = \max\{0, 6, 12\} = 12$
1	7	12	$\max\{v[0][7],\ p[1]+v[0][5],\ 2p[1]+v[0][3]\} = \max\{0, 6, 12\} = 12$
1	8	12	$\max\{v[0][8],\ p[1]+v[0][6],\ 2p[1]+v[0][4]\} = \max\{0, 6, 12\} = 12$
1	9	12	$\max\{v[0][9],\ p[1]+v[0][7],\ 2p[1]+v[0][5]\} = \max\{0, 6, 12\} = 12$
1	10	12	$\max\{v[0][10],\ p[1]+v[0][8],\ 2p[1]+v[0][6]\} = \max\{0, 6, 12\} = 12$

j	k	$v[j][k]$	Action
2	0	0	$\max\{v[1][0]\} = 0$
2	1	0	$\max\{v[1][1]\} = 0$
2	2	6	$\max\{v[1][2]\} = 6$
2	3	10	$\max\{v[1][3],\ p[2]+v[1][0]\} = \max\{6, 10+0\} = 10$
2	4	12	$\max\{v[1][4],\ p[2]+v[1][1]\} = \max\{12, 10+0\} = 12$
2	5	16	$\max\{v[1][5],\ p[2]+v[1][2]\} = \max\{12, 10+6\} = 16$
2	6	20	$\max\{v[1][6],\ p[2]+v[1][3],\ 2p[2]+v[1][0]\} = \max\{12, 16, 20\} = 20$
2	7	22	$\max\{v[1][7],\ p[2]+v[1][4],\ 2p[2]+v[1][1]\} = \max\{12, 22, 20\} = 22$
2	8	26	$\max\{v[1][8],\ p[2]+v[1][5],\ 2p[2]+v[1][2]\} = \max\{12, 22, 26\} = 26$
2	9	26	$\max\{v[1][9],\ p[2]+v[1][6],\ 2p[2]+v[1][3]\} = \max\{12, 22, 26\} = 26$
2	10	32	$\max\{v[1][10],\ p[2]+v[1][7],\ 2p[2]+v[1][4]\} = \max\{12, 22, 32\} = 32$
3	0	0	$\max\{v[2][0]\} = 0$
3	1	0	$\max\{v[2][1]\} = 0$
3	2	6	$\max\{v[2][2]\} = 6$
3	3	10	$\max\{v[2][3]\} = 10$
3	4	12	$\max\{v[2][4]\} = 12$
3	5	16	$\max\{v[2][5],\ p[3]+v[2][0]\} = \max\{16, 12+0\} = 16$
3	6	20	$\max\{v[2][6],\ p[3]+v[2][1]\} = \max\{20, 12+0\} = 20$
3	7	22	$\max\{v[2][7],\ p[3]+v[2][2]\} = \max\{22, 12+6\} = 22$
3	8	26	$\max\{v[2][8],\ p[3]+v[2][3]\} = \max\{26, 12+10\} = 26$
3	9	26	$\max\{v[2][9],\ p[3]+v[2][4]\} = \max\{26, 12+12\} = 26$
3	10	32	$\max\{v[2][10],\ p[3]+v[2][5]\} = \max\{32, 12+16\} = 32$

This is the matrix v after the execution of the DP algorithm:

$v[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$	$k=8$	$k=9$	$k=10$
$j=0$	0	0	0	0	0	0	0	0	0	0	0
$j=1$	0	0	6	6	12	12	12	12	12	12	12
$j=2$	0	0	6	10	12	16	20	22	26	26	32
$j=3$	0	0	6	10	12	16	20	22	26	26	32

  - After the execution of the DP algorithm, the matrix v contains the optimal value $v[j][k]$ for every item type $j \in \{0, 1, \ldots, n\}$ and every capacity value $k \in \{0, 1, \ldots, c\}$.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, i.e., the integer vector $(x_1, x_2, \ldots, x_n)$, it is necessary to store some additional information.
  - For each item type j and capacity value k, in addition to the optimal value $v[j][k]$, we store $s[j][k]$, the number of copies of item type j selected in an optimal solution for the subproblem restricted to item types $\{1, 2, \ldots, j\}$ and capacity k.

– More precisely, $s[j][k]$ is equal to the value d that maximizes the right-hand side of formula (15.10) when computing $v[j][k]$.

This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to construct an optimal solution:

Algorithm

Algorithm: BKP_DP_InfoSol

Input: A BKP instance given by n, arrays $p[1..n]$, $w[1..n]$, $b[1..n]$, and capacity c

Output: The optimal value of the BKP instance and the matrix of copy counts $s[0..n][0..c]$

```
1  Function BKP_DP_InfoSol(n, p, w, b, c)
2      v := new array[0..n][0..c]; s := new array[0..n][0..c]
3      for k := 0 to c do
4          v[0][k] ← 0; s[0][k] ← 0
5      end
6      for j := 1 to n do
7          for k := 0 to c do
8              m := 0; t := 0
9              for d := 0 to min{b[j], ⌊k/w[j]⌋} do
10                 if d p[j] + v[j − 1][k − d w[j]] > m then
11                     m ← d p[j] + v[j − 1][k − d w[j]]; t ← d
12                 end
13             end
14             v[j][k] ← m; s[j][k] ← t
15         end
16     end
17     return v[n][c], s
```

∗ The DP algorithm uses the additional temporary variable t to keep track of the decision made when m is updated. If d copies of item type j are selected, $t \leftarrow d$ stores this decision. At the end of the innermost loop, $s[j][k] \leftarrow t$ stores the number of copies of item type j used, together with the optimal value $v[j][k] \leftarrow m$.

Example 15.39: Computation of the additional information

We consider the BKP instance of Example 1 with a knapsack of capacity $c = 10$ and the following profits, weights, and bounds:

	$j = 1$	$j = 2$	$j = 3$
p_j	6	10	12
w_j	2	3	5
b_j	2	2	1

For each cell with $j \geq 1$ the table shows the values $d\,p[j] + v[j-1][k - d\,w[j]]$ for $d = 0, 1, \ldots, \min\{b[j], \lfloor k/w[j] \rfloor\}$, in this order, and the number t of copies stored in $s[j][k]$. In case of ties the smallest d is kept, because m is updated only when the value is strictly larger. The row $j = 0$ is $s[0][k] = 0$ for every k.

j	k	$s[j][k]$	Values for $d = 0, 1, \ldots$
1	0	0	0
1	1	0	0
1	2	1	0, 6
1	3	1	0, 6
1	4	2	0, 6, 12
1	5	2	0, 6, 12
1	6	2	0, 6, 12
1	7	2	0, 6, 12
1	8	2	0, 6, 12
1	9	2	0, 6, 12
1	10	2	0, 6, 12
2	0	0	0
2	1	0	0
2	2	0	6
2	3	1	6, 10
2	4	0	12, 10
2	5	1	12, 16
2	6	2	12, 16, 20
2	7	1	12, 22, 20
2	8	2	12, 22, 26
2	9	2	12, 22, 26
2	10	2	12, 22, 32
3	0	0	0
3	1	0	0
3	2	0	6
3	3	0	10
3	4	0	12
3	5	0	16, 12
3	6	0	20, 12
3	7	0	22, 18
3	8	0	26, 22
3	9	0	26, 24
3	10	0	32, 28

This is the matrix s after the execution of the DP algorithm:

$s[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$	$k=8$	$k=9$	$k=10$
$j=0$	0	0	0	0	0	0	0	0	0	0	0
$j=1$	0	0	1	1	2	2	2	2	2	2	2
$j=2$	0	0	0	1	0	1	2	1	2	2	2
$j=3$	0	0	0	0	0	0	0	0	0	0	0

For example, $s[1][4] = 2$ since formula (15.10) selects $d = 2$ copies of item type 1 when computing $v[1][4] = 12$, and $s[2][10] = 2$ since $d = 2$ copies of item type

2 are selected when computing $v[2][10] = 32$. Since $s[3][10] = 0$, no copy of item type 3 is used in the optimal solution for the original instance.

– Once the matrix s has been computed, an optimal solution can be reconstructed by scanning the item types from $j = n$ down to $j = 1$, starting from capacity $k = c$ and $x_j = 0$ for all $j \in \{1, 2, \ldots, n\}$:

$$x_j \leftarrow s[j][k], \qquad k \leftarrow k - s[j][k]\, w_j.$$

This yields the vector $(x_1, x_2, \ldots, x_n)$, where each entry x_j is the number of copies of item type j in an optimal solution.

Algorithm

Algorithm: BKP_Reconstruct

Input: The number n of item types, the knapsack capacity c, the array $w[1..n]$ of weights, and the matrix $s[0..n][0..c]$

Output: An optimal solution of the BKP instance

```
1 Function BKP_Reconstruct(n, c, w, s)
2     x := new array[1..n]
3     k := c
4     for j := n to 1 do
5         x[j] ← s[j][k]
6         k ← k − x[j] w[j]
7     end
8     return x
```

Example 15.40: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, we have the following matrix s:

$s[j][k]$	$k=0$	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$	$k=8$	$k=9$	$k=10$
$j=0$	0	0	0	0	0	0	0	0	0	0	0
$j=1$	0	0	1	1	2	2	2	2	2	2	2
$j=2$	0	0	0	1	0	1	2	1	2	2	2
$j=3$	0	0	0	0	0	0	0	0	0	0	0

We can reconstruct an optimal solution by starting from $j = 3$ and $k = 10$ as follows:

Iteration	j	k	$s[j][k]$	$x[1]$	$x[2]$	$x[3]$	Action
–	–	–	–	0	0	0	$k = 10$
1	3	10	0	0	0	0	$x[3] \leftarrow 0$, $k = 10$
2	2	10	2	0	2	0	$x[2] \leftarrow 2$, $k \leftarrow 10 - 2 \cdot 3 = 4$
3	1	4	2	2	2	0	$x[1] \leftarrow 2$, $k \leftarrow 4 - 2 \cdot 2 = 0$

Thus, the vector

$$(2, 2, 0)$$

is an optimal solution. The corresponding total weight is $2 \cdot 2 + 2 \cdot 3 = 4 + 6 = 10 = c$. The corresponding optimal value is $v(3, 10) = 32$.

15.3.2.1 Number of steps

- Let $b_{\max} = \max\{b[j] : j \in \{1, \ldots, n\}\}$, and let $S(n, c, b_{\max})$ denote the number of steps executed by `BKP_DP` in the worst case, where n is the number of item types and c is the knapsack capacity. We compute $S(n, c, b_{\max})$ by counting how many times the lines of the pseudo-code of `BKP_DP` are executed.

- For each pair (j, k) the innermost loop over d executes $\min\{b_j, \lfloor k/w_j \rfloor\} + 1$ iterations, which is at most $b_{\max} + 1$. We count $b_{\max} + 1$ iterations for every pair (j, k) and we assume that the condition at line 10 is true at every iteration, so that $S(n, c, b_{\max})$ is an upper bound on the number of steps. The number of executions of the relevant lines is, at most:

  - Line 2 is executed 1 time.
  - Line 3 is executed $c + 1$ times.
  - Line 4 is executed $c + 1$ times.
  - Line 6 is executed n times.
  - Line 7 is executed $n(c + 1)$ times.
  - Line 8 is executed $n(c + 1)$ times.
  - Line 9 is executed at most $n(c + 1)(b_{\max} + 1)$ times.
  - Line 10 is executed at most $n(c + 1)(b_{\max} + 1)$ times.
  - Line 11 is executed at most $n(c + 1)(b_{\max} + 1)$ times.
  - Line 14 is executed $n(c + 1)$ times.
  - Line 17 is executed 1 time.

  Therefore, the total number of steps in the worst case is

  $$1 + (c + 1) + (c + 1) + n + n(c + 1) + n(c + 1) + 3n(c + 1)(b_{\max} + 1) + n(c + 1) + 1.$$

  Thus,

  $$S(n, c, b_{\max}) = 3n(c + 1)(b_{\max} + 1) + 3n(c + 1) + n + 2c + 4.$$

  Hence,

  $$S(n, c, b_{\max}) = \Theta(nc\, b_{\max}).$$

  Indeed,

  $$\begin{aligned} &\lim_{n,c,b_{\max} \to \infty} \frac{3n(c + 1)(b_{\max} + 1) + 3n(c + 1) + n + 2c + 4}{nc\, b_{\max}} \\ &= \lim_{n,c,b_{\max} \to \infty} \left(\frac{3(c + 1)(b_{\max} + 1)}{c\, b_{\max}} + \frac{3(c + 1)}{c\, b_{\max}} + \frac{1}{c\, b_{\max}} + \frac{2}{n\, b_{\max}} + \frac{4}{nc\, b_{\max}} \right) \\ &= 3 \in \mathbb{R}_{>0}. \end{aligned}$$

The DP algorithm executes

$$S(n, c, b_{\max}) = \Theta(nc\, b_{\max})$$

steps in the worst case.

- The version `BKP_DP_InfoSol` stores the additional matrix s and performs only a constant number of extra operations inside the same loops. Therefore, its number of steps remains $\Theta(nc\, b_{\max})$.
- Replacing every bound b_j by $\min\{b_j, \lfloor c/w_j \rfloor\}$ does not change the problem, so we can assume $b_{\max} \leq c$. Under this assumption the bound $\Theta(nc\, b_{\max})$ is tight: it is attained by the instances in which all the weights are equal to 1.

15.3.3 Typical mistakes

Warning

- **Trying more than b_j copies.** The number of copies of item type j ranges in $\{0, 1, \ldots, \min\{b_j, \lfloor k/w_j \rfloor\}\}$: the bound and the capacity both matter.
- **Reconstructing with the wrong update.** After reading $s[j][k]$ the capacity decreases by $s[j][k]\, w_j$, not by w_j, and the item type index always decreases by one.
- **Counting the steps with $b_{\max}$ for every cell.** The innermost loop has $\min\{b_j, \lfloor k/w_j \rfloor\} + 1$ iterations, so $b_{\max} + 1$ is only an upper bound.
- **Breaking ties inconsistently.** The DP algorithm updates m only on a strictly larger value, so $s[j][k]$ keeps the smallest number of copies that attains the maximum.

15.3.4 In summary

Summary

- The BKP generalizes the KP ($b_j = 1$) and is a restriction of the UKP; the recurrence tries every number d of copies of item type j.
- The matrix $v(j, k)$ is filled in $\Theta(nc\, b_{\max})$ steps, which is pseudo-polynomial.
- The matrix s stores the number of copies of each item type, and an optimal solution is read backwards from (n, c).
- Replacing b_j by $\min\{b_j, \lfloor c/w_j \rfloor\}$ keeps the problem unchanged and gives $b_{\max} \leq c$.

Dashboard of the BKP example. Example 1 ($c = 10$, three item types):

Method	Lower bound	Upper bound	Origin
DP algorithm	32	32	optimal value, solution $(2, 2, 0)$

Chapter 16

Rod cutting and coin change

16.1 A DP algorithm for the Rod Cutting Problem

Prerequisites

- From this module, the chapter «Principles of dynamic programming»: optimal substructure, overlapping subproblems, recurrence relations, base cases, and the top-down and bottom-up approaches.
- From this module, the chapter «Dynamic programming for knapsack problems»: the pairs (item, capacity) and the reconstruction of a solution, which return here.
- From the module *Algorithms*: how to read pseudocode and how to count the steps of an algorithm.

Objectives

At the end of this chapter you will be able to:

- state the RCP, encode a solution as a vector of numbers of pieces, and explain why cutting patterns and vectors are different objects;
- write the recurrence $v(\ell)$ and explain it, base case included;
- run the DP algorithm by hand with the top-down and the bottom-up approach, and with the running best value m;
- reconstruct an optimal solution from the vector s, for the rod and for every shorter rod;
- write the second recurrence $v(j, \ell)$ on pairs (largest piece length, rod length) and count the steps of the DP algorithm.

16.1.1 Problem definition

Definition 16.1: Rod-Cutting Problem

Given:

1. a rod of *length* $n \in \mathbb{Z}_{\geq 1}$,
2. a set $\{1, 2, \ldots, n\}$ of n possible piece lengths,
3. for each piece length $\ell \in \{1, 2, \ldots, n\}$, a *profit* $p_\ell \in \mathbb{Z}_{\geq 1}$,

the **Rod-Cutting Problem** (RCP) asks to decide how many copies of each piece length to cut, so that the total length of the obtained pieces is equal to the rod length, and the total profit of the obtained pieces is maximized.

Example 16.1: Rod cutting problem

We consider a rod of length $n = 4$ and a set $\{1, 2, 3, 4\}$ of 4 possible piece lengths with the following profits:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
p_ℓ	1	5	8	9

We refer to this RCP instance as Example 1.

The number of possible solutions of the RCP is 2^{n-1} since we can cut a rod of length n in 2^{n-1} different ways, with an independent option of cutting (or not) at each possible cutting position.

- The number of possible RCP solutions grows exponentially with the length n of the rod:

$$2^{n-1} = O(2^n).$$

Therefore, enumerating all possible solutions quickly becomes impractical as n increases.

16.1.1.1 Encoding of feasible solutions

A *feasible solution* of the RCP is a way of cutting the rod into pieces, such that the total length of the pieces is equal to the rod length.

- An RCP feasible solution can be described by an integer solution vector

$$(x_1, x_2, \ldots, x_n)$$

where each component has the following meaning:

$$x_\ell = \text{number of pieces of length } \ell, \qquad \forall \ell \in \{1, 2, \ldots, n\};$$

The number of pieces is

$$\sum_{\ell=1}^{n} x_\ell.$$

Since the solution is feasible, the total length of the pieces must be equal to the rod length:

$$\sum_{\ell=1}^{n} \ell\, x_\ell = n.$$

The total profit of a feasible solution is

$$\sum_{\ell=1}^{n} p_\ell x_\ell.$$

- Notice that two or more cutting patterns may have the same vector representation: the vector records how many pieces of each length are obtained, but it does not specify the order in which the pieces are cut. However, cutting patterns with the same vector representation have the same total profit, since the order in which the pieces are cut does not affect the obtained pieces and therefore does not affect the total profit.

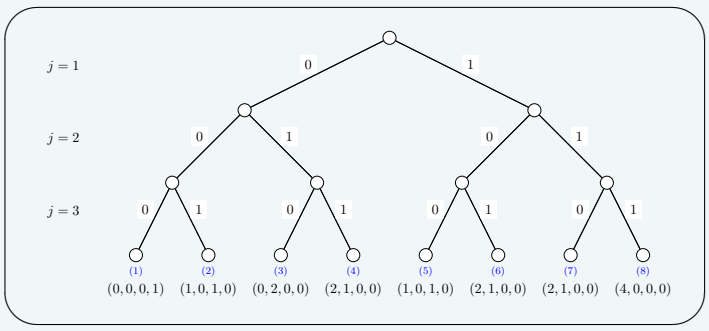
Example 16.2: Enumeration of all possible RCP solutions

We consider the RCP instance of Example 1 with a rod of length $n = 4$ and the profits associated with the possible piece lengths are:

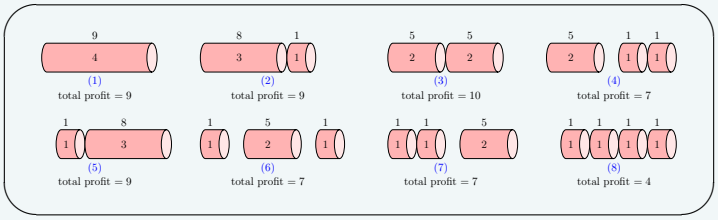
	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
p_ℓ	1	5	8	9

There are $2^{4-1} = 8$ feasible solutions, corresponding to the different ways of cutting the rod.

In the following binary tree, each level corresponds to a possible cutting position: an edge labeled 1 means that the rod is cut at that length, while an edge labeled 0 means that it is not cut at that length. Each leaf represents one complete cutting pattern and reports the corresponding vector.



The figure below shows all possible cutting patterns associated with the leaves of the previous tree, through the labels $(1), (2), \ldots, (8)$. For each obtained piece, the number written inside the piece is its length, while the number above it is its profit.



Label	Solution vector	Number of pieces	Total profit
(1)	$(0, 0, 0, 1)$	1	9
(2)	$(1, 0, 1, 0)$	2	$8 + 1 = 9$
(3)	$(0, 2, 0, 0)$	2	$5 + 5 = 10$
(4)	$(2, 1, 0, 0)$	3	$5 + 1 + 1 = 7$
(5)	$(1, 0, 1, 0)$	2	$1 + 8 = 9$
(6)	$(2, 1, 0, 0)$	3	$1 + 5 + 1 = 7$
(7)	$(2, 1, 0, 0)$	3	$1 + 1 + 5 = 7$
(8)	$(4, 0, 0, 0)$	4	$1 + 1 + 1 + 1 = 4$

16.1.2 A first DP algorithm

- Step 1: computation of the optimal value

For any $\ell \in \{0, 1, \ldots, n\}$, let $v(\ell)$ be the optimal value of the RCP subinstance given by:

1. a rod of length ℓ,
2. a set $\{1, 2, \ldots, \ell\}$ of ℓ possible piece lengths,
3. for each piece length $j \in \{1, 2, \ldots, \ell\}$, a profit p_j.

The recursive formula to compute $v(\ell)$ reads as follows:

$$v(\ell) = \begin{cases} \max\left\{p_j + v(\ell - j) : j \in \{1, 2, \ldots, \ell\}\right\} & \text{if } 1 \leq \ell \leq n, \\ 0 & \text{if } \ell = 0. \end{cases} \tag{16.1}$$

  - The recursive decomposition process of formula (16.1) generates ℓ **independent subproblems**, each corresponding to a possible choice of cutting a first piece of length j from a rod of length ℓ. Here, $v(0) = 0$ since a rod of length 0 cannot be cut and has profit 0.

– The optimal value $v(\ell)$ is obtained by trying all possible first-piece lengths and selecting the one that maximizes the sum $p_j + v(\ell - j)$ of the profit of the first piece and the optimal value of the residual subproblem.

– Hence, an optimal solution for a rod of length ℓ is built by combining the chosen first piece of length j with an optimal solution of the residual subproblem of length $\ell - j$.

$\rightsquigarrow$ **Optimal Substructure verified!**

– Different choices of the first-piece length, possibly made for different values of ℓ, can lead to the same residual length, and therefore the same value $v(\ell)$ may need to be computed several times during the recursive evaluation of formula (16.1).

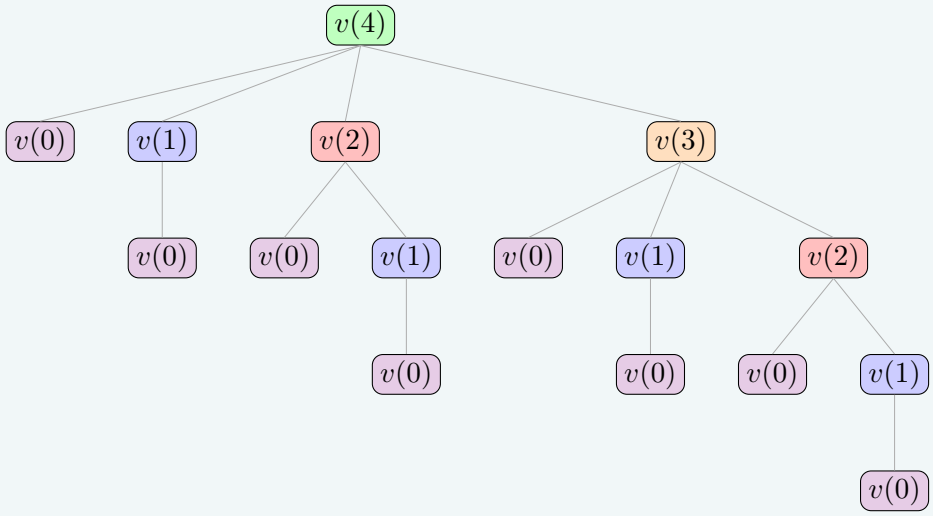
$\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(n)$ of the RCP instance is the **top-down approach**. The method starts from the call with $\ell = n$ and evaluates formula (16.1) recursively: each call for a rod of length ℓ triggers further calls for rods of length $\ell - j$, for all $j \in \{1, 2, \ldots, \ell\}$, until the base case $v(0)$ is reached.

– The **recursion tree** of the function v has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 16.3: Recursion tree

We consider the RCP instance of Example 1. This is the recursion tree of the top-down approach to compute $v(4)$:



The function v is called 16 times, and the same subproblems are solved multiple times: $v(2)$ is called 2 times, $v(1)$ is called 4 times, and $v(0)$ is called 8 times.

All the other calls are made only once.

– The value $v(\ell)$, for a generic rod length ℓ, can be computed once the values $v(j)$ are known for all smaller lengths $j < \ell$.

Example 16.4: Computing the values of the v function

We consider the RCP instance of Example 1 with a rod of length $n = 4$ and the profits associated with the possible piece lengths are:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
p_ℓ	1	5	8	9

The value of the function v can be computed by starting from the base case $v(0) = 0$ and then computing the values of $v(1)$, $v(2)$, $v(3)$, and $v(4)$ in order, as follows:

* Since we know the value of $v(0)$, we can compute $v(1)$, as follows:

$$v(1) = p_1 + v(0) = 1 + 0 = 1$$

* Once we have computed $v(1)$, we can compute $v(2)$, as follows:

$$v(2) = \max\{p_1 + v(1), p_2 + v(0)\} = \max\{1 + 1, 5 + 0\} = 5$$

* Once we have computed $v(2)$ and $v(1)$, we can compute $v(3)$, as follows:

$$v(3) = \max\{p_1 + v(2), p_2 + v(1), p_3 + v(0)\} = \max\{1 + 5, 5 + 1, 8 + 0\} = 8$$

* Once we have computed $v(3)$, $v(2)$ and $v(1)$, we can compute $v(4)$, as follows:

$$\begin{aligned} v(4) &= \max\{p_1 + v(3), p_2 + v(2), p_3 + v(1), p_4 + v(0)\} \\ &= \max\{1 + 8, 5 + 5, 8 + 1, 9 + 0\} = 10 \end{aligned}$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving a subproblem with value $v(\ell)$ depends only on subproblems with values $v(j)$, with $j < \ell$, we can solve the subproblems for increasing values of ℓ, starting from the base case $v(0) = 0$ and building up to $v(n)$: this observation naturally leads to a **much more efficient method** to compute the optimal value $v(n)$ of the RCP instance.

– A **second method** to compute the optimal value $v(n)$ of the RCP instance is the **bottom-up approach**. The method starts from the base case $v(0) = 0$ and evaluates formula (16.1) for increasing rod lengths $\ell \in \{1, 2, \ldots, n\}$: each value $v(\ell)$ is

computed using the already-computed values $v(0), v(1), \ldots, v(\ell-1)$, until the value $v(n)$ of the RCP instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: DP1

Input: An RCP instance given by the length n of the rod and an array $p[1..n]$ of piece profits

Output: The optimal value of the RCP instance

```
1  Function DP1(n, p)
2      v := new array[0..n]
3      v[0] ← 0
4      for ℓ := 1 to n do
5          m := 0
6          for j := 1 to ℓ do
7              if m < p[j] + v[ℓ − j] then
8                  m ← p[j] + v[ℓ − j]
9              end
10         end
11         v[ℓ] ← m
12     end
13     return v[n]
```

* The vector $v[0..n]$ is the **lookup table**: it stores the optimal values of all sub-problems, with the base case $v[0] = 0$.
* The outer loop, over $\ell \in \{1, 2, \ldots, n\}$, computes $v[\ell]$ in increasing order of rod length.
* The inner loop, over $j \in \{1, 2, \ldots, \ell\}$, evaluates all possible first-piece lengths and keeps in m the largest value $p[j] + v[\ell - j]$.
* At the end of the inner loop, $v[\ell] \leftarrow m$ stores the optimal value $v[\ell]$.
* Finally, the optimal value $v[n]$ of the RCP instance is returned.

Example 16.5: Execution of the DP algorithm

We consider the RCP instance of Example 1 with a rod of length $n = 4$ and the profits associated with the possible piece lengths are:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
p_ℓ	1	5	8	9

For each step of the DP algorithm, the table shows the values of ℓ and j, the current value of m, and the cell $v[\ell]$. The cell of m is highlighted in red when updated; the cell of $v[\ell]$ is highlighted in blue when assigned.

ℓ	j	$v[\ell]$	m	Action
–	–	0	–	$v[0] \leftarrow 0$
	–	–	0	$m \leftarrow 0$
1	1	–	1	$p[1]+v[0] = 1+0 = 1 > 0$: $m \leftarrow 1$
	–	1	1	$v[1] \leftarrow m = 1$
	–	–	0	$m \leftarrow 0$
2	1	–	2	$p[1]+v[1] = 1+1 = 2 > 0$: $m \leftarrow 2$
	2	–	5	$p[2]+v[0] = 5+0 = 5 > 2$: $m \leftarrow 5$
	–	5	5	$v[2] \leftarrow m = 5$
	–	–	0	$m \leftarrow 0$
3	1	–	6	$p[1]+v[2] = 1+5 = 6 > 0$: $m \leftarrow 6$
	2	–	6	$p[2]+v[1] = 5+1 = 6 = m$: no update
	3	–	8	$p[3]+v[0] = 8+0 = 8 > 6$: $m \leftarrow 8$
	–	8	8	$v[3] \leftarrow m = 8$
	–	–	0	$m \leftarrow 0$
4	1	–	9	$p[1]+v[3] = 1+8 = 9 > 0$: $m \leftarrow 9$
	2	–	10	$p[2]+v[2] = 5+5 = 10 > 9$: $m \leftarrow 10$
	3	–	10	$p[3]+v[1] = 8+1 = 9 < 10$: no update
	4	–	10	$p[4]+v[0] = 9+0 = 9 < 10$: no update
	–	10	10	$v[4] \leftarrow m = 10$

This is the vector v after the execution of the DP algorithm:

	$\ell = 0$	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$v[\ell]$	0	1	5	8	10

- After the execution of the DP algorithm, the vector v contains the optimal value $v[\ell]$ for every rod length.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, it is necessary to store some additional information.
  - For each rod length ℓ, in addition to the optimal value $v[\ell]$, we store $s[\ell]$, the size of the first piece to cut off in an optimal solution for a rod of length ℓ.
  - Starting from $\ell = n$, the value $s[\ell]$ gives the length of the first piece to cut; the remaining rod has length $\ell - s[\ell]$, and the same rule can be applied again, and so on, until the residual length reaches 0.

    This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to compute an optimal solution:

Algorithm

Algorithm: DP1_InfoSol

Input: An RCP instance given by the length n of the rod and an array $p[1..n]$ of piece profits

Output: The optimal value of the RCP instance and the information vector $s[1..n]$

```
1  Function DP1_InfoSol(n, p)
2      v := new array[0..n]
3      s := new array[1..n]
4      v[0] ← 0
5      for ℓ := 1 to n do
6          m := 0
7          t := 0
8          for j := 1 to ℓ do
9              if m < p[j] + v[ℓ − j] then
10                 m ← p[j] + v[ℓ − j]
11                 t ← j
12             end
13         end
14         v[ℓ] ← m
15         s[ℓ] ← t
16     end
17     return v[n], s
```

∗ The DP algorithm uses the additional temporary variable t to keep track of the decision made when m is updated. If a first piece of length j is selected, $t \leftarrow j$ stores this decision. At the end of the inner loop, $s[\ell] \leftarrow t$ stores the length of the selected first piece, together with the optimal value $v[\ell] \leftarrow m$.

Example 16.6: Computation of the additional information

We consider the RCP instance of Example 1 with a rod of length $n = 4$ and the profits associated with the possible piece lengths are:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
p_ℓ	1	5	8	9

For each step of the DP algorithm, the table shows the values of ℓ and j, the cell $s[\ell]$, the current value of m, and the current value of t. The cell of m is highlighted in red when updated; the cell of t and the cell of $s[\ell]$ are highlighted in blue when updated.

ℓ	j	$s[\ell]$	m	t	Action
–	–	–	–	–	Initial state
1	–	–	0	0	$m \leftarrow 0,\ t \leftarrow 0$
	1	–	1	1	$1{+}0 = 1 > 0$: $m \leftarrow 1,\ t \leftarrow 1$
	–	1	1	1	$v[1] \leftarrow m = 1,\ s[1] \leftarrow t = 1$
2	–	–	0	0	$m \leftarrow 0,\ t \leftarrow 0$
	1	–	2	1	$1{+}1 = 2 > 0$: $m \leftarrow 2,\ t \leftarrow 1$
	2	–	5	2	$5{+}0 = 5 > 2$: $m \leftarrow 5,\ t \leftarrow 2$
	–	2	5	2	$v[2] \leftarrow m = 5,\ s[2] \leftarrow t = 2$
3	–	–	0	0	$m \leftarrow 0,\ t \leftarrow 0$
	1	–	6	1	$1{+}5 = 6 > 0$: $m \leftarrow 6,\ t \leftarrow 1$
	2	–	6	1	$5{+}1 = 6 = m$: no update
	3	–	8	3	$8{+}0 = 8 > 6$: $m \leftarrow 8,\ t \leftarrow 3$
	–	3	8	3	$v[3] \leftarrow m = 8,\ s[3] \leftarrow t = 3$
4	–	–	0	0	$m \leftarrow 0,\ t \leftarrow 0$
	1	–	9	1	$1{+}8 = 9 > 0$: $m \leftarrow 9,\ t \leftarrow 1$
	2	–	10	2	$5{+}5 = 10 > 9$: $m \leftarrow 10,\ t \leftarrow 2$
	3	–	10	2	$8{+}1 = 9 < 10$: no update
	4	–	10	2	$9{+}0 = 9 < 10$: no update
	–	2	10	2	$v[4] \leftarrow m = 10,\ s[4] \leftarrow t = 2$

This is the vector s after the execution of the DP algorithm:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$s[\ell]$	1	2	3	2

– Once the vector s has been computed, an optimal solution of the RCP instance can be computed by repeatedly applying the following rule, starting from $j = n$ and $x_\ell = 0$ for all $\ell \in \{1, 2, \ldots, n\}$:

$$x_{s[j]} \leftarrow x_{s[j]} + 1, \qquad j \leftarrow j - s[j],$$

until $j = 0$. This yields the vector $(x_1, x_2, \ldots, x_n)$, where each entry x_ℓ is the number of pieces of length ℓ in an optimal solution.

– The following pseudo-code implements the computation of an optimal solution, given the vector s.

Algorithm

Algorithm: DP1_SolComputation

Input: The rod length n of the RCP instance and the vector $s[1..n]$
Output: An optimal solution of the RCP instance

```
1  Function DP1_SolComputation(n, s)
2      x := new array[1..n]
3      for ℓ := 1 to n do
4          x[ℓ] ← 0
5      end
6      j := n
7      while j > 0 do
8          x[s[j]] ← x[s[j]] + 1
9          j ← j − s[j]
10     end
11     return x
```

Example 16.7: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, we have the following vector s:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$s[\ell]$	1	2	3	2

We can compute an optimal solution for a rod of length $n = 4$ as follows:

Iteration	j	$s[j]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$j \leftarrow j - s[j]$
–	–	–	0	0	0	0	$j = 4$
1	4	2	0	1	0	0	$j = 4 - 2 = 2$
2	2	2	0	2	0	0	$j = 2 - 2 = 0$

An optimal solution is the vector:

$$(0, 2, 0, 0)$$

– The same computation procedure, based on the vector s, can be applied starting from any rod length ℓ.

Example 16.8: Computing optimal solutions for rod lengths 3 and 2

After the execution of the DP algorithm on the instance of Example 1, we have the following vector s:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$s[\ell]$	1	2	3	2

We can compute an optimal solution for a rod of length $n = 3$ as follows:

Iteration	j	$s[j]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$j \leftarrow j - s[j]$
–	–	–	0	0	0	0	$j = 3$
1	3	3	0	0	1	0	$j = 3 - 3 = 0$

An optimal solution is the vector:

$$(0, 0, 1, 0)$$

We can compute an optimal solution for a rod of length $n = 2$ as follows:

Iteration	j	$s[j]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$j \leftarrow j - s[j]$
–	–	–	0	0	0	0	$j = 2$
1	2	2	0	1	0	0	$j = 2 - 2 = 0$

An optimal solution is the vector:

$$(0, 1, 0, 0)$$

16.1.2.1 Number of steps

- Let $S(n)$ denote the number of steps executed by `DP1` in the worst case, where n is the rod length. We compute $S(n)$ by counting how many times Line 2–Line 13 are executed.

- In the worst case, the condition at line 7 is true at every iteration of the inner loop, and therefore line 8 is executed as often as line 7. The number of executions of the relevant lines is:
  - Line 2 is executed 1 time.
  - Line 3 is executed 1 time.
  - Line 4 is executed n times.
  - Line 5 is executed n times.
  - Line 6 is executed $\sum_{\ell=1}^{n} \ell$ times.
  - Line 7 is executed $\sum_{\ell=1}^{n} \ell$ times.

- Line 8 is executed $\sum_{\ell=1}^{n} \ell$ times.
- Line 11 is executed n times.
- Line 13 is executed 1 time.

Therefore, the total number of steps in the worst case is

$$1 + 1 + n + n + \sum_{\ell=1}^{n} \ell + \sum_{\ell=1}^{n} \ell + \sum_{\ell=1}^{n} \ell + n + 1 = 3 + 3n + 3 \sum_{\ell=1}^{n} \ell.$$

Using $\sum_{\ell=1}^{n} \ell = \dfrac{n(n+1)}{2}$, we obtain

$$S(n) = 3 + 3n + 3 \cdot \frac{n(n+1)}{2} = \frac{3n^2 + 9n + 6}{2}.$$

Hence,

$$S(n) = \Theta(n^2).$$

Indeed,

$$\lim_{n \to \infty} \frac{\frac{3n^2+9n+6}{2}}{n^2} = \lim_{n \to \infty} \frac{1}{2} \left(3 + \frac{9}{n} + \frac{6}{n^2} \right) = \frac{3}{2} \in \mathbb{R}_{>0}.$$

The DP algorithm executes

$$S(n) = \Theta(n^2)$$

steps in the worst case.

16.1.3 A second DP algorithm

- Step 1: computation of the optimal value

For any $j \in \{0, 1, \ldots, n\}$ and $\ell \in \{0, 1, \ldots, n\}$, let $v(j, \ell)$ be the optimal value of the RCP subinstance given by:

1. a rod of length ℓ,
2. a set $\{1, 2, \ldots, j\}$ of j possible piece lengths,
3. for each piece length $k \in \{1, 2, \ldots, j\}$, a profit p_k.

The recursive formula to compute $v(j, \ell)$ reads as follows:

$$v(j, \ell) = \begin{cases} \max \left\{ v(j-1, \ell),\ p_j + v(j, \ell - j) \right\} & \text{if } 1 \leq j \leq \ell, \\ v(j-1, \ell) & \text{if } 1 \leq \ell < j, \\ 0 & \text{if } \ell = 0, \\ -\infty & \text{if } j = 0 \text{ and } \ell > 0. \end{cases} \tag{16.2}$$

– The recursive decomposition process of formula (16.2) generates at most 2 **independent subproblems**, corresponding to whether or not to cut a piece of length j from a rod of length ℓ. If $j > \ell$, a piece of length j cannot be cut from the rod, and therefore the only feasible option is not to cut it. Here, $v(j, 0) = 0$ since a rod of length 0 cannot be cut and has profit 0. Moreover, $v(0, \ell) = -\infty$ for $\ell > 0$: in this case no positive piece length is available, so no feasible solution exists for a rod of positive length. The value $-\infty$ is used to rule out this infeasible case.

– If $1 \leq j \leq \ell$, the optimal value $v(j, \ell)$ is obtained by trying the two possible options, cutting or not cutting a first piece of length j, and selecting the one that maximizes the profit. The corresponding values are $p_j + v(j, \ell - j)$ and $v(j - 1, \ell)$, respectively. If $1 \leq \ell < j$, piece length j cannot be used, and therefore $v(j, \ell) = v(j - 1, \ell)$.

– Hence, an optimal solution for a rod of length ℓ with largest available piece length j is built by combining the chosen decision on piece length j with an optimal solution of the corresponding residual subproblem.

  $\rightsquigarrow$ **Optimal Substructure verified!**

– Different sequences of decisions on the piece lengths, possibly made for different values of j and ℓ, can lead to the same subproblem, and therefore the same value $v(j, \ell)$ may need to be computed several times during the recursive evaluation of formula (16.2).

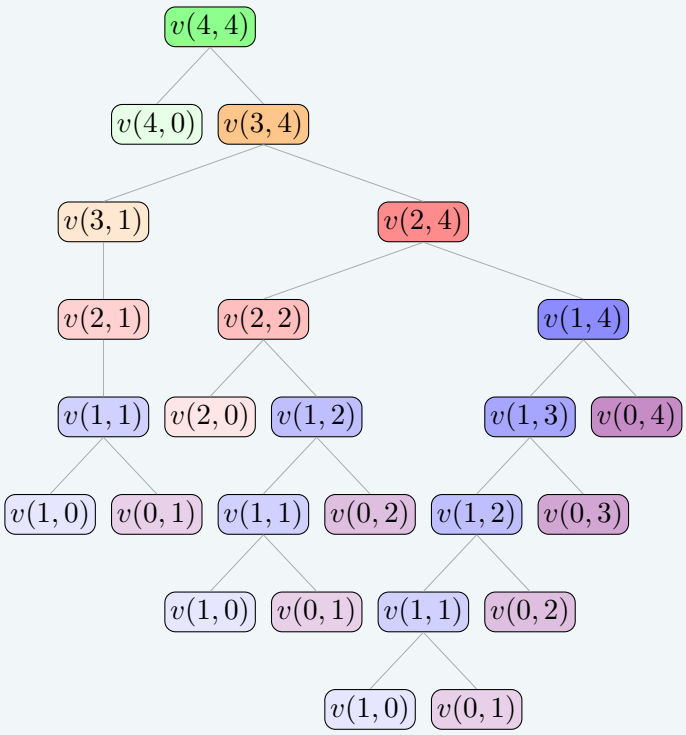
  $\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(n, n)$ of the RCP instance is the **top-down approach**. The method starts from the call with $(j, \ell) = (n, n)$ and evaluates formula (16.2) recursively: each call for a rod of length ℓ and largest available piece length j triggers further calls corresponding to the feasible decisions on piece length j, until one of the base cases $v(j, 0)$ or $v(0, \ell)$ is reached.

– The **recursion tree** of the function v has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 16.9: Recursion tree

We consider the RCP instance of Example 1. This is the recursion tree of the top-down approach to compute $v(4, 4)$:



The function v is called 25 times, and the same subproblems are solved multiple times: $v(1,2)$ is called 2 times, $v(0,2)$ is called 2 times, $v(1,1)$ is called 3 times, $v(0,1)$ is called 3 times, and $v(1,0)$ is called 3 times. All the other calls are made only once.

– The value $v(j,\ell)$ can be computed once $v(j-1,\ell)$ is known and, when $j \leq \ell$, $v(j,\ell-j)$ is also known. Therefore, for a fixed value of j, all values $v(j,\ell)$ can be computed by considering the values of ℓ in increasing order. Starting from the base cases $v(j,0)=0$ and $v(0,\ell)=-\infty$, the values of the function v can be computed row by row, moving to the next value of j only after all values $v(j,\ell)$ for the current value of j have been computed.

Example 16.10: Computing the values of the v function

We consider the RCP instance of Example 1 with a rod of length $n=4$ and the profits associated with the possible piece lengths are:

	$\ell=1$	$\ell=2$	$\ell=3$	$\ell=4$
p_ℓ	1	5	8	9

The values of the function v are computed as follows:

* For $j=1$, only pieces of length 1 can be used. Therefore:

$$v(1,0)=0,$$

$$v(1,1)=\max\{v(0,1),p_1+v(1,0)\}=\max\{-\infty,1+0\}=1,$$

$$v(1,2) = \max\{v(0,2), p_1 + v(1,1)\} = \max\{-\infty, 1+1\} = 2,$$
$$v(1,3) = \max\{v(0,3), p_1 + v(1,2)\} = \max\{-\infty, 1+2\} = 3,$$
$$v(1,4) = \max\{v(0,4), p_1 + v(1,3)\} = \max\{-\infty, 1+3\} = 4.$$

* For $j = 2$, pieces of length 1 and 2 can be used. We obtain:

$$v(2,0) = 0, \qquad v(2,1) = v(1,1) = 1,$$
$$v(2,2) = \max\{v(1,2), p_2 + v(2,0)\} = \max\{2, 5+0\} = 5,$$
$$v(2,3) = \max\{v(1,3), p_2 + v(2,1)\} = \max\{3, 5+1\} = 6,$$
$$v(2,4) = \max\{v(1,4), p_2 + v(2,2)\} = \max\{4, 5+5\} = 10.$$

* For $j = 3$, pieces of length 1, 2, and 3 can be used. We obtain:

$$v(3,0) = 0, \qquad v(3,1) = v(2,1) = 1, \qquad v(3,2) = v(2,2) = 5,$$
$$v(3,3) = \max\{v(2,3), p_3 + v(3,0)\} = \max\{6, 8+0\} = 8,$$
$$v(3,4) = \max\{v(2,4), p_3 + v(3,1)\} = \max\{10, 8+1\} = 10.$$

* For $j = 4$, all piece lengths are available. We obtain:

$$v(4,0) = 0, \qquad v(4,1) = v(3,1) = 1,$$
$$v(4,2) = v(3,2) = 5, \qquad v(4,3) = v(3,3) = 8,$$
$$v(4,4) = \max\{v(3,4), p_4 + v(4,0)\} = \max\{10, 9+0\} = 10.$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving any particular subproblem depends only on subproblems with a smaller value of j, or with the same value of j and a smaller rod length, we can solve the subproblems in increasing order of j and, for each fixed value of j, in increasing order of rod length, starting from the base cases $v(0,0) = 0$ and $v(0,\ell) = -\infty$ for all $\ell \in \{1,\ldots,n\}$ and building up to $v(n,n)$: this observation naturally leads to a **much more efficient method** to compute the optimal value $v(n,n)$ of the RCP instance.

– A **second method** to compute the optimal value $v(n,n)$ of the RCP instance is the **bottom-up approach**. The method starts from the base cases $v(0,0) = 0$ and $v(0,\ell) = -\infty$ for all $\ell \in \{1,\ldots,n\}$ and evaluates formula (16.2) for increasing values of $j \in \{1,2,\ldots,n\}$ and $\ell \in \{0,1,\ldots,n\}$: each value $v(j,\ell)$ is computed using already-computed values, until the value $v(n,n)$ of the RCP instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: DP2

Input: An RCP instance given by the length n of the rod and an array $p[1..n]$ of piece profits

Output: The optimal value of the RCP instance

```
1  Function DP2(n, p)
2      v := new array[0..n][0..n]
3      v[0][0] ← 0
4      for ℓ := 1 to n do
5          v[0][ℓ] ← −∞
6      end
7      for j := 1 to n do
8          for ℓ := 0 to n do
9              if j > ℓ then
10                 v[j][ℓ] ← v[j − 1][ℓ]
11             else
12                 if p[j] + v[j][ℓ − j] > v[j − 1][ℓ] then
13                     v[j][ℓ] ← p[j] + v[j][ℓ − j]
14                 else
15                     v[j][ℓ] ← v[j − 1][ℓ]
16                 end
17             end
18         end
19     end
20     return v[n][n]
```

* The matrix $v[0..n][0..n]$ is the **lookup table**: it stores the optimal values of all subproblems.
* The initialization sets $v[0][0] = 0$ and $v[0][\ell] = -\infty$ for all rod lengths $\ell \in \{1, \ldots, n\}$, since a positive length cannot be filled if no piece length is available.
* The outer loop, over $j \in \{1, 2, \ldots, n\}$, considers the available piece lengths in increasing order.
* The inner loop, over $\ell \in \{0, 1, \ldots, n\}$, computes the best value for each rod length using only pieces of length at most j.
* If $j > \ell$, piece length j cannot be used, and therefore $v[j][\ell] \leftarrow v[j-1][\ell]$.
* Otherwise, the DP algorithm compares the value obtained by not using piece length j, namely $v[j-1][\ell]$, with the value obtained by using one piece of length j, namely $p[j] + v[j][\ell - j]$.
* Finally, the optimal value of the RCP instance is returned as $v[n][n]$.

Example 16.11: Execution of the DP algorithm

We consider the RCP instance of Example 1 with a rod of length $n = 4$ and the profits associated with the possible piece lengths are:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
p_ℓ	1	5	8	9

For each step of the DP algorithm, the table shows the values of j and ℓ, the cell $v[j][\ell]$, and the operation performed.

j	ℓ	$v[j][\ell]$	Action
0	0	0	$v[0][0] \leftarrow 0$
0	1	$-\infty$	$v[0][1] \leftarrow -\infty$
0	2	$-\infty$	$v[0][2] \leftarrow -\infty$
0	3	$-\infty$	$v[0][3] \leftarrow -\infty$
0	4	$-\infty$	$v[0][4] \leftarrow -\infty$
1	0	0	$v[1][0] \leftarrow v[0][0] = 0$
1	1	1	$\max\{v[0][1], p[1] + v[1][0]\} = \max\{-\infty, 1 + 0\} = 1$
1	2	2	$\max\{v[0][2], p[1] + v[1][1]\} = \max\{-\infty, 1 + 1\} = 2$
1	3	3	$\max\{v[0][3], p[1] + v[1][2]\} = \max\{-\infty, 1 + 2\} = 3$
1	4	4	$\max\{v[0][4], p[1] + v[1][3]\} = \max\{-\infty, 1 + 3\} = 4$
2	0	0	$v[2][0] \leftarrow v[1][0] = 0$
2	1	1	$2 > 1$: $v[2][1] \leftarrow v[1][1] = 1$
2	2	5	$\max\{v[1][2], p[2] + v[2][0]\} = \max\{2, 5 + 0\} = 5$
2	3	6	$\max\{v[1][3], p[2] + v[2][1]\} = \max\{3, 5 + 1\} = 6$
2	4	10	$\max\{v[1][4], p[2] + v[2][2]\} = \max\{4, 5 + 5\} = 10$
3	0	0	$v[3][0] \leftarrow v[2][0] = 0$
3	1	1	$3 > 1$: $v[3][1] \leftarrow v[2][1] = 1$
3	2	5	$3 > 2$: $v[3][2] \leftarrow v[2][2] = 5$
3	3	8	$\max\{v[2][3], p[3] + v[3][0]\} = \max\{6, 8 + 0\} = 8$
3	4	10	$\max\{v[2][4], p[3] + v[3][1]\} = \max\{10, 8 + 1\} = 10$
4	0	0	$v[4][0] \leftarrow v[3][0] = 0$
4	1	1	$4 > 1$: $v[4][1] \leftarrow v[3][1] = 1$
4	2	5	$4 > 2$: $v[4][2] \leftarrow v[3][2] = 5$
4	3	8	$4 > 3$: $v[4][3] \leftarrow v[3][3] = 8$
4	4	10	$\max\{v[3][4], p[4] + v[4][0]\} = \max\{10, 9 + 0\} = 10$

This is the matrix v after the execution of the DP algorithm:

$v[j][\ell]$	$\ell = 0$	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$j = 0$	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
$j = 1$	0	1	2	3	4
$j = 2$	0	1	5	6	10
$j = 3$	0	1	5	8	10
$j = 4$	0	1	5	8	10

– After the execution of the DP algorithm, the matrix v contains the optimal value

$v[j][\ell]$ for every subproblem (j, ℓ).

- Step 2: computation of an optimal solution
  - To compute an optimal solution, it is necessary to store some additional information.
  - For each value j of the largest available piece length and each rod length ℓ, in addition to the optimal value $v[j][\ell]$, we store $s[j][\ell]$, which is equal to 1 if one piece of length j is used and to 0 otherwise.
  - Starting from $j = n$ and $\ell = n$, if $s[j][\ell] = 0$, then piece length j is not used and j is decreased by 1. If $s[j][\ell] = 1$, then one piece of length j is used and the residual rod length becomes $\ell - j$. The procedure stops when the residual length reaches 0.

    This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to compute an optimal solution:

Algorithm

Algorithm: DP2_InfoSol

Input: An RCP instance given by the length n of the rod and an array $p[1..n]$ of piece profits

Output: The optimal value of the RCP instance and the matrix of binary information $s[0..n][0..n]$

```
1  Function DP2_InfoSol(n, p)
2      v := new array[0..n][0..n]
3      s := new array[0..n][0..n]
4      v[0][0] ← 0
5      s[0][0] ← 0
6      for ℓ := 1 to n do
7          v[0][ℓ] ← −∞
8          s[0][ℓ] ← 0
9      end
10     for j := 1 to n do
11         for ℓ := 0 to n do
12             if j > ℓ then
13                 v[j][ℓ] ← v[j − 1][ℓ]
14                 s[j][ℓ] ← 0
15             else
16                 if p[j] + v[j][ℓ − j] > v[j − 1][ℓ] then
17                     v[j][ℓ] ← p[j] + v[j][ℓ − j]
18                     s[j][ℓ] ← 1
19                 else
20                     v[j][ℓ] ← v[j − 1][ℓ]
21                     s[j][ℓ] ← 0
22                 end
23             end
24         end
25     end
26     return v[n][n], s
```

∗ The DP algorithm uses the additional matrix s, which keeps track of the decision made when $v[j][\ell]$ is computed. If one piece of length j is selected, $s[j][\ell] \leftarrow 1$ stores this decision; otherwise, $s[j][\ell] \leftarrow 0$ stores that piece length j is not used.

Example 16.12: Computation of the additional information

We consider the RCP instance of Example 1 with a rod of length $n = 4$ and the profits associated with the possible piece lengths are:

	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
p_ℓ	1	5	8	9

For each step of the DP algorithm, the table shows the values of j and ℓ, the cell $s[j][\ell]$, and the operation performed.

j	ℓ	$s[j][\ell]$	Action
0	0	0	$s[0][0] \leftarrow 0$
0	1	0	$s[0][1] \leftarrow 0$
0	2	0	$s[0][2] \leftarrow 0$
0	3	0	$s[0][3] \leftarrow 0$
0	4	0	$s[0][4] \leftarrow 0$
1	0	0	$1 > 0$: $s[1][0] \leftarrow 0$
1	1	1	$p[1] + v[1][0] = 1 + 0 > v[0][1] = -\infty$: $s[1][1] \leftarrow 1$
1	2	1	$p[1] + v[1][1] = 1 + 1 > v[0][2] = -\infty$: $s[1][2] \leftarrow 1$
1	3	1	$p[1] + v[1][2] = 1 + 2 > v[0][3] = -\infty$: $s[1][3] \leftarrow 1$
1	4	1	$p[1] + v[1][3] = 1 + 3 > v[0][4] = -\infty$: $s[1][4] \leftarrow 1$
2	0	0	$2 > 0$: $s[2][0] \leftarrow 0$
2	1	0	$2 > 1$: $s[2][1] \leftarrow 0$
2	2	1	$p[2] + v[2][0] = 5 + 0 > v[1][2] = 2$: $s[2][2] \leftarrow 1$
2	3	1	$p[2] + v[2][1] = 5 + 1 > v[1][3] = 3$: $s[2][3] \leftarrow 1$
2	4	1	$p[2] + v[2][2] = 5 + 5 > v[1][4] = 4$: $s[2][4] \leftarrow 1$
3	0	0	$3 > 0$: $s[3][0] \leftarrow 0$
3	1	0	$3 > 1$: $s[3][1] \leftarrow 0$
3	2	0	$3 > 2$: $s[3][2] \leftarrow 0$
3	3	1	$p[3] + v[3][0] = 8 + 0 > v[2][3] = 6$: $s[3][3] \leftarrow 1$
3	4	0	$p[3] + v[3][1] = 8 + 1 \leq v[2][4] = 10$: $s[3][4] \leftarrow 0$
4	0	0	$4 > 0$: $s[4][0] \leftarrow 0$
4	1	0	$4 > 1$: $s[4][1] \leftarrow 0$
4	2	0	$4 > 2$: $s[4][2] \leftarrow 0$
4	3	0	$4 > 3$: $s[4][3] \leftarrow 0$
4	4	0	$p[4] + v[4][0] = 9 + 0 \leq v[3][4] = 10$: $s[4][4] \leftarrow 0$

This is the matrix s after the execution of the DP algorithm:

$s[j][\ell]$	$\ell = 0$	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$j = 0$	0	0	0	0	0
$j = 1$	0	1	1	1	1
$j = 2$	0	0	1	1	1
$j = 3$	0	0	0	1	0
$j = 4$	0	0	0	0	0

– Once the matrix s has been computed, an optimal solution for a rod of length n can be computed by repeatedly applying the following rule, starting from $(j, \ell) = (n, n)$ and $x_h = 0$ for all $h \in \{1, 2, \ldots, n\}$:

$$\begin{aligned} j &\leftarrow j - 1 && \text{if } s[j][\ell] = 0, \\ x_j &\leftarrow x_j + 1, \quad \ell \leftarrow \ell - j && \text{if } s[j][\ell] = 1. \end{aligned}$$

The procedure stops when $\ell = 0$. This yields the vector $(x_1, x_2, \ldots, x_n)$, where each entry x_h is the number of pieces of length h in an optimal solution.

– The following pseudo-code implements the computation of an optimal solution, given the matrix s.

Algorithm

Algorithm: DP2_SolComputation

Input: The rod length n and the matrix $s[0..n][0..n]$
Output: An optimal solution of the RCP instance

```
1  Function DP2_SolComputation(n, s)
2      x := new array[1..n]
3      for h := 1 to n do
4          x[h] ← 0
5      end
6      j := n
7      ℓ := n
8      while ℓ > 0 do
9          if s[j][ℓ] = 0 then
10             j ← j − 1
11         else
12             x[j] ← x[j] + 1
13             ℓ ← ℓ − j
14         end
15     end
16     return x
```

Example 16.13: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, we have the following matrix s:

$s[j][\ell]$	$\ell=0$	$\ell=1$	$\ell=2$	$\ell=3$	$\ell=4$
$j=0$	0	0	0	0	0
$j=1$	0	1	1	1	1
$j=2$	0	0	1	1	1
$j=3$	0	0	0	1	0
$j=4$	0	0	0	0	0

We can compute an optimal solution for a rod of length $n=4$ as follows:

Iteration	j	ℓ	$s[j][\ell]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	Action
1	4	4	0	0	0	0	0	$j \leftarrow 3$
2	3	4	0	0	0	0	0	$j \leftarrow 2$
3	2	4	1	0	1	0	0	$x[2] \leftarrow 1$, $\ell \leftarrow 4-2=2$
4	2	2	1	0	2	0	0	$x[2] \leftarrow 2$, $\ell \leftarrow 2-2=0$

An optimal solution is the vector:

$$(0, 2, 0, 0)$$

– The same computation procedure, based on the matrix s, can be applied to any rod length $\ell \leq n$ by starting from $j = n$.

Example 16.14: Computing optimal solutions

After the execution of the DP algorithm on the instance of Example 1, we have the following matrix s:

$s[j][\ell]$	$\ell = 0$	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$j = 0$	0	0	0	0	0
$j = 1$	0	1	1	1	1
$j = 2$	0	0	1	1	1
$j = 3$	0	0	0	1	0
$j = 4$	0	0	0	0	0

We can compute an optimal solution for a rod of length $\ell = 3$ as follows:

Iteration	j	ℓ	$s[j][\ell]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	Action
1	4	3	0	0	0	0	0	$j \leftarrow 3$
2	3	3	1	0	0	1	0	$x[3] \leftarrow 1$, $\ell \leftarrow 3 - 3 = 0$

An optimal solution is the vector:

$$(0, 0, 1, 0)$$

We can compute an optimal solution for a rod of length $\ell = 2$ as follows:

Iteration	j	ℓ	$s[j][\ell]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	Action
1	4	2	0	0	0	0	0	$j \leftarrow 3$
2	3	2	0	0	0	0	0	$j \leftarrow 2$
3	2	2	1	0	1	0	0	$x[2] \leftarrow 1$, $\ell \leftarrow 2 - 2 = 0$

An optimal solution is the vector:

$$(0, 1, 0, 0)$$

16.1.3.1 Number of steps

- Let $S(n)$ denote the number of steps executed by `DP2` in the worst case, where n is the rod length. We compute $S(n)$ by counting how many times the lines of the pseudo-code of `DP2` are executed.

- The number of times each line of code is executed is:
  - Line 2 is executed 1 time.
  - Line 3 is executed 1 time.
  - Line 4 is executed n times.
  - Line 5 is executed n times.
  - Line 7 is executed n times.
  - Line 8 is executed $n(n+1)$ times.
  - Line 9 is executed $n(n+1)$ times.
  - Line 12 is checked only when $j \leq \ell$, that is

$$\sum_{j=1}^{n}(n-j+1) = \sum_{j=1}^{n} j = \frac{n(n+1)}{2}$$

    times.
  - One among Line 10, Line 13, and Line 15 is executed for each pair (j, ℓ), and therefore these assignment lines are executed $n(n+1)$ times in total.
  - Line 20 is executed 1 time.

  Therefore, the total number of steps in the worst case is

$$1+1+n+n+n+n(n+1)+n(n+1)+\frac{n(n+1)}{2}+n(n+1)+1.$$

  Thus,

$$S(n) = 3+3n+3n(n+1)+\frac{n(n+1)}{2} = \frac{7n^2+13n+6}{2}.$$

  Hence,

$$S(n) = \Theta(n^2).$$

  Indeed,

$$\lim_{n\to\infty} \frac{\frac{7n^2+13n+6}{2}}{n^2} = \lim_{n\to\infty} \frac{1}{2}\left(7+\frac{13}{n}+\frac{6}{n^2}\right) = \frac{7}{2} \in \mathbb{R}_{>0}.$$

  The DP algorithm executes

$$S(n) = \Theta(n^2)$$

  steps in the worst case.

16.1.4 Typical mistakes

Warning

- **Counting vectors instead of cutting patterns.** The 2^{n-1} cutting patterns of a rod of length n are more than the solution vectors, because different patterns can give the same vector.
- **Forgetting the base case.** $v(0) = 0$ is used when the first piece has the length of the whole rod: $v(\ell)$ contains the term $p_\ell + v(0)$.
- **Using row $j-1$ in the second recurrence.** The term $p_j + v(j, \ell - j)$ stays in row j: otherwise a piece length could be cut only once.
- **Stopping the reconstruction early.** The rule $x_{s[\ell]} \leftarrow x_{s[\ell]} + 1$, $\ell \leftarrow \ell - s[\ell]$ is repeated until $\ell = 0$; the last piece completes the rod.

16.1.5 In summary

Summary

- The RCP asks how to cut a rod of length n into pieces, with each length available any number of times, to maximize the total profit.
- The recurrence $v(\ell) = \max_{j \le \ell}\{p_j + v(\ell - j)\}$ has the optimal substructure and overlapping subproblems, and $v(0) = 0$.
- The vector v is filled bottom-up in $\Theta(n^2)$ steps, which is polynomial because the input has n profits.
- The vector s of the first pieces gives an optimal solution for every rod length at no extra cost.

Dashboard of the RCP example. Example 1 (rod of length 4):

Method	Lower bound	Upper bound	Origin
DP algorithm	10	10	optimal value, two pieces of length 2

16.2 A DP algorithm for the Coin Change Problem

Prerequisites

- From this chapter, the DP algorithms for the knapsack problems, in particular the UKP: unlimited supply of every item type, and the reconstruction of a solution.
- From this module, the chapter «Principles of dynamic programming»: optimal substructure, overlapping subproblems, base cases.
- From the module *Algorithms*: pseudocode conventions and the size of the input.

Objectives

At the end of this chapter you will be able to:

- explain why the CCP asks for an exact amount and why an instance can be infeasible;
- write the recurrence $g(k)$ with the value $+\infty$ for the amounts that cannot be returned;
- run the DP algorithm by hand, following the comparisons with the running best value m;
- reconstruct an optimal solution as the vector $(x_1, \ldots, x_n)$ when the instance is feasible;
- count the steps of the DP algorithm.

16.2.1 Problem definition

Definition 16.2: Coin Change Problem

Given:

1. a vector $w \in \mathbb{Z}^n_{\geq 1}$ of n *denominations*, each available in unlimited supply,
2. an integer *amount of change* $c \in \mathbb{Z}_{\geq 0}$ to return,

the **Coin Change Problem** (CCP) asks to decide how many coins of each denomination to use so that their total value is exactly equal to c, and the total number of coins used is minimized. If no combination of coins returns exactly c, the instance is *infeasible*.

Example 16.15: CCP instance

We consider an amount of change $c = 10$ to return, and $n = 3$ denominations:

$$w = (2, 3, 5).$$

We refer to this CCP instance as Example 1.

Unlike the Knapsack Problem, where the total weight of the selected items must not exceed the capacity, the CCP requires the total value of the selected coins to be *exactly equal* to c. Moreover, since every denomination is available in unlimited supply, the number of coins of a given denomination used in a solution is not bounded a priori.

16.2.1.1 Encoding of feasible solutions

A *feasible solution* of the CCP is a choice of the number of coins of each denomination whose total value is exactly equal to the amount of change c.

- A CCP feasible solution can be described by the integer vector

$$(x_1, x_2, \ldots, x_n)$$

where each component has the following meaning:

$$x_j = \text{number of coins of denomination } j \text{ used}, \qquad \forall j \in \{1, 2, \ldots, n\}, \quad x_j \geq 0.$$

Since the solution is feasible, the total value of the selected coins must be exactly equal to c:

$$\sum_{j=1}^{n} w_j x_j = c.$$

The total number of coins of a feasible solution is

$$\sum_{j=1}^{n} x_j.$$

- Since $\sum_{j=1}^{n} w_j x_j = c$, every component x_j of a feasible solution satisfies $x_j \leq \lfloor c/w_j \rfloor$. Therefore, the number of feasible solutions is finite, but it grows quickly with c and n, and enumerating all of them to find the one using the fewest coins quickly becomes impractical.

16.2.2 A DP algorithm

- Step 1: computation of the optimal value

For any $k \in \{0, 1, \ldots, c\}$, let $g(k)$ be the optimal value of the CCP subinstance given by:

1. an integer amount of change k to return,
2. the same set $\{1, 2, \ldots, n\}$ of denominations, each available in unlimited supply.

The recursive formula to compute $g(k)$ reads as follows:

$$g(k) = \begin{cases} 0 & \text{if } k = 0, \\ +\infty & \text{if } k \geq 1 \text{ and } w_j > k \text{ for every } j \in \{1, 2, \ldots, n\}, \\ \min\limits_{j\,:\,w_j \leq k} \{1 + g(k - w_j)\} & \text{if } k \geq 1 \text{ and } w_j \leq k \text{ for some } j \in \{1, 2, \ldots, n\}. \end{cases} \tag{16.3}$$

  - The recursive decomposition process of formula (16.3) generates at most n **independent subproblems**, corresponding to the possible choices of a denomination j, with $w_j \leq k$, used as one of the coins returning the amount of change k. The base case is $g(0) = 0$ since no coin is needed to return a null amount of change, and $g(k) = +\infty$ if no denomination is small enough to be used, since in this case the amount of change k cannot be returned at all.
  - The optimal value $g(k)$ is obtained by trying every denomination j with $w_j \leq k$ and selecting the one that minimizes the sum $1 + g(k - w_j)$ of the single coin of denomination j and the optimal value of the residual subproblem, returning the remaining amount of change $k - w_j$.
  - Hence, an optimal solution for the subinstance with amount of change k is built by combining the chosen coin of denomination j with an optimal solution of the corresponding residual subproblem, returning the amount of change $k - w_j$.

    $\rightsquigarrow$ **Optimal Substructure verified!**

  - Different choices of the denomination used as the first coin, possibly made for different values of k, can lead to the same residual amount of change, and therefore the same value $g(k)$ may need to be computed several times during the recursive evaluation of formula (16.3).

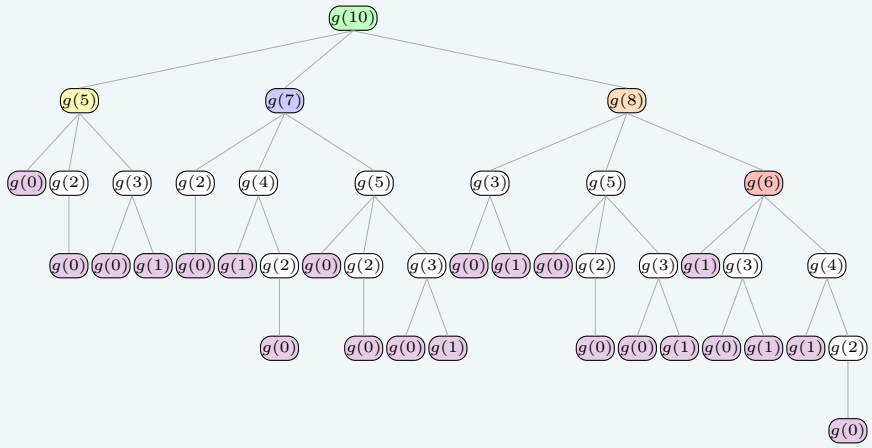
    $\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $g(c)$ of the CCP instance is the **top-down approach**. The method starts from the call with $k = c$ and evaluates formula (16.3) recursively: each call tries every denomination j with $w_j \leq k$, until the amount of change is equal to 0 or no denomination is small enough to be used.

– The **recursion tree** of the function g has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 16.16: Recursion tree

We consider the CCP instance of Example 1. This is the recursion tree of the top-down approach to compute $g(10)$:



The function g is called 42 times, and the same subproblems are solved multiple times: $g(0)$ is called 14 times, $g(1)$ is called 8 times, $g(2)$ is called 6 times, $g(3)$ is called 5 times, $g(4)$ is called 2 times, and $g(5)$ is called 3 times. All the other calls are made only once.

– The value $g(k)$ can be computed once the values $g(k')$, for every smaller amount of change $k' < k$, are known. Therefore, all values $g(k)$ can be computed by considering the amount of change k in increasing order.

Example 16.17: Computing the values of the g function

We consider the CCP instance of Example 1, with amount of change $c = 10$ to return and denominations $w = (2, 3, 5)$.
The values of the function g are computed as follows:

* The base case gives $g(0) = 0$.

* Once we have computed $g(0)$, we can compute $g(1)$: since no denomination satisfies $w_j \leq 1$, we obtain

$$g(1) = +\infty.$$

* Once we have computed $g(0)$, we can compute $g(2)$:

$$g(2) = 1 + g(0) = 1 + 0 = 1.$$

* Once we have computed $g(1)$ and $g(2)$, we can compute $g(4)$, trying every denomination j with $w_j \leq 4$:

$$g(4) = \min\{1 + g(2),\ 1 + g(1)\} = \min\{1 + 1,\ 1 + \infty\} = 2.$$

* Once we have computed $g(2)$, $g(4)$, and $g(5)$, we can compute $g(7)$, trying every denomination j with $w_j \leq 7$:

$$g(7) = \min\{1 + g(5),\ 1 + g(4),\ 1 + g(2)\} = \min\{1 + 1,\ 1 + 2,\ 1 + 1\} = 2.$$

Here, the denominations $w_1 = 2$ and $w_3 = 5$ are tied: both give value 2. Since the bottom-up algorithm only updates the optimal value when a strictly smaller value is found, the first denomination achieving the minimum, $w_1 = 2$, is the one recorded.

* Finally, once we have computed $g(5)$, $g(7)$, and $g(8)$, we can compute $g(10)$, trying every denomination j with $w_j \leq 10$:

$$g(10) = \min\{1 + g(8),\ 1 + g(7),\ 1 + g(5)\} = \min\{1 + 2,\ 1 + 2,\ 1 + 1\} = 2.$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**.

– Since solving any particular subproblem depends only on subproblems with a smaller amount of change, we can solve the subproblems in increasing order of k, starting from the base case $g(0) = 0$ and building up to $g(c)$: this observation naturally leads to a **much more efficient method** to compute the optimal value $g(c)$ of the CCP instance.

– A **second method** to compute the optimal value $g(c)$ of the CCP instance is the **bottom-up approach**. The method starts from the base case and evaluates formula (16.3) for increasing values of $k \in \{1, 2, \ldots, c\}$: each value $g(k)$ is computed using already-computed values, until the value $g(c)$ of the CCP instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: CCP_DP

Input: A CCP instance given by n, the array $w[1..n]$ of denominations, and the amount of change c

Output: The optimal value

```
1  Function CCP_DP(n, w, c)
2      g := new array[0..c]
3      g[0] ← 0
4      for k := 1 to c do
5          m := +∞
6          for j := 1 to n do
7              if w[j] ≤ k and 1 + g[k − w[j]] < m then
8                  m ← 1 + g[k − w[j]]
9              end
10         end
11         g[k] ← m
12     end
13     return g[c]
```

* The array $g[0..c]$ is the **lookup table**: it stores the optimal values of all subproblems.
* The instruction $g[0] \leftarrow 0$ initializes the base case, corresponding to a null amount of change.
* The outer loop, over $k \in \{1, 2, \ldots, c\}$, computes the entries of the array in increasing order of amount of change.
* The inner loop, over $j \in \{1, 2, \ldots, n\}$, tries every denomination and keeps in m the smallest value found so far; m is initialized to $+\infty$, so that $g[k]$ remains $+\infty$ if no denomination j with $w[j] \leq k$ is found.
* At the end of the inner loop, $g[k] \leftarrow m$ stores the optimal value $g[k]$.
* Finally, the optimal value of the CCP instance is returned as $g[c]$.

Example 16.18: Execution of the DP algorithm

We consider the CCP instance of Example 1, with amount of change $c = 10$ to return and denominations $w = (2, 3, 5)$.

For each step of the DP algorithm, the table shows the values of k and j, the cell $g[k]$, the current value of m, and the comparison performed. The cell of m is highlighted in red when updated; the cell of $g[k]$ is highlighted in blue when assigned.

k	j	$g[k]$	m	Action
–	–	0	–	$g[0] \leftarrow 0$
1	–	–	$+\infty$	$m := +\infty$
	1	–	$+\infty$	$w[1] = 2 > 1$: skip
	2	–	$+\infty$	$w[2] = 3 > 1$: skip
	3	–	$+\infty$	$w[3] = 5 > 1$: skip
	–	$+\infty$	$+\infty$	$g[1] \leftarrow m = +\infty$
2	–	–	$+\infty$	$m := +\infty$
	1	–	1	$1+g[0] = 1 < m$: $m \leftarrow 1$
	2	–	1	$w[2] = 3 > 2$: skip
	3	–	1	$w[3] = 5 > 2$: skip
	–	1	1	$g[2] \leftarrow m = 1$
3	–	–	$+\infty$	$m := +\infty$
	1	–	$+\infty$	$1+g[1] = +\infty \not< m = +\infty$: no update
	2	–	1	$1+g[0] = 1 < m$: $m \leftarrow 1$
	3	–	1	$w[3] = 5 > 3$: skip
	–	1	1	$g[3] \leftarrow m = 1$
4	–	–	$+\infty$	$m := +\infty$
	1	–	2	$1+g[2] = 2 < m$: $m \leftarrow 2$
	2	–	2	$1+g[1] = +\infty \not< m = 2$: no update
	3	–	2	$w[3] = 5 > 4$: skip
	–	2	2	$g[4] \leftarrow m = 2$
5	–	–	$+\infty$	$m := +\infty$
	1	–	2	$1+g[3] = 2 < m$: $m \leftarrow 2$
	2	–	2	$1+g[2] = 2 \not< m = 2$: no update
	3	–	1	$1+g[0] = 1 < m$: $m \leftarrow 1$
	–	1	1	$g[5] \leftarrow m = 1$
6	–	–	$+\infty$	$m := +\infty$
	1	–	3	$1+g[4] = 3 < m$: $m \leftarrow 3$
	2	–	2	$1+g[3] = 2 < m$: $m \leftarrow 2$
	3	–	2	$1+g[1] = +\infty \not< m = 2$: no update
	–	2	2	$g[6] \leftarrow m = 2$
7	–	–	$+\infty$	$m := +\infty$
	1	–	2	$1+g[5] = 2 < m$: $m \leftarrow 2$
	2	–	2	$1+g[4] = 3 \not< m = 2$: no update
	3	–	2	$1+g[2] = 2 \not< m = 2$: no update
	–	2	2	$g[7] \leftarrow m = 2$
8	–	–	$+\infty$	$m := +\infty$
	1	–	3	$1+g[6] = 3 < m$: $m \leftarrow 3$
	2	–	2	$1+g[5] = 2 < m$: $m \leftarrow 2$
	3	–	2	$1+g[3] = 2 \not< m = 2$: no update
	–	2	2	$g[8] \leftarrow m = 2$
9	–	–	$+\infty$	$m := +\infty$
	1	–	3	$1+g[7] = 3 < m$: $m \leftarrow 3$
	2	–	3	$1+g[6] = 3 \not< m = 3$: no update
	3	–	3	$1+g[4] = 3 \not< m = 3$: no update
	–	3	3	$g[9] \leftarrow m = 3$
10	–	–	$+\infty$	$m := +\infty$
	1			$: m \leftarrow 3$
	2			$= 3$: no update
	3			$: m \leftarrow 2$
	–	2	2	$g[10] \leftarrow m = 2$

This is the array g after the execution of the DP algorithm:

k	0	1	2	3	4	5	6	7	8	9	10
$g[k]$	0	$+\infty$	1	1	2	1	2	2	2	3	2

- After the execution of the DP algorithm, the array g contains the optimal value $g[k]$ for every amount of change $k \in \{0, 1, \ldots, c\}$, where $g[c]$ is the minimum number of coins needed to return the amount of change c.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, i.e., the integer vector $(x_1, x_2, \ldots, x_n)$, it is necessary to store some additional information.
  - For each amount of change k, in addition to the optimal value $g[k]$, we store $s[k]$, the index of the denomination of the coin used to obtain the optimal value $g[k]$ in formula (16.3).
  - More precisely, $s[k]$ is equal to the index j of the denomination w_j that minimizes the right-hand side of formula (16.3) when computing $g[k]$.

  This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to construct an optimal solution:

Algorithm

Algorithm: CCP_DP_InfoSol

Input: A CCP instance given by n, the array $w[1..n]$ of denominations, and the amount of change c

Output: The optimal value and the array $s[0..c]$ of the indices of the denominations used

```
1  Function CCP_DP_InfoSol(n, w, c)
2      g := new array[0..c]; s := new array[0..c]
3      g[0] ← 0; s[0] ← 0
4      for k := 1 to c do
5          m := +∞; t := 0
6          for j := 1 to n do
7              if w[j] ≤ k and 1 + g[k − w[j]] < m then
8                  m ← 1 + g[k − w[j]]; t ← j
9              end
10         end
11         g[k] ← m; s[k] ← t
12     end
13     return g[c], s
```

∗ The DP algorithm uses the additional temporary variable t to keep track of the decision made when m is updated. If the denomination $w[j]$ is used, $t \leftarrow j$ stores its index. At the end of the inner loop, $s[k] \leftarrow t$ stores the index of the denomination of the coin used, together with the optimal value $g[k] \leftarrow m$.

Example 16.19: Computation of the additional information

We consider the CCP instance of Example 1, with amount of change $c = 10$ to return and denominations $w = (2, 3, 5)$.

This is the array s after the execution of the DP algorithm:

k	0	1	2	3	4	5	6	7	8	9	10
$s[k]$	0	0	1	2	1	3	2	1	2	1	3

For example, $s[5] = 3$ since formula (16.3) selects the denomination $w_3 = 5$ when computing $g[5] = 1$, and $s[10] = 3$ since the denomination $w_3 = 5$ is also selected when computing $g[10] = 2$. The entry $s[1]$ is left undefined (0, by convention), since $g[1] = +\infty$ is infeasible.

– Once the array s has been computed, and provided that $g[c] < +\infty$, an optimal solution can be reconstructed by repeatedly using one coin of denomination $s[k]$ and moving to the residual amount of change $k - w_{s[k]}$, starting from $k = c$ and $x_j = 0$ for all $j \in \{1, 2, \ldots, n\}$, until $k = 0$:

$$x_{s[k]} \leftarrow x_{s[k]} + 1, \qquad k \leftarrow k - w_{s[k]}.$$

This yields the vector $(x_1, x_2, \ldots, x_n)$, where each entry x_j is the number of coins of denomination j in an optimal solution.

– The condition $g[c] < +\infty$ is necessary: if $g[c] = +\infty$ the instance is infeasible, $s[c] = 0$ and there is no solution to reconstruct. If $g[c] < +\infty$, then $s[k] \geq 1$ for every amount of change k visited by the procedure.

Algorithm

Algorithm: CCP_Reconstruct

Input: The number n of denominations, the array $w[1..n]$, the amount of change c with $g[c] < +\infty$, and the array $s[0..c]$

Output: An optimal solution

```
1  Function CCP_Reconstruct(n, w, c, s)
2      x := new array[1..n]
3      for j := 1 to n do
4          x[j] ← 0
5      end
6      k := c
7      while k > 0 do
8          x[s[k]] ← x[s[k]] + 1
9          k ← k − w[s[k]]
10     end
11     return x
```

Example 16.20: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 1, we have the following array s:

k	0	1	2	3	4	5	6	7	8	9	10
$s[k]$	0	0	1	2	1	3	2	1	2	1	3

We can reconstruct an optimal solution by starting from $k = 10$ as follows:

Iteration	k	$s[k]$	$x[1]$	$x[2]$	$x[3]$	$k \leftarrow k - w_{s[k]}$
–	–	–	0	0	0	$k = 10$
1	10	3	0	0	1	$k = 10 - 5 = 5$
2	5	3	0	0	2	$k = 5 - 5 = 0$

Thus, the vector

$$(0, 0, 2)$$

is an optimal solution: it uses two coins of denomination $w_3 = 5$. The corresponding total value is $2 \cdot 5 = 10 = c$. The corresponding optimal value is $g(10) = 2$.

16.2.2.1 Number of steps

- Let $S(n, c)$ denote the number of steps executed by `CCP_DP` in the worst case, where n is the number of denominations and c is the amount of change. We compute $S(n, c)$ by counting how many times the lines of the pseudo-code of `CCP_DP` are executed.

- In the worst case, the condition at line 7 is true at every iteration of the inner loop, and therefore line 8 is executed as often as line 7. The number of executions of the relevant lines is:
  - Line 2 is executed 1 time.
  - Line 3 is executed 1 time.
  - Line 4 is executed c times.
  - Line 5 is executed c times.
  - Line 6 is executed cn times.
  - Line 7 is executed cn times.
  - Line 8 is executed cn times.
  - Line 11 is executed c times.
  - Line 13 is executed 1 time.

  Therefore, the total number of steps in the worst case is
  $$1 + 1 + c + c + cn + cn + cn + c + 1.$$
  Thus,
  $$S(n, c) = 3 + 3c + 3cn.$$
  Hence,
  $$S(n, c) = \Theta(nc).$$

  The DP algorithm executes
  $$S(n, c) = \Theta(nc)$$
  steps in the worst case.

  Indeed,
  $$\lim_{n,c\to\infty} \frac{3cn + 3c + 3}{nc} = \lim_{n,c\to\infty} \left(3 + \frac{3}{n} + \frac{3}{nc}\right) = 3 \in \mathbb{R}_{>0}.$$
- The version `CCP_DP_InfoSol` stores the additional array s and performs only a constant number of extra operations inside the same loops. Therefore, its number of steps remains $\Theta(nc)$.

16.2.3 Typical mistakes

Warning

- **Using 0 instead of $+\infty$.** An amount that cannot be returned has $g(k) = +\infty$; with the value 0 the DP algorithm would claim that no coin is needed.
- **Reconstructing an infeasible instance.** If $g[c] = +\infty$ then $s[c] = 0$ and the reconstruction would never stop: the DP algorithm must first check that $g[c] < +\infty$.
- **Storing the value of the coin instead of its index.** The vector x is indexed by the denominations: $s[k]$ stores the index j, and the coin value is $w_{s[k]}$.
- **Confusing the CCP with the UKP.** The CCP asks for exactly c and minimizes the number of coins; the UKP asks for at most c and maximizes the profit.

16.2.4 In summary

Summary

- The CCP asks for the minimum number of coins whose total value is exactly c; the instance is infeasible when $g(c) = +\infty$.
- The recurrence $g(k) = \min_{j:\, w_j \le k}\{1 + g(k - w_j)\}$ is computed bottom-up in $\Theta(nc)$ steps, which is pseudo-polynomial.
- An optimal solution is read backwards from $s[c]$, the index of the last coin used, and only if $g[c] < +\infty$.

Dashboard of the CCP example. Example 1 ($c = 10$, $w = (2, 3, 5)$):

Method	Lower bound	Upper bound	Origin
DP algorithm	2	2	minimum number of coins, solution $(0, 0, 2)$

Chapter 17

Dynamic programming for lot sizing

Prerequisites

- From this module, the chapters on lot sizing: the single-item problem, its MIP model with the variables x_t, s_t, y_t, and the example of the MIP course.
- From this module, the chapter «Principles of dynamic programming» and the DP algorithms of the previous chapters: recurrence, base cases, lookup table and reconstruction of a solution.
- From the module *Algorithms*: pseudocode, and the size of the input.

Where we were

- **As a problem and as a model**: single-item lot sizing, its MIP model and its formulations (lot-sizing chapters).
- **Now, as a table**: the DP algorithm of Wagner and Whitin gives the optimal value of the uncapacitated problem in $\Theta(m^2)$ steps, and a DP algorithm on the inventory handles the capacitated one.

Objectives

At the end of this chapter you will be able to:

- state the zero-inventory ordering property, and prove it with the exchange argument;
- compute the cost $c_{j,t}$ of a production run, and the matrix of the costs, in $O(m^2)$ steps;
- write the recurrence $v(t)$ over the last production run, run the DP algorithm by hand and reconstruct an optimal plan;
- explain why the property fails with capacities, and write the recurrence $v(t, k)$ on the pairs (period, final inventory);
- count the steps of the two DP algorithms, and say why the second one is pseudo-polynomial.

17.1 Problem definition

- A company plans the production of **one product** over a horizon of m time periods (weeks, months, ...). The demand of each period must be satisfied, either by the production of the period or by the inventory kept from the previous periods.

- Producing in a period requires a **setup**, which has a fixed cost, independent of the produced quantity; each produced unit has a variable cost, and each unit kept in stock at the end of a period has a holding cost. Producing rarely saves setup costs but increases the holding costs: the problem is to find the best compromise.

- This is the lot-sizing problem of the chapter on lot sizing, restricted to a **single product**: we drop the product index p, we measure the capacity in units of product (variable production time $vt_t = 1$, no setup time), and we assume that there is no initial inventory.

Definition 17.1: Uncapacitated Lot-Sizing Problem

Given:

1. a set $T = \{1, 2, \dots, m\}$ of $m \in \mathbb{Z}_{\geq 1}$ *time periods*,
2. for each period $t \in T$, a *demand* $d_t \in \mathbb{Z}_{\geq 1}$,
3. for each period $t \in T$, a *setup cost* $sc_t \in \mathbb{Z}_{\geq 0}$, a *variable production cost* $vc_t \in \mathbb{Z}_{\geq 0}$ per unit, and a *holding cost* $hc_t \in \mathbb{Z}_{\geq 0}$ per unit in stock at the end of period t,

the **Uncapacitated Lot-Sizing Problem** (ULS) asks to decide how many units to produce in each period, so that, starting with an empty inventory and ending with an empty inventory, the demand of every period is satisfied, and the total cost of setups, production, and holding is minimized.

- For any $j, t \in T$, we denote by
$$D_{j,t} = \sum_{k=j}^{t} d_k$$
the total demand of periods $j, j+1, \dots, t$, with $D_{j,t} = 0$ if $j > t$.

- With the variables of the chapter on lot sizing (x_t production, s_t inventory at the end of period t, y_t setup), the ULS reads as follows, where $s_0 = 0$ and the big-M value of period t is the total demand $D_{t,m}$ of the remaining periods:

$$\min \sum_{t \in T} (sc_t\, y_t + vc_t\, x_t + hc_t\, s_t) \tag{17.1}$$

$$s_{t-1} + x_t = d_t + s_t \qquad t \in T \tag{17.2}$$

$$x_t \leq D_{t,m}\, y_t \qquad t \in T \tag{17.3}$$

$$s_m = 0 \tag{17.4}$$
$$x_t,\ s_t \geq 0 \qquad t \in T \tag{17.5}$$
$$y_t \in \{0,1\} \qquad t \in T \tag{17.6}$$

- Constraints (17.2) are the flow balancing constraints, constraints (17.3) force a setup in every period with positive production, and constraint (17.4) imposes an empty inventory at the end of the horizon (with nonnegative costs, a final inventory is never useful).

Example 17.1: Uncapacitated lot sizing

We consider $m = 5$ periods with the following data:

t	1	2	3	4	5
d_t	20	10	30	40	10
sc_t	50	50	50	50	50
vc_t	0	0	0	0	0
hc_t	1	1	1	1	1

We refer to this ULS instance as Example 17.1. It is the uncapacitated instance used in the MIP course (setup cost 50, holding cost 1, no variable production cost).

17.1.1 Encoding of feasible solutions

A *feasible solution* of the ULS is a production plan that satisfies the demand of every period, starting and ending with an empty inventory.

- A ULS feasible solution can be described by a solution vector

$$(x_1, x_2, \ldots, x_m)$$

where each component has the following meaning:

$$x_t = \text{number of units produced in period } t, \qquad \forall t \in \{1, 2, \ldots, m\}.$$

The inventory at the end of period t is determined by the vector:

$$s_t = \sum_{j=1}^{t} x_j - D_{1,t}, \qquad \forall t \in \{1, 2, \ldots, m\}.$$

Since the solution is feasible, the inventories are nonnegative and the last one is empty:

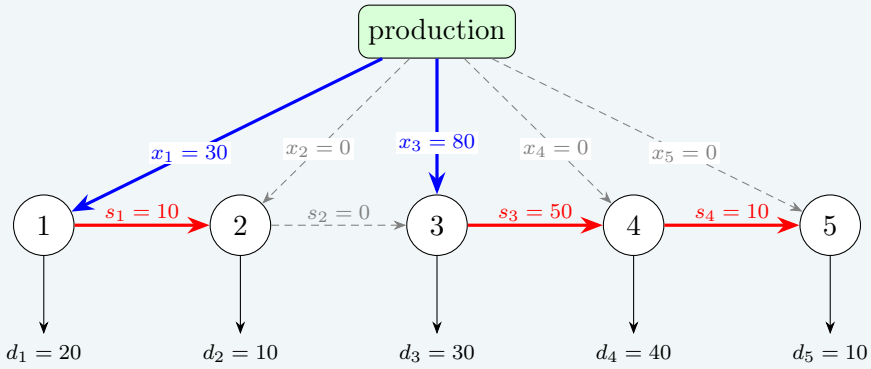
$$s_t \geq 0 \quad \forall t \in \{1, 2, \ldots, m-1\}, \qquad s_m = 0.$$

A setup is paid in period t if and only if $x_t > 0$. The total cost of a feasible solution is

$$\sum_{t=1,\ x_t>0}^{m} sc_t + \sum_{t=1}^{m} vc_t\, x_t + \sum_{t=1}^{m} hc_t\, s_t.$$

Example 17.2: A feasible solution

We consider the ULS instance of Example 17.1. The following network represents the feasible solution $(30, 0, 80, 0, 0)$: each period is a node, the production x_t enters node t, the demand d_t leaves it, and the inventory s_t flows from node t to node $t+1$. Thick arcs carry a positive flow.



The setups are in periods 1, 3, the inventories are $(s_1, \dots, s_5) = (10, 0, 50, 10, 0)$, and the total cost is (there is no variable production cost)

$$\underbrace{50 + 50}_{\text{setup}} + \underbrace{1 \cdot 10 + 1 \cdot 0 + 1 \cdot 50 + 1 \cdot 10 + 1 \cdot 0}_{\text{holding}} = 100 + 70 = 170.$$

- A production quantity can take many values, and a ULS instance has a huge number of feasible solutions (infinitely many, if the quantities are not required to be integer). The next property shows that only very few of them need to be considered.

17.1.2 The zero-inventory ordering property

Theorem 17.2: Zero-inventory ordering (Wagner and Whitin, 1958)

Every ULS instance has an optimal solution in which no period with positive production starts with a positive inventory, that is,

$$s_{t-1}\, x_t = 0 \qquad \forall t \in \{1, 2, \dots, m\}.$$

- *Proof.* First, an optimal solution exists: the vector $(d_1, d_2, \dots, d_m)$ is feasible, and for each of the 2^m choices of the set of periods with a setup, the remaining problem is a linear program over a bounded polyhedron (since $s_m = 0$, every x_t is between 0 and $D_{1,m}$), so it has an optimal solution whenever it is feasible.

- Among all optimal solutions, consider one with the **smallest number of periods with positive production**, and suppose, by contradiction, that there exists a period t with $x_t > 0$ and $s_{t-1} > 0$.

- Since $s_0 = 0$ and $s_{t-1} > 0$, something is produced before period t: let $j < t$ be the last period before t with $x_j > 0$. No unit is produced in periods $j+1, \dots, t-1$, so the inventory can only decrease from period j to period $t-1$:

$$s_k \geq s_{t-1} > 0 \qquad \forall k \in \{j, j+1, \dots, t-1\}.$$

- For a number ε, move ε units of production from period j to period t: replace x_j by $x_j - \varepsilon$, x_t by $x_t + \varepsilon$, and s_k by $s_k - \varepsilon$ for all $k \in \{j, \dots, t-1\}$. All balancing constraints remain satisfied, and the new plan is feasible for every ε with

$$-x_t \leq \varepsilon \leq \min\{x_j,\ s_{t-1}\}.$$

- As long as x_j and x_t stay positive, the setups do not change and the cost changes by $\varepsilon\,\Delta$, where

$$\Delta = vc_t - vc_j - \sum_{k=j}^{t-1} hc_k$$

  is the cost of producing one unit in period t minus the cost of producing it in period j and holding it until period t.

- If $\Delta < 0$, a small $\varepsilon > 0$ gives a feasible solution with a smaller cost, which contradicts optimality.

- If $\Delta \geq 0$, choose $\varepsilon = -x_t$: the whole production of period t is moved to period j, the cost changes by $-x_t\,\Delta \leq 0$, and the setup cost $sc_t \geq 0$ of period t is saved. The new solution is optimal and has one period with positive production less, which contradicts the choice of the solution.

- Hence no such period t exists. □

In an optimal solution with the zero-inventory ordering property, each period j with positive production produces exactly the demands of periods $j, j+1, \dots, t$, where $t+1$ is the next period with positive production (or $t = m$ if there is none):

$$x_j = D_{j,t}.$$

- Indeed, $s_{j-1} = 0$ (zero-inventory ordering in period j), $s_t = 0$ (zero-inventory ordering in period $t+1$, or $s_m = 0$), and nothing is produced in periods $j+1, \dots, t$; summing the balancing constraints of periods $j, \dots, t$ gives $x_j = D_{j,t}$.

- Period 1 always produces, since $s_0 = 0$ and $d_1 \geq 1$. A solution with the zero-inventory ordering property is therefore determined by the set of its production periods, which contains period 1 and any subset of $\{2, 3, \dots, m\}$.

The number of ULS solutions with the zero-inventory ordering property is 2^{m-1}, since each period $t \in \{2, 3, \dots, m\}$ can independently produce or not.

- The number of these solutions grows exponentially with the number m of periods:

$$2^{m-1} = O(2^m).$$

Therefore, enumerating them quickly becomes impractical as m increases.

Example 17.3: Enumeration of the solutions with the zero-inventory ordering property

We consider the ULS instance of Example 17.1:

t	1	2	3	4	5
d_t	20	10	30	40	10
sc_t	50	50	50	50	50
vc_t	0	0	0	0	0
hc_t	1	1	1	1	1

There are $2^{5-1} = 16$ solutions with the zero-inventory ordering property. For each of them, the table reports the production periods, the solution vector, the inventories, and the total cost, written as setup cost plus holding cost. The optimal costs are highlighted.

Label	Production periods	Solution vector	Inventories $(s_1, \dots, s_5)$	Total cost
(1)	$\{1\}$	$(110, 0, 0, 0, 0)$	$(90, 80, 50, 10, 0)$	$50 + 230 = 280$
(2)	$\{1, 5\}$	$(100, 0, 0, 0, 10)$	$(80, 70, 40, 0, 0)$	$100 + 190 = 290$
(3)	$\{1, 4\}$	$(60, 0, 0, 50, 0)$	$(40, 30, 0, 10, 0)$	$100 + 80 = 180$
(4)	$\{1, 4, 5\}$	$(60, 0, 0, 40, 10)$	$(40, 30, 0, 0, 0)$	$150 + 70 = 220$
(5)	$\{1, 3\}$	$(30, 0, 80, 0, 0)$	$(10, 0, 50, 10, 0)$	$100 + 70 = 170$
(6)	$\{1, 3, 5\}$	$(30, 0, 70, 0, 10)$	$(10, 0, 40, 0, 0)$	$150 + 50 = 200$
(7)	$\{1, 3, 4\}$	$(30, 0, 30, 50, 0)$	$(10, 0, 0, 10, 0)$	$150 + 20 = 170$
(8)	$\{1, 3, 4, 5\}$	$(30, 0, 30, 40, 10)$	$(10, 0, 0, 0, 0)$	$200 + 10 = 210$
(9)	$\{1, 2\}$	$(20, 90, 0, 0, 0)$	$(0, 80, 50, 10, 0)$	$100 + 140 = 240$
(10)	$\{1, 2, 5\}$	$(20, 80, 0, 0, 10)$	$(0, 70, 40, 0, 0)$	$150 + 110 = 260$
(11)	$\{1, 2, 4\}$	$(20, 40, 0, 50, 0)$	$(0, 30, 0, 10, 0)$	$150 + 40 = 190$
(12)	$\{1, 2, 4, 5\}$	$(20, 40, 0, 40, 10)$	$(0, 30, 0, 0, 0)$	$200 + 30 = 230$
(13)	$\{1, 2, 3\}$	$(20, 10, 80, 0, 0)$	$(0, 0, 50, 10, 0)$	$150 + 60 = 210$
(14)	$\{1, 2, 3, 5\}$	$(20, 10, 70, 0, 10)$	$(0, 0, 40, 0, 0)$	$200 + 40 = 240$
(15)	$\{1, 2, 3, 4\}$	$(20, 10, 30, 50, 0)$	$(0, 0, 0, 10, 0)$	$200 + 10 = 210$
(16)	$\{1, 2, 3, 4, 5\}$	$(20, 10, 30, 40, 10)$	$(0, 0, 0, 0, 0)$	$250 + 0 = 250$

The optimal value of the ULS instance is 170, attained by 2 solutions: (5) and (7).

17.2 A DP algorithm for the ULS

17.2.1 The cost of a production run

- By the zero-inventory ordering property, an optimal solution is a sequence of **production runs**: a run produces in a period j the demands of periods $j, \ldots, t$, and the next run starts in period $t+1$ with an empty inventory.

- For any $j, t \in T$ with $j \leq t$, let $c_{j,t}$ be the cost of the production run that produces in period j the demands of periods $j, \ldots, t$:

$$c_{j,t} = sc_j + vc_j\, D_{j,t} + \sum_{k=j}^{t-1} hc_k\, D_{k+1,t}. \tag{17.7}$$

- The first term is the setup cost of period j, the second term is the variable cost of the $D_{j,t}$ produced units, and the last term is the holding cost: at the end of each period $k \in \{j, \ldots, t-1\}$ the inventory contains the demands $D_{k+1,t}$ of the following periods of the run.

- For a fixed period j, the costs $c_{j,j}, c_{j,j+1}, \ldots, c_{j,m}$ can be computed one after the other. A unit used in period t and produced in period j costs

$$u_{j,t} = vc_j + \sum_{k=j}^{t-1} hc_k,$$

so that $u_{j,j} = vc_j$, $u_{j,t} = u_{j,t-1} + hc_{t-1}$, and adding period t to the run adds d_t units of cost $u_{j,t}$:

$$c_{j,j} = sc_j + vc_j\, d_j, \qquad c_{j,t} = c_{j,t-1} + u_{j,t}\, d_t \quad \text{if } t > j.$$

This is the pseudo-code that computes the matrix of the production-run costs:

Algorithm

Algorithm: CostMatrix

Input: A ULS instance given by the number m of periods and the arrays $d[1..m]$, $sc[1..m]$, $vc[1..m]$, $hc[1..m]$

Output: The matrix of the production-run costs

```
1  Function CostMatrix(m, d, sc, vc, hc)
2      c := new array[1..m][1..m]
3      for j := 1 to m do
4          u := vc[j]
5          g := sc[j] + vc[j] · d[j]
6          c[j][j] ← g
7          for t := j + 1 to m do
8              u ← u + hc[t − 1]
9              g ← g + u · d[t]
10             c[j][t] ← g
11         end
12     end
13     return c
```

- The variable u is the unit cost $u_{j,t}$ and the variable g is the cost $c_{j,t}$ of the current run; only the cells $c[j][t]$ with $j \leq t$ are used.

Example 17.4: Computation of the production-run costs

We consider the ULS instance of Example 17.1:

t	1	2	3	4	5
d_t	20	10	30	40	10
sc_t	50	50	50	50	50
vc_t	0	0	0	0	0
hc_t	1	1	1	1	1

For each step, the table shows the values of j and t, the current value of u, the cell $c[j][t]$, and the operation performed; the cell $c[j][t]$ is highlighted in blue when assigned.

j	t	u	$c[j][t]$	Action
1	1	0	50	$u \leftarrow vc[1] = 0$, $g \leftarrow sc[1]+vc[1] \cdot d[1] = 50+0 \cdot 20 = 50$, $c[1][1] \leftarrow 50$
1	2	1	60	$u \leftarrow u+hc[1] = 0+1 = 1$, $g \leftarrow g+u \cdot d[2] = 50+1 \cdot 10 = 60$, $c[1][2] \leftarrow 60$
1	3	2	120	$u \leftarrow u+hc[2] = 1+1 = 2$, $g \leftarrow g+u \cdot d[3] = 60+2 \cdot 30 = 120$, $c[1][3] \leftarrow 120$
1	4	3	240	$u \leftarrow u+hc[3] = 2+1 = 3$, $g \leftarrow g+u \cdot d[4] = 120+3 \cdot 40 = 240$, $c[1][4] \leftarrow 240$
1	5	4	280	$u \leftarrow u+hc[4] = 3+1 = 4$, $g \leftarrow g+u \cdot d[5] = 240+4 \cdot 10 = 280$, $c[1][5] \leftarrow 280$
2	2	0	50	$u \leftarrow vc[2] = 0$, $g \leftarrow sc[2]+vc[2] \cdot d[2] = 50+0 \cdot 10 = 50$, $c[2][2] \leftarrow 50$
2	3	1	80	$u \leftarrow u+hc[2] = 0+1 = 1$, $g \leftarrow g+u \cdot d[3] = 50+1 \cdot 30 = 80$, $c[2][3] \leftarrow 80$
2	4	2	160	$u \leftarrow u+hc[3] = 1+1 = 2$, $g \leftarrow g+u \cdot d[4] = 80+2 \cdot 40 = 160$, $c[2][4] \leftarrow 160$
2	5	3	190	$u \leftarrow u+hc[4] = 2+1 = 3$, $g \leftarrow g+u \cdot d[5] = 160+3 \cdot 10 = 190$, $c[2][5] \leftarrow 190$
3	3	0	50	$u \leftarrow vc[3] = 0$, $g \leftarrow sc[3]+vc[3] \cdot d[3] = 50+0 \cdot 30 = 50$, $c[3][3] \leftarrow 50$
3	4	1	90	$u \leftarrow u+hc[3] = 0+1 = 1$, $g \leftarrow g+u \cdot d[4] = 50+1 \cdot 40 = 90$, $c[3][4] \leftarrow 90$
3	5	2	110	$u \leftarrow u+hc[4] = 1+1 = 2$, $g \leftarrow g+u \cdot d[5] = 90+2 \cdot 10 = 110$, $c[3][5] \leftarrow 110$
4	4	0	50	$u \leftarrow vc[4] = 0$, $g \leftarrow sc[4]+vc[4] \cdot d[4] = 50+0 \cdot 40 = 50$, $c[4][4] \leftarrow 50$
4	5	1	60	$u \leftarrow u+hc[4] = 0+1 = 1$, $g \leftarrow g+u \cdot d[5] = 50+1 \cdot 10 = 60$, $c[4][5] \leftarrow 60$
5	5	0	50	$u \leftarrow vc[5] = 0$, $g \leftarrow sc[5]+vc[5] \cdot d[5] = 50+0 \cdot 10 = 50$, $c[5][5] \leftarrow 50$

This is the matrix c after the execution of the algorithm (for instance, $c_{1,3} = sc_1 + vc_1\, D_{1,3} + hc_1\, D_{2,3} + hc_2\, D_{3,3} = 50 + 0 \cdot 60 + 1 \cdot 40 + 1 \cdot 30 = 120$ by formula (17.7)):

$c[j][t]$	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
$j=1$	50	60	120	240	280
$j=2$	–	50	80	160	190
$j=3$	–	–	50	90	110
$j=4$	–	–	–	50	60
$j=5$	–	–	–	–	50

17.2.2 Step 1 and Step 2

- Step 1: computation of the optimal value

For any $t \in \{1, 2, \ldots, m\}$, let $v(t)$ be the optimal value of the ULS subinstance given by:

1. a set $\{1, 2, \ldots, t\}$ of t time periods,
2. for each period $j \in \{1, 2, \ldots, t\}$, a demand d_j,
3. for each period $j \in \{1, 2, \ldots, t\}$, a setup cost sc_j, a variable production cost vc_j, and a holding cost hc_j.

The recursive formula to compute $v(t)$ reads as follows:

$$v(t) = \begin{cases} \min \left\{ v(j-1) + c_{j,t} : j \in \{1, 2, \ldots, t\} \right\} & \text{if } 1 \leq t \leq m, \\ 0 & \text{if } t = 0. \end{cases} \tag{17.8}$$

– The subinstance with periods $\{1, \ldots, t\}$ is a ULS instance of the same type: its inventory must be empty at the end of period t. Here, $v(0) = 0$, since an empty set of periods has no demand and no cost.

– The recursive decomposition process of formula (17.8) generates t **independent subproblems**, each corresponding to a possible choice of the period j of the **last production run**, which produces the demands of periods $j, \ldots, t$. By the zero-inventory ordering property applied to the subinstance, the inventory is empty at the end of period $j-1$, and the residual subproblem is the ULS subinstance with periods $\{1, \ldots, j-1\}$.

– The optimal value $v(t)$ is obtained by trying all possible periods of the last production run and selecting the one that minimizes the sum $v(j-1) + c_{j,t}$ of the optimal value of the residual subproblem and the cost of the last run.

– Hence, an optimal solution for periods $\{1, \ldots, t\}$ is built by combining the chosen last production run in period j with an optimal solution of the residual subproblem with periods $\{1, \ldots, j-1\}$.

$\rightsquigarrow$ **Optimal Substructure verified!**

– Different choices of the last production run, possibly made for different values of t, can lead to the same residual set of periods, and therefore the same value $v(t)$ may need to be computed several times during the recursive evaluation of formula (17.8).

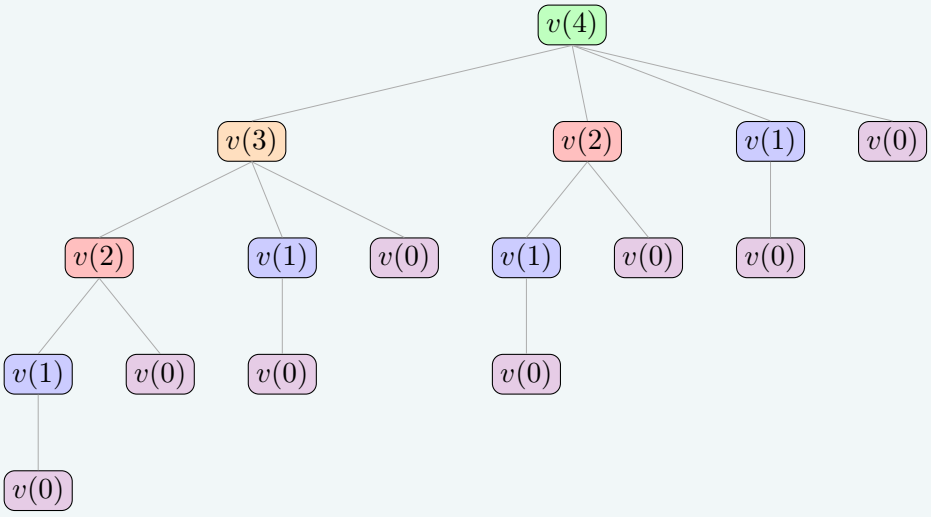
$\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(m)$ of the ULS instance is the **top-down approach**. The method starts from the call with $t = m$ and evaluates formula (17.8) recursively: each call for periods $\{1, \ldots, t\}$ triggers further calls for periods $\{1, \ldots, j-1\}$, for all $j \in \{1, 2, \ldots, t\}$, until the base case $v(0)$ is reached.

– The **recursion tree** of the function v has one node for each recursive call, labeled with the function call it represents. The root represents the initial call. The children of a node represent the recursive calls made by that call. The leaf nodes represent the base cases.

Example 17.5: Recursion tree

We consider the first four periods of the ULS instance of Example 17.1 (with five periods the tree has 32 nodes). This is the recursion tree of the top-down approach to compute $v(4)$:



The children of a node $v(t)$ are ordered by $j \in \{1, \ldots, t\}$: the child $v(j-1)$ corresponds to a last production run in period j. The function v is called 16 times, and the same subproblems are solved multiple times: $v(2)$ is called 2 times, $v(1)$ is called 4 times, and $v(0)$ is called 8 times. All the other calls are made only once.

– The value $v(t)$, for a generic period t, can be computed once the values $v(j)$ are known for all smaller periods $j < t$.

Example 17.6: Computing the values of the v function

We consider the ULS instance of Example 17.1, with the matrix c of the production-run costs:

$c[j][t]$	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
$j=1$	50	60	120	240	280
$j=2$	–	50	80	160	190
$j=3$	–	–	50	90	110
$j=4$	–	–	–	50	60
$j=5$	–	–	–	–	50

The value of the function v can be computed by starting from the base case $v(0) = 0$ and then computing the values of $v(1)$, $v(2)$, $v(3)$, $v(4)$, and $v(5)$ in order, as follows:

* Since we know the value of $v(0)$, we can compute $v(1)$, as follows:

$$v(1) = v(0) + c_{1,1} = 0 + 50 = 50$$

* Once we have computed $v(1)$ and $v(0)$, we can compute $v(2)$, as follows:

$$\begin{aligned} v(2) &= \min\{v(0) + c_{1,2}, v(1) + c_{2,2}\} \\ &= \min\{0 + 60, 50 + 50\} = 60 \end{aligned}$$

* Once we have computed $v(2)$, $v(1)$ and $v(0)$, we can compute $v(3)$, as follows:

$$\begin{aligned} v(3) &= \min\{v(0) + c_{1,3}, v(1) + c_{2,3}, v(2) + c_{3,3}\} \\ &= \min\{0 + 120, 50 + 80, 60 + 50\} = 110 \end{aligned}$$

* Once we have computed $v(3)$, $v(2)$, $v(1)$ and $v(0)$, we can compute $v(4)$, as follows:

$$\begin{aligned} v(4) &= \min\{v(0) + c_{1,4}, v(1) + c_{2,4}, v(2) + c_{3,4}, v(3) + c_{4,4}\} \\ &= \min\{0 + 240, 50 + 160, 60 + 90, 110 + 50\} = 150 \end{aligned}$$

* Once we have computed $v(4)$, $v(3)$, $v(2)$, $v(1)$ and $v(0)$, we can compute $v(5)$, as follows:

$$\begin{aligned} v(5) &= \min\{v(0) + c_{1,5}, v(1) + c_{2,5}, v(2) + c_{3,5}, v(3) + c_{4,5}, v(4) + c_{5,5}\} \\ &= \min\{0 + 280, 50 + 190, 60 + 110, 110 + 60, 150 + 50\} = 170 \end{aligned}$$

– We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed. The values are stored in a **lookup table**, the vector $v[0..m]$, whose cell $v[t]$ contains $v(t)$.

– Since solving a subproblem with value $v(t)$ depends only on subproblems with values $v(j-1)$, with $j-1 < t$, we can solve the subproblems for increasing values of t, starting from the base case $v(0) = 0$ and building up to $v(m)$: this observation

naturally leads to a **much more efficient method** to compute the optimal value $v(m)$ of the ULS instance.

– A **second method** to compute the optimal value $v(m)$ of the ULS instance is the **bottom-up approach**. The method starts from the base case $v(0) = 0$ and evaluates formula (17.8) for increasing periods $t \in \{1, 2, \ldots, m\}$: each value $v(t)$ is computed using the already-computed values $v(0), v(1), \ldots, v(t-1)$, until the value $v(m)$ of the ULS instance is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: DP_WW

Input: A ULS instance given by the number m of periods and the arrays $d[1..m]$, $sc[1..m]$, $vc[1..m]$, $hc[1..m]$

Output: The optimal value

```
1  Function DP_WW(m, d, sc, vc, hc)
2      c := CostMatrix(m, d, sc, vc, hc)
3      v := new array[0..m]
4      v[0] ← 0
5      for t := 1 to m do
6          b := +∞
7          for j := 1 to t do
8              if v[j − 1] + c[j][t] < b then
9                  b ← v[j − 1] + c[j][t]
10             end
11         end
12         v[t] ← b
13     end
14     return v[m]
```

* The matrix c contains the production-run costs, computed by the algorithm CostMatrix.
* The vector $v[0..m]$ is the **lookup table**: it stores the optimal values of all subproblems, with the base case $v[0] = 0$.
* The outer loop, over $t \in \{1, 2, \ldots, m\}$, computes $v[t]$ in increasing order of the last period.
* The inner loop, over $j \in \{1, 2, \ldots, t\}$, evaluates all possible periods of the last production run and keeps in b the smallest value $v[j-1] + c[j][t]$; the variable b starts from $+\infty$, so the first comparison always updates it.
* At the end of the inner loop, $v[t] \leftarrow b$ stores the optimal value $v[t]$.

∗ Finally, the optimal value $v[m]$ of the ULS instance is returned.

Example 17.7: Execution of the DP algorithm

We consider the ULS instance of Example 17.1, with the matrix c of the production-run costs:

$c[j][t]$	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
$j=1$	50	60	120	240	280
$j=2$	–	50	80	160	190
$j=3$	–	–	50	90	110
$j=4$	–	–	–	50	60
$j=5$	–	–	–	–	50

For each step of the DP algorithm, the table shows the values of t and j, the current value of b, and the cell $v[t]$. The cell of b is highlighted in red when updated; the cell of $v[t]$ is highlighted in blue when assigned.

t	j	$v[t]$	b	Action
–	–	0	–	$v[0] \leftarrow 0$
	–	–	$+\infty$	$b \leftarrow +\infty$
1	1	–	50	$v[0]+c[1][1] = 0+50 = 50 < +\infty$: $b \leftarrow 50$
	–	50	50	$v[1] \leftarrow b = 50$
	–	–	$+\infty$	$b \leftarrow +\infty$
2	1	–	60	$v[0]+c[1][2] = 0+60 = 60 < +\infty$: $b \leftarrow 60$
	2	–	60	$v[1]+c[2][2] = 50+50 = 100 > 60$: no update
	–	60	60	$v[2] \leftarrow b = 60$
	–	–	$+\infty$	$b \leftarrow +\infty$
3	1	–	120	$v[0]+c[1][3] = 0+120 = 120 < +\infty$: $b \leftarrow 120$
	2	–	120	$v[1]+c[2][3] = 50+80 = 130 > 120$: no update
	3	–	110	$v[2]+c[3][3] = 60+50 = 110 < 120$: $b \leftarrow 110$
	–	110	110	$v[3] \leftarrow b = 110$
	–	–	$+\infty$	$b \leftarrow +\infty$
4	1	–	240	$v[0]+c[1][4] = 0+240 = 240 < +\infty$: $b \leftarrow 240$
	2	–	210	$v[1]+c[2][4] = 50+160 = 210 < 240$: $b \leftarrow 210$
	3	–	150	$v[2]+c[3][4] = 60+90 = 150 < 210$: $b \leftarrow 150$
	4	–	150	$v[3]+c[4][4] = 110+50 = 160 > 150$: no update
	–	150	150	$v[4] \leftarrow b = 150$
	–	–	$+\infty$	$b \leftarrow +\infty$
5	1	–	280	$v[0]+c[1][5] = 0+280 = 280 < +\infty$: $b \leftarrow 280$
	2	–	240	$v[1]+c[2][5] = 50+190 = 240 < 280$: $b \leftarrow 240$
	3	–	170	$v[2]+c[3][5] = 60+110 = 170 < 240$: $b \leftarrow 170$
	4	–	170	$v[3]+c[4][5] = 110+60 = 170 = b$: no update
	5	–	170	$v[4]+c[5][5] = 150+50 = 200 > 170$: no update
	–	170	170	$v[5] \leftarrow b = 170$

This is the vector v after the execution of the DP algorithm:

	$t=0$	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
$v[t]$	0	50	60	110	150	170

- After the execution of the DP algorithm, the vector v contains the optimal value $v[t]$ for every set of periods $\{1, \ldots, t\}$.

- *Remark (shortest path).* Formula (17.8) computes a shortest path in a network with nodes $0, 1, \ldots, m$ and one arc from node $j-1$ to node t, of cost $c_{j,t}$, for every $j \leq t$. The value $v(t)$ is the length of a shortest path from node 0 to node t. This is the shortest-path formulation of the lot-sizing chapters: an arc is a production run.

- Step 2: computation of an optimal solution
  - To compute an optimal solution, it is necessary to store some additional information.
  - For each period t, in addition to the optimal value $v[t]$, we store $r[t]$, the period of the last production run in an optimal solution for periods $\{1, \ldots, t\}$.
  - Starting from $t = m$, the value $r[t]$ gives the period of the last production run, which produces $D_{r[t],t}$ units; the remaining periods are $\{1, \ldots, r[t]-1\}$, and the same rule can be applied again, and so on, until no period remains.

    This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to compute an optimal solution:

Algorithm

Algorithm: DP_WW_InfoSol

Input: A ULS instance given by the number m of periods and the arrays $d[1..m]$, $sc[1..m]$, $vc[1..m]$, $hc[1..m]$

Output: The optimal value and the information vector $r[1..m]$

```
1  Function DP_WW_InfoSol(m, d, sc, vc, hc)
2      c := CostMatrix(m, d, sc, vc, hc)
3      v := new array[0..m]
4      r := new array[1..m]
5      v[0] ← 0
6      for t := 1 to m do
7          b := +∞
8          a := 0
9          for j := 1 to t do
10             if v[j − 1] + c[j][t] < b then
11                 b ← v[j − 1] + c[j][t]
12                 a ← j
13             end
14         end
15         v[t] ← b
16         r[t] ← a
17     end
18     return v[m], r
```

* The DP algorithm uses the additional temporary variable a to keep track of the decision made when b is updated. If the last production run in period j is selected, $a \leftarrow j$ stores this decision. At the end of the inner loop, $r[t] \leftarrow a$ stores the period of the selected last production run, together with the optimal value $v[t] \leftarrow b$.

* Because of the strict comparison, in case of ties the smallest period j is stored.

Example 17.8: Computation of the additional information

We consider the ULS instance of Example 17.1, with the matrix c of the production-run costs:

$c[j][t]$	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
$j=1$	50	60	120	240	280
$j=2$	–	50	80	160	190
$j=3$	–	–	50	90	110
$j=4$	–	–	–	50	60
$j=5$	–	–	–	–	50

For each step of the DP algorithm, the table shows the values of t and j, the cell $r[t]$, the current value of b, and the current value of a. The cell of b is highlighted

in red when updated; the cell of a and the cell of $r[t]$ are highlighted in blue when updated.

t	j	$r[t]$	b	a	Action
–	–	–	–	–	Initial state
	–	–	$+\infty$	0	$b \leftarrow +\infty$, $a \leftarrow 0$
1	1	–	50	1	$v[0]+c[1][1] = 0+50 = 50 < +\infty$: $b \leftarrow 50$, $a \leftarrow 1$
	–	1	50	1	$v[1] \leftarrow b = 50$, $r[1] \leftarrow a = 1$
	–	–	$+\infty$	0	$b \leftarrow +\infty$, $a \leftarrow 0$
2	1	–	60	1	$v[0]+c[1][2] = 0+60 = 60 < +\infty$: $b \leftarrow 60$, $a \leftarrow 1$
	2	–	60	1	$v[1]+c[2][2] = 50+50 = 100 > 60$: no update
	–	1	60	1	$v[2] \leftarrow b = 60$, $r[2] \leftarrow a = 1$
	–	–	$+\infty$	0	$b \leftarrow +\infty$, $a \leftarrow 0$
3	1	–	120	1	$v[0]+c[1][3] = 0+120 = 120 < +\infty$: $b \leftarrow 120$, $a \leftarrow 1$
	2	–	120	1	$v[1]+c[2][3] = 50+80 = 130 > 120$: no update
	3	–	110	3	$v[2]+c[3][3] = 60+50 = 110 < 120$: $b \leftarrow 110$, $a \leftarrow 3$
	–	3	110	3	$v[3] \leftarrow b = 110$, $r[3] \leftarrow a = 3$
	–	–	$+\infty$	0	$b \leftarrow +\infty$, $a \leftarrow 0$
4	1	–	240	1	$v[0]+c[1][4] = 0+240 = 240 < +\infty$: $b \leftarrow 240$, $a \leftarrow 1$
	2	–	210	2	$v[1]+c[2][4] = 50+160 = 210 < 240$: $b \leftarrow 210$, $a \leftarrow 2$
	3	–	150	3	$v[2]+c[3][4] = 60+90 = 150 < 210$: $b \leftarrow 150$, $a \leftarrow 3$
	4	–	150	3	$v[3]+c[4][4] = 110+50 = 160 > 150$: no update
	–	3	150	3	$v[4] \leftarrow b = 150$, $r[4] \leftarrow a = 3$
	–	–	$+\infty$	0	$b \leftarrow +\infty$, $a \leftarrow 0$
5	1	–	280	1	$v[0]+c[1][5] = 0+280 = 280 < +\infty$: $b \leftarrow 280$, $a \leftarrow 1$
	2	–	240	2	$v[1]+c[2][5] = 50+190 = 240 < 280$: $b \leftarrow 240$, $a \leftarrow 2$
	3	–	170	3	$v[2]+c[3][5] = 60+110 = 170 < 240$: $b \leftarrow 170$, $a \leftarrow 3$
	4	–	170	3	$v[3]+c[4][5] = 110+60 = 170 = b$: no update
	5	–	170	3	$v[4]+c[5][5] = 150+50 = 200 > 170$: no update
	–	3	170	3	$v[5] \leftarrow b = 170$, $r[5] \leftarrow a = 3$

This is the vector r after the execution of the DP algorithm:

	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
$r[t]$	1	1	3	3	3

– Once the vector r has been computed, an optimal solution of the ULS instance can be computed by repeatedly applying the following rule, starting from $t = m$ and $x_j = 0$ for all $j \in \{1, 2, \ldots, m\}$:

$$x_{r[t]} \leftarrow D_{r[t],t}, \qquad t \leftarrow r[t] - 1,$$

until $t = 0$. This yields the vector $(x_1, x_2, \ldots, x_m)$, where each entry x_j is the number of units produced in period j in an optimal solution.

– The following pseudo-code implements the computation of an optimal solution, given the vector r.

Algorithm

Algorithm: DP_WW_SolComputation

Input: The number m of periods and the demands $d[1..m]$ of the ULS instance, and the vector $r[1..m]$

Output: An optimal solution of the ULS instance

```
1  Function DP_WW_SolComputation(m, d, r)
2      x := new array[1..m]
3      for j := 1 to m do
4          x[j] ← 0
5      end
6      t := m
7      while t > 0 do
8          x[r[t]] ← Σ_{k=r[t]}^{t} d[k]
9          t ← r[t] − 1
10     end
11     return x
```

Example 17.9: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 17.1, we have the following vector r:

	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
$r[t]$	1	1	3	3	3

We can compute an optimal solution for $m = 5$ periods as follows:

Iteration	t	$r[t]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$x[5]$	$t \leftarrow r[t]-1$
–	–	–	0	0	0	0	0	$t=5$
1	5	3	0	0	80	0	0	$t=3-1=2$
2	2	1	30	0	80	0	0	$t=1-1=0$

* $t = 5$, $r[5] = 3$: the last production run of periods $1, \ldots, 5$ is in period 3 and covers the demands of periods $3, \ldots, 5$, so $x[3] \leftarrow d_3 + d_4 + d_5 = 30 + 40 + 10 = 80$
* $t = 2$, $r[2] = 1$: the last production run of periods $1, \ldots, 2$ is in period 1 and covers the demands of periods $1, \ldots, 2$, so $x[1] \leftarrow d_1 + d_2 = 20 + 10 = 30$

An optimal solution is the vector:

$$(30, 0, 80, 0, 0)$$

The corresponding optimal value is $v[5] = 170$, the same value obtained by the enumeration of Example 17.3 and by Gurobi on the MIP model.

– *Remark (ties).* In the computation of $v[5]$, the run in period 4 gives the same value

as the run in period 3, but the strict comparison keeps $j = 3$. Choosing $j = 4$ instead leads to the other optimal solution $(30, 0, 30, 50, 0)$ of Example 17.3. The DP algorithm returns one optimal solution, not all of them.

17.2.3 Number of steps

- Let $S_c(m)$ denote the number of steps executed by CostMatrix in the worst case, where m is the number of periods. The number of executions of its lines is:
  - Line 2 is executed 1 time.
  - Line 3, Line 4, Line 5, and Line 6 are executed m times each.
  - Line 7, Line 8, Line 9, and Line 10 are executed $\sum_{j=1}^{m}(m-j) = \frac{m(m-1)}{2}$ times each.
  - Line 13 is executed 1 time.

  Therefore,
  $$S_c(m) = 2 + 4m + 4 \cdot \frac{m(m-1)}{2} = 2m^2 + 2m + 2.$$
- Let $S(m)$ denote the number of steps executed by DP_WW in the worst case. Line 2 executes CostMatrix, that is, $S_c(m)$ steps. In the worst case, the condition at line 8 is true at every iteration of the inner loop, and therefore line 9 is executed as often as line 8. The number of executions of the other lines is:
  - Line 3 is executed 1 time.
  - Line 4 is executed 1 time.
  - Line 5 is executed m times.
  - Line 6 is executed m times.
  - Line 7 is executed $\sum_{t=1}^{m} t$ times.
  - Line 8 is executed $\sum_{t=1}^{m} t$ times.
  - Line 9 is executed $\sum_{t=1}^{m} t$ times.
  - Line 12 is executed m times.
  - Line 14 is executed 1 time.

  Therefore, the total number of steps in the worst case is
  $$S(m) = S_c(m)+3+3m+3\sum_{t=1}^{m} t = 2m^2+2m+2+3+3m+3\cdot\frac{m(m+1)}{2} = \frac{7m^2+13m+10}{2}.$$
  Hence,
  $$S(m) = \Theta(m^2).$$
  Indeed,
  $$\lim_{m\to\infty} \frac{\frac{7m^2+13m+10}{2}}{m^2} = \lim_{m\to\infty} \frac{1}{2}\left(7 + \frac{13}{m} + \frac{10}{m^2}\right) = \frac{7}{2} \in \mathbb{R}_{>0}.$$

The DP algorithm executes

$$S(m) = \Theta(m^2)$$

steps in the worst case.

- DP_WW_InfoSol only adds a constant number of operations inside the same loops, and therefore its number of steps has the same order of growth.

17.3 A DP algorithm for the capacitated case

- In most production systems the quantity that can be produced in a period is limited by the capacity of the resource.

Definition 17.3: Capacitated Lot-Sizing Problem

Given:

1. a set $T = \{1, 2, \ldots, m\}$ of $m \in \mathbb{Z}_{\geq 1}$ *time periods*,
2. for each period $t \in T$, a *demand* $d_t \in \mathbb{Z}_{\geq 1}$,
3. for each period $t \in T$, a *setup cost* $sc_t \in \mathbb{Z}_{\geq 0}$, a *variable production cost* $vc_t \in \mathbb{Z}_{\geq 0}$ per unit, and a *holding cost* $hc_t \in \mathbb{Z}_{\geq 0}$ per unit in stock at the end of period t,
4. for each period $t \in T$, a *capacity* $cap_t \in \mathbb{Z}_{\geq 1}$, the maximum number of units that can be produced in period t,

the **Capacitated Lot-Sizing Problem** (CLS) asks to decide how many units to produce in each period, without exceeding the capacities, so that, starting with an empty inventory and ending with an empty inventory, the demand of every period is satisfied, and the total cost of setups, production, and holding is minimized.

- The MIP model is the model of the ULS with the additional constraints $x_t \leq cap_t$, for all $t \in T$ (the big-M value of period t becomes $\min\{cap_t, D_{t,m}\}$). A feasible solution is encoded by the same vector $(x_1, x_2, \ldots, x_m)$ as in the ULS, with the additional requirement $x_t \leq cap_t$ for all $t \in T$.

- A CLS instance is feasible if and only if the capacities of the first t periods cover the demands of the first t periods, $\sum_{j=1}^{t} cap_j \geq D_{1,t}$, for all $t \in T$.

- Since all data are integer, we can restrict the production quantities to integer values: once the setups are fixed, the remaining problem is a minimum-cost flow problem with integer demands and capacities, which has an integer optimal solution.

- The **zero-inventory ordering property does not hold** for the CLS: a period may have to produce while units are still in stock, because the capacity does not allow to produce everything in the period where it would be cheapest.

Example 17.10: Capacitated lot sizing

We consider Example 17.1 measured in units of 10 products (hence the holding cost is 10 per unit), with a constant capacity of 4 units per period:

t	1	2	3	4	5
d_t	2	1	3	4	1
sc_t	50	50	50	50	50
vc_t	0	0	0	0	0
hc_t	10	10	10	10	10
cap_t	4	4	4	4	4

We refer to this CLS instance as Example 17.10. The optimal solution of Example 17.1, in units of 10, is the vector $(3, 0, 8, 0, 0)$, of cost 170: it is not feasible for Example 17.10, since period 3 produces more than 4 units.

- Step 1: computation of the optimal value
- For any period $t \in \{1, \dots, m\}$, the **production cost function** of period t is

$$f_t(x) = \begin{cases} 0 & \text{if } x = 0, \\ sc_t + vc_t\, x & \text{if } x \in \{1, 2, \dots, cap_t\}. \end{cases}$$

The inventory at the end of period t never exceeds the demand $D_{t+1,m}$ of the following periods, since the inventory must be empty at the end of period m.

For any $t \in \{1, 2, \dots, m\}$ and $k \in \{0, 1, \dots, D_{t+1,m}\}$, let $v(t, k)$ be the optimal value of the CLS subinstance given by:

1. a set $\{1, 2, \dots, t\}$ of t time periods,
2. for each period $j \in \{1, 2, \dots, t\}$, a demand d_j,
3. for each period $j \in \{1, 2, \dots, t\}$, a setup cost sc_j, a variable production cost vc_j, and a holding cost hc_j,
4. for each period $j \in \{1, 2, \dots, t\}$, a capacity cap_j,
5. an inventory equal to k at the end of period t.

The recursive formula to compute $v(t, k)$ reads as follows:

$$v(t,k) = \begin{cases} \min\left\{v(t-1, k+d_t-x) + f_t(x) : x \in X_t(k)\right\} & \text{if } 1 \le t \le m, \\ \quad + hc_t\, k & \\ 0 & \text{if } t = 0 \text{ and } k = 0, \\ +\infty & \text{if } t = 0 \text{ and } k > 0, \end{cases} \tag{17.9}$$

where $X_t(k) = \{0, 1, \ldots, \min\{cap_t,\, k + d_t\}\}$ is the set of the possible productions of period t.

- The pair (t, k) is the **state**: the last period of the subinstance and the inventory at its end. For $t = m$ and $k = 0$ we obtain the CLS instance itself, so its optimal value is $v(m, 0)$.
- The recursive decomposition process of formula (17.9) generates at most $cap_t + 1$ **independent subproblems**, each corresponding to a possible choice of the production x of the last period t.
- If x units are produced in period t and k units remain at its end, the inventory at the end of period $t-1$ is $k + d_t - x$: the residual subproblem is the CLS subinstance with periods $\{1, \ldots, t-1\}$ and final inventory $k + d_t - x$.
- The production x is at most cap_t, and at most $k + d_t$ because the inventory at the end of period $t-1$ cannot be negative. Moreover $k + d_t - x \leq D_{t+1,m} + d_t = D_{t,m}$, so the residual subproblem is a state.
- The base case is $v(0, 0) = 0$: no period, no inventory, no cost.
- For $k > 0$ we set $v(0, k) = +\infty$: the initial inventory is empty, so no feasible solution has a positive inventory before period 1. The value $+\infty$ rules out this case, and it propagates to every state that cannot be reached within the capacities, with the convention $+\infty + a = +\infty$.
- The optimal value $v(t, k)$ is obtained by trying all possible productions x of period t and selecting the one that minimizes the sum $v(t-1, k + d_t - x) + f_t(x)$ of the optimal value of the residual subproblem and the production cost of period t; the holding cost $hc_t\, k$ of the final inventory does not depend on x.
- Hence, an optimal solution of the subinstance of state (t, k) is built by combining the chosen production of period t with an optimal solution of the residual subproblem of state $(t-1, k + d_t - x)$.

  $\rightsquigarrow$ **Optimal Substructure verified!**

- Different sequences of production decisions can lead to the same state. For instance, in Example 17.10, starting from the state $(5, 0)$, the decisions $x_5 = 0$ and $x_4 = 4$ lead to the state $(3, 1)$, and the decisions $x_5 = 1$ and $x_4 = 3$ lead to the same state $(3, 1)$.
- Therefore the same value $v(t, k)$ may need to be computed several times during the recursive evaluation of formula (17.9).

  $\rightsquigarrow$ **Overlapping Subproblems verified!**

– A **first method** to compute the optimal value $v(m, 0)$ of the CLS instance is the **top-down approach**. The method starts from the call with $(t, k) = (m, 0)$ and evaluates formula (17.9) recursively: each call for a state (t, k) triggers further calls for the states $(t - 1, k + d_t - x)$, for all $x \in \{0, \ldots, \min\{cap_t, k + d_t\}\}$, until one of the base cases $v(0, k)$ is reached.

– The value $v(t, k)$ can be computed once the values $v(t - 1, k')$ are known for all $k' \in \{0, \ldots, D_{t,m}\}$. We can avoid solving the same subproblems repeatedly by storing the value of each subproblem as soon as it is computed, in a **lookup table**: the matrix $v[0..m][0..D_{1,m}]$, whose cell $v[t][k]$ contains $v(t, k)$ for $k \leq D_{t+1,m}$ (the other cells are not used).

– A **second method** to compute the optimal value $v(m, 0)$ of the CLS instance is the **bottom-up approach**. The method starts from the base cases $v(0, 0) = 0$ and $v(0, k) = +\infty$ for all $k \in \{1, \ldots, D_{1,m}\}$.

– It evaluates formula (17.9) for increasing values of $t \in \{1, \ldots, m\}$ and, for each fixed t, for $k \in \{0, 1, \ldots, D_{t+1,m}\}$: each value $v(t, k)$ uses already-computed values of row $t - 1$, until $v(m, 0)$ is obtained.

This is the pseudo-code of the **DP algorithm using the bottom-up approach**:

Algorithm

Algorithm: DP_CLS

Input: A CLS instance given by the number m of periods and the arrays $d[1..m]$, $sc[1..m]$, $vc[1..m]$, $hc[1..m]$, $cap[1..m]$

Output: The optimal value

```
1  Function DP_CLS(m, d, sc, vc, hc, cap)
2      R := new array[0..m]
3      R[m] ← 0
4      for t := m − 1 downto 0 do
5          R[t] ← R[t + 1] + d[t + 1]
6      end
7      v := new array[0..m][0..R[0]]
8      v[0][0] ← 0
9      for k := 1 to R[0] do
10         v[0][k] ← +∞
11     end
12     for t := 1 to m do
13         for k := 0 to R[t] do
14             b := +∞
15             for x := 0 to min{cap[t], k + d[t]} do
16                 if v[t − 1][k + d[t] − x] + f_t(x) + hc[t] · k < b then
17                     b ← v[t − 1][k + d[t] − x] + f_t(x) + hc[t] · k
18                 end
19             end
20             v[t][k] ← b
21         end
22     end
23     return v[m][0]
```

* The vector $R[0..m]$ contains the remaining demands $R[t] = D_{t+1,m}$, computed backwards from $R[m] = 0$: the states of period t are the pairs (t, k) with $k \leq R[t]$.

* The matrix v is the **lookup table**. The initialization sets $v[0][0] = 0$ and $v[0][k] = +\infty$ for all $k \in \{1, \ldots, R[0]\}$, since the initial inventory is empty.

* The outer loop, over $t \in \{1, 2, \ldots, m\}$, considers the periods in increasing order, and the second loop, over $k \in \{0, \ldots, R[t]\}$, considers all final inventories of period t.

* The inner loop, over the production x of period t, keeps in b the smallest value $v[t-1][k+d[t]-x] + f_t(x) + hc[t] \cdot k$; the value $f_t(x)$ is 0 for $x = 0$ and $sc[t] + vc[t] \cdot x$ otherwise. If all candidates are $+\infty$, the comparison is never true and $v[t][k] = +\infty$: the state (t, k) cannot be reached within the capacities.

* Finally, the optimal value $v[m][0]$ of the CLS instance is returned.

Example 17.11: Execution of the DP algorithm

We consider the CLS instance of Example 17.10:

t	1	2	3	4	5
d_t	2	1	3	4	1
sc_t	50	50	50	50	50
vc_t	0	0	0	0	0
hc_t	10	10	10	10	10
cap_t	4	4	4	4	4

The remaining demands are $(R[0], \ldots, R[5]) = (11, 9, 8, 5, 1, 0)$.

For each step of the DP algorithm, the table shows the values of t and k, the cell $v[t][k]$, and the operation performed. The candidates of the minimum are listed for $x = 0, 1, \ldots, \min\{cap_t, k + d_t\}$.

t	k	$v[t][k]$	Action
0	0	0	$v[0][0] \leftarrow 0$
0	1	$+\infty$	$v[0][1] \leftarrow +\infty$
0	2	$+\infty$	$v[0][2] \leftarrow +\infty$
0	3	$+\infty$	$v[0][3] \leftarrow +\infty$
0	4	$+\infty$	$v[0][4] \leftarrow +\infty$
0	5	$+\infty$	$v[0][5] \leftarrow +\infty$
0	6	$+\infty$	$v[0][6] \leftarrow +\infty$
0	7	$+\infty$	$v[0][7] \leftarrow +\infty$
0	8	$+\infty$	$v[0][8] \leftarrow +\infty$
0	9	$+\infty$	$v[0][9] \leftarrow +\infty$
0	10	$+\infty$	$v[0][10] \leftarrow +\infty$
0	11	$+\infty$	$v[0][11] \leftarrow +\infty$
1	0	50	$\min\{v[0][2]+f_1(0), v[0][1]+f_1(1), v[0][0]+f_1(2)\}+hc_1 \cdot 0 = \min\{+\infty, +\infty, 0+50\}+10 \cdot 0 = 50$
1	1	60	$\min\{v[0][3]+f_1(0), v[0][2]+f_1(1), v[0][1]+f_1(2), v[0][0]+f_1(3)\}+hc_1 \cdot 1 = \min\{+\infty, +\infty, +\infty, 0+50\}+10 \cdot 1 = 60$
1	2	70	$\min\{v[0][4]+f_1(0), v[0][3]+f_1(1), v[0][2]+f_1(2), v[0][1]+f_1(3), v[0][0]+f_1(4)\}+hc_1 \cdot 2 = \min\{+\infty, +\infty, +\infty, +\infty, 0+50\}+10 \cdot 2 = 70$
1	3	$+\infty$	the candidates use the cells $v[0][1], \ldots, v[0][5]$, which are all $+\infty$: $v[1][3] \leftarrow +\infty$
1	4	$+\infty$	the candidates use the cells $v[0][2], \ldots, v[0][6]$, which are all $+\infty$: $v[1][4] \leftarrow +\infty$
1	5	$+\infty$	the candidates use the cells $v[0][3], \ldots, v[0][7]$, which are all $+\infty$: $v[1][5] \leftarrow +\infty$
1	6	$+\infty$	the candidates use the cells $v[0][4], \ldots, v[0][8]$, which are all $+\infty$: $v[1][6] \leftarrow +\infty$
1	7	$+\infty$	the candidates use the cells $v[0][5], \ldots, v[0][9]$, which are all $+\infty$: $v[1][7] \leftarrow +\infty$
1	8	$+\infty$	the candidates use the cells $v[0][6], \ldots, v[0][10]$, which are all $+\infty$: $v[1][8] \leftarrow +\infty$
1	9	$+\infty$	the candidates use the cells $v[0][7], \ldots, v[0][11]$, which are all $+\infty$: $v[1][9] \leftarrow +\infty$
2	0	60	$\min\{v[1][1]+f_2(0), v[1][0]+f_2(1)\}+hc_2 \cdot 0 = \min\{60+0, 50+50\}+10 \cdot 0 = 60$
2	1	80	$\min\{v[1][2]+f_2(0), v[1][1]+f_2(1), v[1][0]+f_2(2)\}+hc_2 \cdot 1 = \min\{70+0, 60+50, 50+50\}+10 \cdot 1 = 80$
2	2	120	$\min\{v[1][3]+f_2(0), v[1][2]+f_2(1), v[1][1]+f_2(2), v[1][0]+f_2(3)\}+hc_2 \cdot 2 = \min\{+\infty, 70+50, 60+50, 50+50\}+10 \cdot 2 = 120$
2	3	130	$\min\{v[1][4]+f_2(0), v[1][3]+f_2(1), v[1][2]+f_2(2), v[1][1]+f_2(3), v[1][0]+f_2(4)\}+hc_2 \cdot 3 = \min\{+\infty, +\infty, 70+50, 60+50, 50+50\}+10 \cdot 3 = 130$
2	4	150	$\min\{v[1][5]+f_2(0), v[1][4]+f_2(1), v[1][3]+f_2(2), v[1][2]+f_2(3), v[1][1]+f_2(4)\}+hc_2 \cdot 4 = \min\{+\infty, +\infty, +\infty, 70+50, 60+50\}+10 \cdot 4 = 150$
2	5	170	$\min\{v[1][6]+f_2(0), v[1][5]+f_2(1), v[1][4]+f_2(2), v[1][3]+f_2(3), v[1][2]+f_2(4)\}+hc_2 \cdot 5 = \min\{+\infty, +\infty, +\infty, +\infty, 70+50\}+10 \cdot 5 = 170$
2	6	$+\infty$	the candidates use the cells $v[1][3], \ldots, v[1][7]$, which are all $+\infty$: $v[2][6] \leftarrow +\infty$
2	7	$+\infty$	the candidates use the cells $v[1][4], \ldots, v[1][8]$, which are all $+\infty$: $v[2][7] \leftarrow +\infty$
2	8	$+\infty$	the candidates use the cells $v[1][5], \ldots, v[1][9]$, which are all $+\infty$: $v[2][8] \leftarrow +\infty$
3	0	110	$\min\{v[2][3]+f_3(0), v[2][2]+f_3(1), v[2][1]+f_3(2), v[2][0]+f_3(3)\}+hc_3 \cdot 0 = \min\{130+0, 120+50, 80+50, 60+50\}+10 \cdot 0 = 110$

3	1	120	$\min\{v[2][4]+f_3(0), v[2][3]+f_3(1), v[2][2]+f_3(2), v[2][1]+f_3(3), v[2][0]+f_3(4)\}+hc_3 \cdot 1 = \min\{150+0, 130+50, 120+50, 80+50, 60+50\}+10 \cdot 1 = 120$
3	2	150	$\min\{v[2][5]+f_3(0), v[2][4]+f_3(1), v[2][3]+f_3(2), v[2][2]+f_3(3), v[2][1]+f_3(4)\}+hc_3 \cdot 2 = \min\{170+0, 150+50, 130+50, 120+50, 80+50\}+10 \cdot 2 = 150$
3	3	200	$\min\{v[2][6]+f_3(0), v[2][5]+f_3(1), v[2][4]+f_3(2), v[2][3]+f_3(3), v[2][2]+f_3(4)\}+hc_3 \cdot 3 = \min\{+\infty, 170+50, 150+50, 130+50, 120+50\}+10 \cdot 3 = 200$
3	4	220	$\min\{v[2][7]+f_3(0), v[2][6]+f_3(1), v[2][5]+f_3(2), v[2][4]+f_3(3), v[2][3]+f_3(4)\}+hc_3 \cdot 4 = \min\{+\infty, +\infty, 170+50, 150+50, 130+50\}+10 \cdot 4 = 220$
3	5	250	$\min\{v[2][8]+f_3(0), v[2][7]+f_3(1), v[2][6]+f_3(2), v[2][5]+f_3(3), v[2][4]+f_3(4)\}+hc_3 \cdot 5 = \min\{+\infty, +\infty, +\infty, 170+50, 150+50\}+10 \cdot 5 = 250$
4	0	160	$\min\{v[3][4]+f_4(0), v[3][3]+f_4(1), v[3][2]+f_4(2), v[3][1]+f_4(3), v[3][0]+f_4(4)\}+hc_4 \cdot 0 = \min\{220+0, 200+50, 150+50, 120+50, 110+50\}+10 \cdot 0 = 160$
4	1	180	$\min\{v[3][5]+f_4(0), v[3][4]+f_4(1), v[3][3]+f_4(2), v[3][2]+f_4(3), v[3][1]+f_4(4)\}+hc_4 \cdot 1 = \min\{250+0, 220+50, 200+50, 150+50, 120+50\}+10 \cdot 1 = 180$
5	0	180	$\min\{v[4][1]+f_5(0), v[4][0]+f_5(1)\}+hc_5 \cdot 0 = \min\{180+0, 160+50\}+10 \cdot 0 = 180$

This is the matrix v after the execution of the DP algorithm (a dot marks a pair (t, k) with $k > D_{t+1,m}$, which is not a state):

$v[t][k]$	0	1	2	3	4	5	6	7	8	9	10	11
$t=0$	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
$t=1$	50	60	70	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	·	·
$t=2$	60	80	120	130	150	170	$+\infty$	$+\infty$	$+\infty$	·	·	·
$t=3$	110	120	150	200	220	250	·	·	·	·	·	·
$t=4$	160	180	·	·	·	·	·	·	·	·	·	·
$t=5$	180	·	·	·	·	·	·	·	·	·	·	·

The optimal value of the CLS instance is $v[5][0] = 180$.

- After the execution of the DP algorithm, the matrix v contains the optimal value $v[t][k]$ for every state (t, k).

- Step 2: computation of an optimal solution
  - To compute an optimal solution, it is necessary to store some additional information.
  - For each state (t, k), in addition to the optimal value $v[t][k]$, we store $r[t][k]$, the production of period t in an optimal solution of the subinstance of state (t, k); for the states with value $+\infty$ we store 0.

– Starting from $(t, k) = (m, 0)$, the value $r[t][k]$ gives the production of period t; the inventory at the end of period $t-1$ is then $k + d_t - r[t][k]$, and the same rule can be applied again to period $t-1$, and so on, until $t = 0$.

This is the pseudo-code of the **DP algorithm using the bottom-up approach** with the storing of the information necessary to compute an optimal solution:

Algorithm

Algorithm: DP_CLS_InfoSol

Input: A CLS instance given by the number m of periods and the arrays $d[1..m]$, $sc[1..m]$, $vc[1..m]$, $hc[1..m]$, $cap[1..m]$

Output: The optimal value and the information matrix $r[0..m][0..D_{1,m}]$

```
1  Function DP_CLS_InfoSol(m, d, sc, vc, hc, cap)
2      R := new array[0..m]
3      R[m] ← 0
4      for t := m − 1 downto 0 do
5          R[t] ← R[t + 1] + d[t + 1]
6      end
7      v := new array[0..m][0..R[0]]
8      r := new array[0..m][0..R[0]]
9      v[0][0] ← 0
10     r[0][0] ← 0
11     for k := 1 to R[0] do
12         v[0][k] ← +∞
13         r[0][k] ← 0
14     end
15     for t := 1 to m do
16         for k := 0 to R[t] do
17             b := +∞
18             a := 0
19             for x := 0 to min{cap[t], k + d[t]} do
20                 if v[t − 1][k + d[t] − x] + f_t(x) + hc[t] · k < b then
21                     b ← v[t − 1][k + d[t] − x] + f_t(x) + hc[t] · k
22                     a ← x
23                 end
24             end
25             v[t][k] ← b
26             r[t][k] ← a
27         end
28     end
29     return v[m][0], r
```

∗ The DP algorithm uses the additional temporary variable a to keep track of the decision made when b is updated: if the production x is selected, $a \leftarrow x$ stores this decision, and at the end of the inner loop $r[t][k] \leftarrow a$ stores it, together with

the optimal value $v[t][k] \leftarrow b$. Because of the strict comparison, in case of ties the smallest production is stored.

Example 17.12: Computation of the additional information

We consider the CLS instance of Example 17.10:

t	1	2	3	4	5
d_t	2	1	3	4	1
sc_t	50	50	50	50	50
vc_t	0	0	0	0	0
hc_t	10	10	10	10	10
cap_t	4	4	4	4	4

For each step of the DP algorithm, the table shows the values of t and k, the cell $r[t][k]$, and the production x for which the minimum of the corresponding row of Example 17.11 is attained first.

t	k	$r[t][k]$	Action
0	0	0	$r[0][0] \leftarrow 0$
0	1	0	$r[0][1] \leftarrow 0$
0	2	0	$r[0][2] \leftarrow 0$
0	3	0	$r[0][3] \leftarrow 0$
0	4	0	$r[0][4] \leftarrow 0$
0	5	0	$r[0][5] \leftarrow 0$
0	6	0	$r[0][6] \leftarrow 0$
0	7	0	$r[0][7] \leftarrow 0$
0	8	0	$r[0][8] \leftarrow 0$
0	9	0	$r[0][9] \leftarrow 0$
0	10	0	$r[0][10] \leftarrow 0$
0	11	0	$r[0][11] \leftarrow 0$
1	0	2	first minimum at $x = 2$, i.e., $v[0][0]+f_1(2)+hc_1 \cdot 0 = 0+50+0 = 50$: $r[1][0] \leftarrow 2$
1	1	3	first minimum at $x = 3$, i.e., $v[0][0]+f_1(3)+hc_1 \cdot 1 = 0+50+10 = 60$: $r[1][1] \leftarrow 3$
1	2	4	first minimum at $x = 4$, i.e., $v[0][0]+f_1(4)+hc_1 \cdot 2 = 0+50+20 = 70$: $r[1][2] \leftarrow 4$
1	3	0	all candidates equal $+\infty$: $r[1][3] \leftarrow 0$
1	4	0	all candidates equal $+\infty$: $r[1][4] \leftarrow 0$
1	5	0	all candidates equal $+\infty$: $r[1][5] \leftarrow 0$
1	6	0	all candidates equal $+\infty$: $r[1][6] \leftarrow 0$
1	7	0	all candidates equal $+\infty$: $r[1][7] \leftarrow 0$
1	8	0	all candidates equal $+\infty$: $r[1][8] \leftarrow 0$
1	9	0	all candidates equal $+\infty$: $r[1][9] \leftarrow 0$
2	0	0	first minimum at $x = 0$, i.e., $v[1][1]+f_2(0)+hc_2 \cdot 0 = 60+0+0 = 60$: $r[2][0] \leftarrow 0$
2	1	0	first minimum at $x = 0$, i.e., $v[1][2]+f_2(0)+hc_2 \cdot 1 = 70+0+10 = 80$: $r[2][1] \leftarrow 0$
2	2	3	first minimum at $x = 3$, i.e., $v[1][0]+f_2(3)+hc_2 \cdot 2 = 50+50+20 = 120$: $r[2][2] \leftarrow 3$
2	3	4	first minimum at $x = 4$, i.e., $v[1][0]+f_2(4)+hc_2 \cdot 3 = 50+50+30 = 130$: $r[2][3] \leftarrow 4$

2	4	4	first minimum at $x = 4$, i.e., $v[1][1]+f_2(4)+hc_2 \cdot 4 = 60+50+40 = 150$: $r[2][4] \leftarrow 4$
2	5	4	first minimum at $x = 4$, i.e., $v[1][2]+f_2(4)+hc_2 \cdot 5 = 70+50+50 = 170$: $r[2][5] \leftarrow 4$
2	6	0	all candidates equal $+\infty$: $r[2][6] \leftarrow 0$
2	7	0	all candidates equal $+\infty$: $r[2][7] \leftarrow 0$
2	8	0	all candidates equal $+\infty$: $r[2][8] \leftarrow 0$
3	0	3	first minimum at $x = 3$, i.e., $v[2][0]+f_3(3)+hc_3 \cdot 0 = 60+50+0 = 110$: $r[3][0] \leftarrow 3$
3	1	4	first minimum at $x = 4$, i.e., $v[2][0]+f_3(4)+hc_3 \cdot 1 = 60+50+10 = 120$: $r[3][1] \leftarrow 4$
3	2	4	first minimum at $x = 4$, i.e., $v[2][1]+f_3(4)+hc_3 \cdot 2 = 80+50+20 = 150$: $r[3][2] \leftarrow 4$
3	3	4	first minimum at $x = 4$, i.e., $v[2][2]+f_3(4)+hc_3 \cdot 3 = 120+50+30 = 200$: $r[3][3] \leftarrow 4$
3	4	4	first minimum at $x = 4$, i.e., $v[2][3]+f_3(4)+hc_3 \cdot 4 = 130+50+40 = 220$: $r[3][4] \leftarrow 4$
3	5	4	first minimum at $x = 4$, i.e., $v[2][4]+f_3(4)+hc_3 \cdot 5 = 150+50+50 = 250$: $r[3][5] \leftarrow 4$
4	0	4	first minimum at $x = 4$, i.e., $v[3][0]+f_4(4)+hc_4 \cdot 0 = 110+50+0 = 160$: $r[4][0] \leftarrow 4$
4	1	4	first minimum at $x = 4$, i.e., $v[3][1]+f_4(4)+hc_4 \cdot 1 = 120+50+10 = 180$: $r[4][1] \leftarrow 4$
5	0	0	first minimum at $x = 0$, i.e., $v[4][1]+f_5(0)+hc_5 \cdot 0 = 180+0+0 = 180$: $r[5][0] \leftarrow 0$

This is the matrix r after the execution of the DP algorithm:

$r[t][k]$	0	1	2	3	4	5	6	7	8	9	10	11
$t = 0$	0	0	0	0	0	0	0	0	0	0	0	0
$t = 1$	2	3	4	0	0	0	0	0	0	0	·	·
$t = 2$	0	0	3	4	4	4	0	0	0	·	·	·
$t = 3$	3	4	4	4	4	4	·	·	·	·	·	·
$t = 4$	4	4	·	·	·	·	·	·	·	·	·	·
$t = 5$	0	·	·	·	·	·	·	·	·	·	·	·

– Once the matrix r has been computed, an optimal solution of the CLS instance can be computed by applying the following rule for $t = m, m-1, \ldots, 1$, starting from $k = 0$:

$$x_t \leftarrow r[t][k], \qquad k \leftarrow k + d_t - x_t.$$

This yields the vector $(x_1, x_2, \ldots, x_m)$, where each entry x_t is the number of units produced in period t in an optimal solution.

– The following pseudo-code implements the computation of an optimal solution, given the matrix r.

Algorithm

Algorithm: DP_CLS_SolComputation

Input: The number m of periods and the demands $d[1..m]$ of the CLS instance, and the matrix $r[0..m][0..D_{1,m}]$

Output: An optimal solution of the CLS instance

```
1 Function DP_CLS_SolComputation(m, d, r)
2     x := new array[1..m]
3     k := 0
4     for t := m downto 1 do
5         x[t] ← r[t][k]
6         k ← k + d[t] − x[t]
7     end
8     return x
```

Example 17.13: Computing an optimal solution

After the execution of the DP algorithm on the instance of Example 17.10, we have the matrix r of Example 17.12. We can compute an optimal solution as follows:

Iteration	t	k	$r[t][k]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$x[5]$	$k \leftarrow k + d[t] - x[t]$
–	–	–	–	0	0	0	0	0	$t = 5,\ k = 0$
1	5	0	0	0	0	0	0	0	$k = 0 + 1 - 0 = 1$
2	4	1	4	0	0	0	4	0	$k = 1 + 4 - 4 = 1$
3	3	1	4	0	0	4	4	0	$k = 1 + 3 - 4 = 0$
4	2	0	0	0	0	4	4	0	$k = 0 + 1 - 0 = 1$
5	1	1	3	3	0	4	4	0	$k = 1 + 2 - 3 = 0$

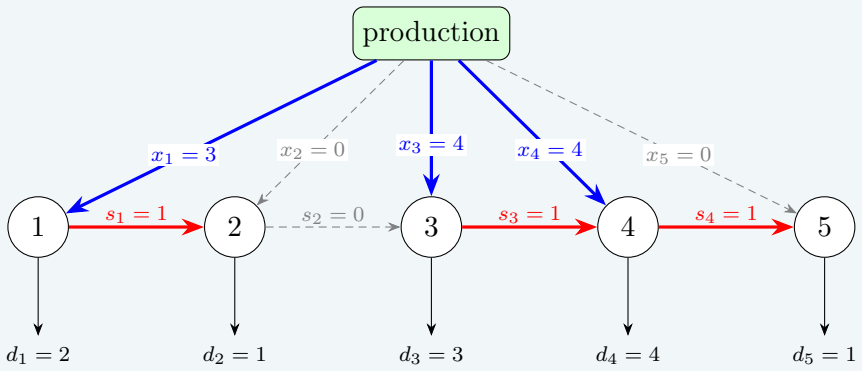
An optimal solution is the vector:

$$(3, 0, 4, 4, 0)$$

The setups are in periods 1, 3, 4, the inventories are $(s_1, \ldots, s_5) = (1, 0, 1, 1, 0)$, and the total cost is (there is no variable production cost)

$$\underbrace{50 + 50 + 50}_{\text{setup}} + \underbrace{10 \cdot 1 + 10 \cdot 0 + 10 \cdot 1 + 10 \cdot 1 + 10 \cdot 0}_{\text{holding}} = 150 + 30 = 180.$$

The corresponding optimal value is $v[5][0] = 180$, the same value given by Gurobi on the MIP model; the capacity costs $180 - 170$ with respect to Example 17.1. The network of the solution is:



Period 4 produces although one unit is in stock at the end of period 3: the solution does not have the zero-inventory ordering property (and it is the unique optimal solution with integer quantities).

17.3.1 Number of steps

- Let $S(m, D, C)$ denote the number of steps executed by DP_CLS in the worst case, where m is the number of periods, $D = D_{1,m}$ is the total demand, and $C = \max_{t \in T} cap_t$ is the largest capacity. For every $t \geq 1$ we have $R[t] + 1 \leq D$ (since $d_1 \geq 1$), and the inner loop has at most $C + 1$ iterations. In the worst case, the condition at line 16 is always true. The number of executions of the lines is:
  - Line 2, Line 3, Line 7, and Line 8 are executed 1 time each.
  - Line 4 and Line 5 are executed m times each.
  - Line 9 and Line 10 are executed D times each.
  - Line 12 is executed m times.
  - Line 13, Line 14, and Line 20 are executed $\sum_{t=1}^{m}(R[t] + 1) \leq mD$ times each.
  - Line 15, Line 16, and Line 17 are executed at most $mD(C + 1)$ times each.
  - Line 23 is executed 1 time.

  Therefore,

$$S(m, D, C) \leq 5 + 3m + 2D + 3mD + 3mD(C + 1) = 5 + 3m + 2D + 6mD + 3mDC,$$

  and hence

$$S(m, D, C) = O(mDC).$$

The DP algorithm executes

$$S(m, D, C) = O(m\,D\,C)$$

steps in the worst case. The number of steps depends on the values D and C, not only on the number m of periods: the DP algorithm is **pseudo-polynomial**.

- DP_CLS_InfoSol only adds a constant number of operations inside the same loops, and therefore its number of steps has the same order of growth.

- *Example.* The capacitated instance of the MIP heuristics notes has 6 periods, demands $(50, 30, 10, 10, 40, 10)$, setup cost 40, holding cost 1, and capacity 80. Measured in units of 10, it has 35 states with $t \geq 1$.

- The DP algorithm gives the optimal value 160, with the production plan $(50, 50, 0, 0, 50, 0)$ (in the original units): the same value proved optimal by the MIP solver.

17.3.2 The case of constant capacity

- With general capacities cap_t the CLS is NP-hard, so a polynomial algorithm is not expected and the pseudo-polynomial DP algorithm is a natural exact method for small values of D and C.

- When the capacity is the same in all periods, $cap_t = C$ for all $t \in T$, the CLS can be solved in polynomial time, because the zero-inventory ordering property is replaced by a weaker one:

If $cap_t = C$ for all $t \in T$, there is an optimal solution in which, between two consecutive periods with empty inventory, at most one period produces a quantity strictly between 0 and C; all the other periods produce 0 or C.

- We do not develop the polynomial algorithms based on this property here.

- Example 17.10 has constant capacity $C = 4$. Its optimal solution $(3, 0, 4, 4, 0)$ has inventories $(1, 0, 1, 1, 0)$, empty at the end of periods 0, 2, and 5.

- In periods 1–2 only period 1 produces a quantity strictly between 0 and 4, and in periods 3–5 every period produces 0 or 4, as stated by the property.

17.4 Typical mistakes

Warning

- **Applying the zero-inventory ordering property with capacities.** With capacities a period can produce while units are in stock: in Example 2 the unique optimal plan violates the property.

- **Computing the cost of a run from scratch.** Computing every $c_{j,t}$ from its definition costs $O(m^3)$ steps inside the DP algorithm; the incremental computation of CostMatrix keeps $\Theta(m^2)$.

- **Forgetting the empty final inventory.** The subinstance with periods $\{1, \ldots, t\}$ ends with an empty inventory: this is why the run in period j produces exactly $D_{j,t}$ units.

- **Reading one optimal plan as the only one.** Example 1 has two optimal plans:

the DP algorithm returns the one with the smallest period j, because it updates only on a strictly smaller value.

- **Calling $O(mDC)$ polynomial.** The running time contains the total demand D and the largest capacity C, which are numbers of the input: the DP algorithm is pseudo-polynomial.

17.5 In summary

Summary

- If all costs are non-negative, the uncapacitated problem has an optimal plan in which a period produces only when the inventory is zero (Wagner and Whitin), so $x_j = D_{j,t}$ for every production run.
- The cost $c_{j,t}$ of every run is computed in $\Theta(m^2)$ steps, and the recurrence $v(t) = \min_j\{v(j-1) + c_{j,t}\}$ gives the optimal value in $\Theta(m^2)$ steps; it is a shortest path.
- With capacities the property fails: the state is the pair (period, final inventory), and the DP algorithm takes $O(mDC)$ steps, which is pseudo-polynomial.
- With a constant capacity a weaker property holds, and the problem is polynomial; with general capacities it is NP-hard.

Dashboard of the lot-sizing examples.

Instance	Lower bound	Upper bound	Origin
Example 1 (uncapacitated, $m = 5$)	170	170	DP algorithm of Wagner and Whitin
Example 2 (capacitated, $C = 4$)	180	180	DP algorithm on (period, inventory)